

Three Lessons from Evaluating Language Models on Reasoning Tasks

By

Alexander F. Gu

S.B., Massachusetts Institute of Technology (2021).

M.Eng., Massachusetts Institute of Technology (2022).

Submitted to the Department of Electrical Engineering and Computer Science
in Partial Fulfillment of the Requirements for the Degree of

DOCTOR OF PHILOSOPHY

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

September 2026

© 2026 Alexander F. Gu. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Alexander F. Gu
Department of Electrical Engineering and Computer Science
August 7, 2026

Certified by: Armando Solar-Lezama
Distinguished Professor of Computing
Thesis Supervisor

Accepted by: Leslie A. Kolodziejski
Professor of Electrical Engineering and Computer Science
Chair, Department Committee on Graduate Students

THESIS COMMITTEE

THESIS SUPERVISOR

Armando Solar-Lezama

*Distinguished Professor of Computing
MIT Schwarzman College of Computing*

THESIS READERS

Kaiyu Yang

*Research Lead, Verifiable AI Team
Apodex*

Jacob Andreas

*Associate Professor
MIT EECS and CSAIL*

Three Lessons from Evaluating Language Models on Reasoning Tasks

by

Alexander F. Gu

Submitted to the Department of Electrical Engineering and Computer Science
on August 7, 2026 in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

ABSTRACT

An evaluation is a lens through which we view the capabilities of a model. As language models become increasingly capable at programming, mathematics, and logical reasoning, the design of this lens becomes central to scientific progress: a benchmark can reveal a capability that was previously hidden, but it can also distort our understanding when it measures the wrong behavior or contains confounding biases.

This thesis studies three lessons from evaluating language models on reasoning tasks. First, focusing on the right metrics without bias is crucial. For code and mathematical reasoning, final correctness alone can miss important distinctions: a model may pass tests while producing brittle or low-quality code, solve an inequality while giving an invalid proof, or appear strong because an evaluation contains trivial negatives or contaminated examples. Second, looking beyond the score reveals hidden nuances. Deeper analysis can surface structure that aggregate performance conceals, such as differences between code generation and code execution, gains from distillation that do not reflect deeper understanding, and complementary failure modes between chain-of-thought prompting and neurosymbolic reasoning. Third, what we measure shapes what we build. Once evaluations become standard, they define what counts as progress and influence the capabilities that future systems are optimized to demonstrate.

Together, these lessons argue that evaluation should be treated not as a secondary measurement step, but as a central part of building and understanding AI systems for code and reasoning.

Thesis supervisor: Armando Solar-Lezama

Title: Distinguished Professor of Computing

Acknowledgments

I feel incredibly lucky to have Armando as my advisor throughout my PhD. Armando always made sure I was enjoying myself, giving me the rare combination of complete freedom yet continual support. I never felt shy going to him about anything on my mind, whether it was about research papers, career decisions, or the most fun adventures to have at a conference destination. Armando's advice has always been honest yet encouraging. Early in my PhD, I asked Armando how to read papers. He didn't answer the question, but the advice he gave me was more useful than any direct answer he could have given: before reading any related work, I should sit with the problem myself for a while so I can have fresh ideas before being polluted. This advice shaped the way I did research moving forward: to develop my own taste and way of approaching problems instead of imitating what others had done. Since then, Armando has taught me much more: how to pick the right problems, how to develop ideas, and how to present my work. More than anything, he taught me how to trust myself and think differently. It's all because of Armando that these years have been as intellectually exciting as they have been enjoyable.

I am also very fortunate to have Jacob and Kaiyu on my committee. Through co-advising my MEng project, Jacob taught me a lot about both NLP and research more broadly. During meetings, he always asked me what the core question I wanted to answer was, beyond just have a cool demo or improving some benchmark numbers. This pushed me to always think deeper about fundamental research questions, something I always thought about in the years that followed.

Kaiyu introduced me to theorem proving, which later became the focus of my latter half of the PhD. Having worked and believed in the field long before it became mainstream, he showed me what it meant to continuously pursue a vision while embracing the changes of the time. Beyond research, Kaiyu has been a constant source of support, and he always gave me earnest advice, helping me navigate all kinds of situations during my PhD.

My brilliant mentors and friends at Meta FAIR helped me grow in all kinds of ways. Gabriel and Baptiste opened the doors to the AI CodeGen world. Sida encouraged me to always think creatively yet remain disciplined. Aram showed me what it means to do anything you can to unblock any obstacle in the way of your goals. I also grew a lot through all the other collaborators, mentors, and new friends I met at Meta, through research and life adventures alike: Amaury, Bartosz, Chris, Dmitrii, Edouard, Fabian, Francois, Gautier, Hugh, Kunhao, Luyu, Scott, Volker, Weijia, Yuxiang, Zacharias, and so many more.

Although I knew very little about ML systems at the time, the AMMO team at NVIDIA gave me the opportunity to grow and learn about the field. My mentor Huizi is one of the most ambitious people I know, and his ability to execute is just as impressive. Through him,

I learned as much about ML systems as I learned about how to build and grow a startup. I was also fortunate to have Di, Frida, Kai, and Lucas as wonderful colleagues during my summer there.

Math Inc was where I began my first full-time job, and I want to thank everyone there, past and present. At Math Inc, I have been fortunate to work in an environment defined by ambitious people and a shared conviction in even the most moonshot ideas. Thank you for the opportunity to learn from all of you, and for pushing me to think bigger about what is possible.

I had a wonderful experience working at Amazon AWS with the Automated Reasoning Group. I'm lucky to have had Hongzhou, Soonho, and Leo as mentors, humble yet extremely talented. Through my time there, they showed me all the impactful ways formal methods were being used outside of an academic setting. I'm also grateful for all the other friends and colleagues I met there: Hongshan, Zhizhen, Brando, Andrew, Chenyang, Zhixing, Joseph, and Indraneel, all of whom are extremely passionate about AI for math and formal verification.

I would also like to thank all the mentors who took a chance on me in my naive undergraduate years. My very first research mentors at MIT, Costis and Dylan, introduced me to multi-agent RL, and I especially thank them for their patience when it was clear that they could have finished what I had done in under half the time. Lily and Luca guided me, pointed me to interesting directions in ML robustness, gave me more and more freedom, and constantly supported me with research advice.

In my third year of undergrad, I joined the OptiML group. I will never forget the first problem Suvrit fantasized that I would help him solve: proving a better bound of Polyak-Lojasiewicz functions. While I never achieved the final goal, he taught me how to persevere and progress through the hardest problems. I'm also glad to have had the opportunity to learn from the rest of the group: Alp, Chulhee, Hadi, Horia, Jingzhao, Josh, Kwangjun, Tiancheng, Xiang, and Yi. Everyone's passion for research and optimization was inspiring: group meetings were always a highlight of the week, and there was never an abundance of fun research problems to work on.

I thank Hima for introducing me to the world of explainability research, for giving me so much advice on my course project for almost a year after the class had ended, and for teaching me how to be a creative researcher.

I thank my MIT-IBM mentors Songtao and Parikshit for teaching me a lot about bilevel optimization, helping me grind through some of the nastiest proofs I've ever written in a paper, and repeatedly giving me hope when I felt like I was hitting a wall.

One of the best parts of graduate school is all the brilliant people I got to meet and learn from. First, I believe I am part of one of the happiest research groups at MIT. Everyone's interests are so diverse, and I've learned so many cool things I'd never even thought I wanted to learn: biology, robotics, cognitive science, programming languages, systems, and more. You all have brought much joy to my research experience: Ivan, Jimmy, Kavi, Kliment, Leo, Maddy, Megan, Michael, Nathan, Omar, Ramya, Ria, and Zed. Thanks to Jack for mentoring me during my MEng and teaching me a lot about databases and to Theo for being someone I could always talk to in the lab about AI for code ideas. Finally, I'm lucky to have had the opportunity to mentor so many talented undergraduate and masters students: Clarise, Daniela, Ethan, Gilford, Haijia, Ivy, Jacob, Tamara, Tianyi, and Yeabsira.

Despite living hundreds of miles away when I started my PhD, Naman and Wen-Ding felt

like second labmates to me. We always brainstormed the craziest ideas, updated each other on the latest, wrote many papers, and went on escapades during conferences and beyond. In early 2024, I learned that ACL 2024 was in Thailand, the perfect place to propose to my then-girlfriend. I was determined to make it happen. In under a month and a half, Naman, Wen-Ding, Theo and Celine helped me put together and submit a paper, and to this day I am eternally grateful. Throughout my PhD, the AI4Code community grew larger and larger, and I am fortunate to have many other friends to brainstorm and chat about code-related ideas (and struggles) from time to time: Dylan, Jenny, Kexun, Manish, Wenting, Zhaoyu, and Zora.

My many talented co-authors, collaborators, colleagues, and friends gave me the chance to think about and work on on a variety of diverse projects, some: Alexis, Antoine, Ben, Ced, Chanwoo, Diganta, Don, Gabe, Hung, Jiayi, Kaifeng, Pan, Pin-Yu, Prerna, Rayan, Roger, Saiteja, Samy, Satyapriya, Shahin, Shannon, Sirui, Terry, Tessa, Yijia, and Ziyang. You have all taught me so much, and I always look forward to being inspired by your ideas when we meet.

I thank my family, who was there long before I was, for constantly reminding me to chase my dreams. And I thank my partner and love for always giving me the confidence and support that we can achieve anything together.

Finally, I dedicate this thesis to my mom, who passed away from breast cancer early in my PhD. Ever since I was young, my mom always went out of her way to find me opportunities, supporting all the pursuits and passions I could ever have dreamed of. As I grew up, she was always my primary source of support, whether I had questions on my homework or came back from school feeling depressed.

In her final years, during her battle with cancer, my mom was repeatedly told that her life was nearing its end. Despite her own struggles and limited ability to take care of herself, she never stopped prioritizing my success. When I was applying to grad school, she spent all day and night researching professors and reporting back to me. When I started my PhD in AI for code, a field she knew absolutely nothing about, she did not stop, always sending me articles or even papers on the latest news.

I know that my mom would have loved to see me complete this thesis. Although that cherished dream can no longer come true in the way we had hoped, I carry her with me in memory and in everything this milestone represents. I will never forget the sacrifices she made for me, the love with which she supported me, and all the wisdom she has imparted on me since day one.

Contents

<i>Acknowledgments</i>	7
<i>List of Figures</i>	15
<i>List of Tables</i>	19
1 Introduction	21
1.1 Lesson 1: Focusing on the Right Metrics Without Bias Is Crucial	22
1.2 Lesson 2: Looking Beyond the Score Reveals Hidden Nuances	24
1.3 Lesson 3: What We Measure Shapes What We Build	25
1.4 Discussion	26
2 Focusing on the Right Metrics Without Bias Is Crucial	27
2.1 CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution	28
2.1.1 Benchmark Construction	29
2.1.2 Evaluation	31
2.1.3 Impact of Anonymizing Functions	32
2.1.4 Discussion	33
2.2 SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks	33
2.2.1 Measuring Code Slop	35
2.2.2 Measuring Correctness and Cost Over Time	35
2.2.3 Measuring Code Quality Over Time	37
2.2.4 Calibration Against Human Repository Histories	38
2.2.5 Discussion	38
2.3 IneqMath: Solving Inequality Proofs with Large Language Models	39
2.3.1 Task Setup, Dataset, and Metrics	39
2.3.2 The Soundness Gap	40
2.3.3 Discussion	43
2.4 The Counterfeit Conundrum: Can Code Language Models Grasp the Nuances of Their Incorrect Generations?	43
2.4.1 Experimental Setup	44
2.4.2 Can code language models understand counterfeit samples?	46
2.4.3 Discussion	49
2.5 LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code	50
2.5.1 Benchmark Construction	50

2.5.2	Effects of Contamination	50
2.5.3	Holistic Evaluation	53
2.5.4	Discussion	55
2.6	Conclusion	55
3	Looking Beyond the Score Reveals Hidden Nuances	57
3.1	CRUXEval: Generalization, Chain of Thought, Fine-Tuning, and Qualitative Failures	57
3.1.1	Quantitative Analysis	57
3.1.2	Chain of Thought Prompting	60
3.1.3	Fine-tuning Experiments	62
3.1.4	Qualitative Analysis	63
3.1.5	Discussion	65
3.2	Counterfeit Conundrum: The Nature of Counterfeit Samples	65
3.2.1	Do counterfeit samples from different models or problems differ significantly?	65
3.2.2	Qualitative Analysis	69
3.2.3	Discussion	71
3.3	LiveCodeBench: Overfitting, Post-Training, and Open vs. Closed Models	72
3.3.1	Overfitting to HumanEval+	72
3.3.2	Role of Post-Training	73
3.3.3	Gap Between Closed and Open Models	73
3.3.4	Discussion	74
3.4	LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers	74
3.4.1	LINC: Logical Inference via Neurosymbolic Computation	75
3.4.2	Experiments	76
3.4.3	Results & Discussion	77
3.4.4	Error Analysis	79
3.4.5	Discussion	83
3.5	Discussion	83
4	What We Measure Shapes What We Build	85
4.1	CRUXEval	85
4.1.1	Inspiring a Family of Code Reasoning Benchmarks	85
4.1.2	Explicitly Improving Code Execution	86
4.1.3	Adoption by Frontier Model Developers	86
4.2	LiveCodeBench	86
4.2.1	Contamination-Aware Benchmarks	87
4.2.2	Adoption by Frontier Model Developers	87
4.3	LeanDojo: Theorem Proving with Retrieval-Augmented Language Models	88
4.3.1	LeanDojo Benchmark	88
4.3.2	ReProver: A Retrieval-Augmented Model for Lean	90
4.3.3	Results	91
4.3.4	Opening Up Future Theorem-Proving Research	91

4.4	ProofOptimizer: Training Language Models to Simplify Proofs without Human Demonstrations	92
4.4.1	Proof Simplification: Task and Metrics	93
4.4.2	ProofOptimizer: LLMs for Proof Simplification	94
4.4.3	Results	95
4.4.4	Additional Benefits of Proof Simplification	97
4.4.5	Conclusion	99
4.5	Discussion	99
5	Future Research and Remaining Challenges Toward AI for Software Engineering	101
5.1	Introduction	101
5.2	Tasks in AI Software Engineering	101
5.2.1	Formal Verification	102
5.3	Challenges	103
5.3.1	Long-Horizon Code Planning	103
5.3.2	High Logical Complexity and OOD Domains	104
5.4	Paths Forward	105
5.4.1	Inference-Time Approaches	105
5.5	Discussion	107
6	Conclusion	109
	<i>Bibliography</i>	111

List of Figures

2.1	Difference between model pairs on bootstrap samples of various sizes. The whiskers show (2.5, 97.5) and boxes show (25, 75) percentiles.	31
2.2	Main Results. Boxes show (25, 75) percentiles, whiskers show (2.5, 97.5), and the middle bar shows the median ($\approx$ mean).	32
2.3	Iterative evaluation in SlopCodeBench. A task unfolds over checkpoints. At each step, the agent extends its prior solution to satisfy an updated specification. Correctness, erosion, and verbosity are scored externally and withheld from the agent.	34
2.4	SlopCodeBench shows that correctness degrades as iterative problems progress while cost per checkpoint rises. The left panel separates strict, isolated, and core solve rates; the right panel shows mean cost per checkpoint over the trajectory.	37
2.5	Structural erosion and verbosity increase as agents progress through SlopCodeBench tasks. Iterative evaluation makes this accumulation visible because quality is measured at every checkpoint, not only on the final artifact.	37
2.6	Agent trajectories have higher verbosity and erosion than sampled human repository histories, and they degrade faster over normalized trajectory progress.	38
2.7	IneqMath evaluates both the final answer and the reasoning trace. A solution is counted as fully correct only when it passes the final-answer judge and all step-wise soundness judges.	40
2.8	IneqMath highlights the gap between final-answer accuracy and full reasoning soundness. The chapter-level lesson is that final-answer success is not enough evidence of sound mathematical reasoning.	42
2.9	Scaling improves final-answer accuracy more reliably than it improves full reasoning soundness.	42
2.10	Scaling law in test-time computation for reasoning LLMs.	43
2.11	Example of a problem specification with incorrect, counterfeit, and correct programs.	44
2.12	Distribution of correct, counterfeit, and incorrect samples by model, benchmark, and problem difficulty, showing their prevalence across all settings.	45
2.13	Models other than GPT-4 struggle to classify samples as correct or counterfeit and are much better at assessing the correctness of correct samples than counterfeit samples.	47
2.14	Models are much better at executing correct samples than counterfeit samples, and even often execute counterfeit samples as if they were correct.	48

2.15	In the absence of execution information, repair underperformed resampling in almost all settings. Samples above the $y = x$ resampling baseline have been coloured in blue for clarity. Vertical lines indicate 95% confidence intervals over repair samples.	49
2.16	Contamination across code generation, test-output prediction, self-repair, and code execution scenarios over time.	51
2.17	Performance on problems released over different months for ATCODER. . . .	52
2.18	Live evaluation over time for various models on code generation in LIVECODEBENCH.	52
2.19	Overview of the four scenarios present in LIVECODEBENCH.	53
2.20	Scenario-level performance profiles in LIVECODEBENCH.	54
2.21	Model performances across the four scenarios available in LIVECODEBENCH. .	55
3.1	Correlation between HumanEval pass@1 scores and CRUXEVAL-O pass@1 scores	58
3.2	Correlation between Input and Output Prediction Scores, with and without CoT	59
3.3	Correlation between predictions on input (left) and output (right)	60
3.4	pass@5 score vs. pass@1 score with and without CoT	61
3.5	Confusion Matrix of Direct Prediction vs. CoT Prediction ($T = 0.2$)	62
3.6	Improvements and Limits of CRUXEVAL Performance after Fine-Tuning . .	63
3.7	Mean accuracy of correctness checking, execution, and repair on HumanEval (error bars are 1 std). A problem’s difficulty is determined by the pass@1 of the evaluation model. In general, correctness checking accuracy is uncorrelated with problem difficulty, while both execution and repair accuracies are only very weakly correlated.	66
3.8	Correlation between generation and each of our three tasks	67
3.9	Heatmap of accuracies for correctness checking and execution prediction. Models do not seem to perceive their own generations differently from those of other models.	68
3.10	Accuracy of correctness and execution tasks on counterfeit samples. Both strong and weak models are able to generate difficult counterfeit samples. . .	68
3.11	Scatter plot comparing PASS@1 of models on HUMANEVAL+ versus LCB-Easy (time-window Sep 2023 to May 2024). The green-shaded region marks models with aligned performance across benchmarks, while the red-shaded region marks models with much stronger performance on HUMANEVAL+.	72
3.12	Comparison of open-access and closed-access models on the LIVECODEBENCH-Easy code-generation scenario.	74
3.13	Overview of the LINC approach. Starting from a problem in natural language, the language model first samples first-order logic translations of the premises and conclusion. The translated formulas are then passed to an automated theorem prover, which filters invalid syntax and predicts labels for the valid samples. Finally, a majority-vote step aggregates the remaining candidate labels.	75

3.14	This figure outlines the string concatenation workflow for each of our conditions. We start with the original problem, provide ICL examples through an intermediate markup language, and finally append the problem to evaluate. At this stage, we allow the model to autoregressively sample until producing a stop token.	77
3.15	Results of each model on the FOLIO and ProofWriter datasets. Accuracies are for bootstrapped 10-way majority vote for all models. Error bars are ± 1 bootstrapped standard deviation. Dotted, black line is the accuracy obtained by always guessing the most common label in the dataset.	78
3.16	Accuracy per necessary proof depth in ProofWriter. Accuracies reported are for bootstrapped 10-way majority vote, and shaded areas cover ± 1 bootstrapped standard deviation. Black, dotted lines reflect the expected success rate of guessing a random label, which is $1/3$ in all subsets per our experiment design.	79
3.17	Analyzing and comparing the mistakes made by GPT-4 on the FOLIO dataset.	80
4.1	LeanDojo connects data extraction, proof-state interaction, premise retrieval, and tactic generation into a reproducible theorem-proving environment. . . .	89
4.2	Lean libraries contain clusters of similar theorems and proofs. Benchmark splits must account for this or they risk measuring memorization.	90
4.3	Overview of our pipeline. ProofOptimizer reduces the shortest generated proof of a Putnam problem from 1097 to 76 tokens.	93
4.4	Iterative Shortening: per-iteration improvement (left) and effect of proof length (right)	97
4.5	Training loss (left) and miniF2F score (right) after SFT on simplified vs. original proofs.	98
4.6	Simplified proofs are frequently faster than original proofs on miniF2F and PutnamBench.	99

List of Tables

2.1	Impact of Anonymization on CRUXEVAL	33
2.2	Per-model performance on SlopCodeBench. For each model we report its best just-solve run, ranked by isolated solve rate with core, strict, and \$/checkpoint as tiebreakers. Solve rates use a fixed 196-checkpoint denominator; missing checkpoints count as unsolved. Other metrics are only for ran checkpoints. Bold marks the best value per column.	36
2.3	Verbosity and erosion at HEAD commit, by GitHub-star tier and against agent checkpoints.	38
2.4	Evaluation performance of chat and reasoning LLMs on the INEQMATH benchmark (the test set). <i>Bnd.</i> denotes bound problems and <i>Rel.</i> denotes relation ones. We report: (1) <i>Overall Acc</i> , which reflects the correctness of both the final answer and intermediate steps; (2) <i>Answer Acc</i> , which measures final answer correctness alone; and (3) <i>Step Acc</i> , which evaluates the accuracy of intermediate steps across four error categories— <i>Toy Case</i> , <i>Logical Gap</i> , <i>Numerical Approximation</i> , and <i>Numerical Computation</i> . Blue superscripts $\downarrow$ indicate accuracy drop (<i>Overall Acc</i> - <i>Answer Acc</i>) from step-wise errors. <u>Underlining</u> denotes best result within each model category; boldface highlights best overall performance. Default max token limit for reasoning LLMs is 10K.	41
4.1	Theorem proving Pass@1 (%) on the LeanDojo Benchmark test sets. ReProver outperforms tidy, GPT-4, and an ablation that generates tactics without retrieval. All methods degrade substantially on the novel_premises split. .	91
4.2	Min@k and Red@k throughout expert iteration and online RL. Our RL model has strong @1 results, while our ExpIt model has strong @32 results. RL metrics are Gaussian-smoothed.	96
4.3	Iterative shortening achieves significant reduction for Seed-Prover’s IMO 2025 proofs.	97

Chapter 1

Introduction

In June 2020, OpenAI released GPT-3 [1], a language model that felt strong for its time but was still unreliable for many knowledge-based tasks. In the two years that followed, models showed progress through systems like Codex [2], Copilot [3], and InstructGPT [4], but they remained brittle for programming, mathematics, and deeper reasoning tasks. In November 2022, however, a breakthrough seemed to occur. ChatGPT was released, and for the first time an AI system felt like a generally usable assistant. Just four months later, in March 2023, GPT-4 [5] was made available, bringing a large jump in reliability and performance.

Suddenly, the pace of progress made it seem like models could do anything, faster than our benchmarks could keep up. By the time GPT-4 was released, the community had built a substantial evaluation ecosystem: MMLU [6] tested broad academic knowledge, BIG-Bench [7] and BIG-Bench Hard [8] collected diverse challenging tasks, and HELM [9] organized many evaluations into a more systematic comparison framework. Other benchmarks targeted narrower capabilities that had long been difficult for language models, from commonsense and science reasoning in WinoGrande [10], HellaSwag [11], and ARC Challenge [12], to grade-school mathematical reasoning in GSM8K [13], to code generation in HumanEval [2], MBPP [14], and APPS [15]. Many of these evaluations had been designed as long-term challenges and stood for years, yet were now starting to saturate. On MMLU, GPT-4 reached 86%, approaching the $\sim 90\%$ accuracy of human domain experts and nearly doubling the original GPT-3’s 44%. It exceeded 85% on GSM8K and WinoGrande and surpassed 95% on HellaSwag and ARC Challenge. The same pattern held for code: on HumanEval, a popular coding benchmark, GPT-4 scored 67%, up from GPT-3.5’s 48% and the original Codex’s 29%.

At the same time, the models seemed to be able to do much more than what the benchmarks had sought to measure. One example of this was the “Sparks of AGI” report [16], which set aside standardized benchmark-based evaluation in favor of impressive qualitative demonstrations of GPT-4: drawing a TikZ unicorn, writing a poem-based proof that there are infinitely many primes, coding a functioning game from scratch, and displaying theory of mind. For a while thereafter, the best way to understand these models was not to rely on academic benchmarks, but rather to browse community demonstrations of what could be accomplished.

The perception in the academic community therefore shifted. Previously, each individual research community (NLP, coding, vision) developed models more or less with open recipes,

guided by benchmarks that served as a north star for progress. GPT-4 broke this status quo. It was the strongest system across all of these domains at once, yet almost nobody outside of OpenAI understood how it had been trained. With the strongest systems now closed and hard to understand from the outside, what we needed from an evaluation was itself changing: beyond serving as a guide to scientific progress, evaluations increasingly had to help us make sense of behaviors and quirks we could only observe externally. More broadly, it was a ripe time to rethink the role of evaluations.

Ultimately, an evaluation needs to help model builders and scientists understand the capabilities and limitations of a model. The challenge is to design evaluations that identify meaningful shortcomings, advance our scientific understanding of the current state of the art, and suggest avenues for moving forward as a field. In this thesis, I will share three lessons from my experience designing evaluations for language models in the several years that followed GPT-4, particularly for coding, mathematics, and logical reasoning.

1.1 Lesson 1: Focusing on the Right Metrics Without Bias Is Crucial

The first lesson is that focusing on the right metrics without bias is crucial. In science, almost anything can be measured, but not everything is meaningful. The first question we face when designing any evaluation is what we should measure. The second is whether the resulting measurement is biased by confounding factors in the data or evaluation setup. In this thesis, I use the term *bias* in an evaluation design sense: a systematic distortion that makes a score reflect something other than the capability the evaluation is intended to measure. Notably, this is distinct from statistical bias or social bias. Here, the biases arise from confounding factors in the data, task, or metric design, such as data contamination.

When humans learn programming, they not only learn to write code, but also gain the ability to read code, understand it, simulate its execution behavior, debug it, and develop a suite of other programming-related skills. In late 2023, language models like GPT-4 were just starting to become proficient at writing Python code. These models learned their coding abilities by training on trillions of tokens of GitHub code. While humans show a correlation between various programming-related capabilities, it was unclear whether this was the same for models: if they could generalize to any capabilities beyond code generation. Motivated by this, we designed CRUXEVAL [17], a benchmark asking language models to simulate the forward and reverse execution of very simple Python programs consisting of 3-13 lines with no complex control flow or calculations.

When we released CRUXEVAL, many people in the community were skeptical about whether CRUXEVAL was even a useful metric to measure. After all, a simple way to ace the forward execution subset of CRUXEVAL is to simply run the code in a Python interpreter. However, this metric was useful for several reasons. First, language models that can effectively program should have a good understanding and mental model of code. While function-level single-shot code generation benchmarks like HumanEval used to be the standard for coding evaluations, real-world programming requires a much wider set of skills. Second, the benchmark allows us to scientifically understand new phenomena of coding

language models. For example, by showing that language models did have some ability to perform both forward and reverse execution, we saw early signs that language models did have some degree of generalization despite just training on raw code tokens. Overall, CRUXEval was a successful metric because it was just on the boundary of what frontier models could do, was simple enough for humans that we would expect models to be able to solve it, and taught the community something via comparing it with existing benchmark.

IneqMath [18] is another example of focusing on the right metric. In IneqMath, we introduced a benchmark for evaluating the ability of language models to solve inequality problems. When we measured the accuracy of Grok 3 mini [19], we saw that it was able to get the answer correct 72% of the time, indicating a high level of proficiency at solving inequalities. However, upon closer examination, when changing the metric from evaluating at the final answer to evaluating the entire reasoning trace and solution, we found that the accuracy dropped to 6%, a stark difference that leads to an entirely different conclusion. By focusing on evaluating full reasoning traces instead of just the final answer (common in prior mathematical reasoning evaluations), we uncover that models frequently guess the final answer correctly while hallucinating incorrect proofs.

After choosing the right metric, we must also account for potential biases in our evaluation design before arriving at conclusions. One example of this is our LiveCodeBench evaluation [20]. Prior to LiveCodeBench, coding evaluations primarily relied on static benchmarks like HumanEval, fixed datasets that were released once and never updated. However, this fails to account for data contamination: language models are often trained on data from the internet, which could potentially include these benchmarks. While many models are trained with best-effort decontamination processes, it is not possible to completely remove all traces of these benchmarks from the training set. Using time-controlled evaluations, we eliminate this bias. As evidence of this, DeepSeek-Instruct-33B [21] has a published data cutoff date of September 2023. When evaluated on problems between May and August 2023, it had an average accuracy of around 50%. However, when evaluated on problems between September 2023 and February 2024, the average accuracy dropped to around 20%, significantly lower. Similarly, the accuracy of DeepSeek-Base-33B (same cutoff date) plummeted from a pass@1 of 60% on May 2023 LeetCode problems to around 0% on September 2023 problems.

These examples preview the main idea of Chapter 2, but they are not exhaustive. The chapter will also discuss how iterative code-quality metrics in SlopCodeBench [22] and counterfeit-focused metrics in the Counterfeit Conundrum [23] expose failures that standard accuracy would obscure. Together, these case studies show that the right metric and the right controls can change the conclusion one draws from an evaluation. Once the metric exposes a gap, the next step is to analyze the behavior that produced it. Through in-depth analysis, we can often discover phenomena that are not obvious from the final scores alone, leading to the next lesson.

1.2 Lesson 2: Looking Beyond the Score Reveals Hidden Nuances

Often, the most interesting scientific insights only appear after looking beyond surface-level metrics with deeper analysis. In CRUXEVAL, the execution metric already revealed that models could perform some execution; deeper analysis revealed more about how different coding capabilities relate to one another. The most striking finding came when we looked at the relationship between HumanEval scores and CRUXEVAL scores across a suite of models. While the forward execution accuracy was highly correlated with the reverse execution accuracy, the HumanEval score showed a different trend. For base models, HumanEval and CRUXEVAL performance were correlated. However, distilled models (WizardCoder, Phind, Phi) broke this trend, significantly beating their base models at HumanEval but not CRUXEVAL. This reveals a key limit of fine-tuning: while distillation on NL-to-code samples can greatly boost performance on code generation, it does not necessarily improve a model’s underlying understanding of code.

Another illustration of this is when we developed LINC [24], a neurosymbolic method for logical reasoning. The task is that, given a set of natural language premises and a conclusion, one must determine if the conclusion is true, false, or unknown based on the premises. The standard method of doing this was via chain-of-thought (CoT) prompting, having a language model reason through the premises to make a deduction about the validity of the conclusion. Our method, LINC, first formalized everything into a first-order logic program and then executed a first-order logic prover to determine the answer. In most evaluations, LINC outperformed CoT, which might lead one to believe it is a superior method for performing logical reasoning. However, in one case, on the FOLIO benchmark [25] using GPT-4 as the base model, this was not the case: LINC only scored 73% vs. CoT’s 75%.

After this result, we conducted a series of analyses to understand why LINC had underperformed in this specific case. First, we did a qualitative analysis of the errors of both approaches, and discovered they were very different. Errors in LINC occurred when the models failed to properly capture implicit and explicit information that was in the natural language statements (failure modes impossible in CoT), while errors in CoT occurred from making incorrect natural language logical deductions and failing to find the right reasoning chain (failure modes impossible in LINC). Upon further quantitative analysis, we found that LINC had worse recall but better precision on True/False predictions, and that LINC and CoT mispredicted on different examples. These nuances were not apparent from looking at the performance scores and only came to light after this deeper analysis.

Chapter 3 expands this lesson beyond these two examples. We return to the Counterfeit Conundrum to understand what kinds of wrong programs models find confusing, and to LiveCodeBench to analyze overfitting, post-training, and differences between open and closed models. The common thread is that the aggregate score is a starting point: the scientific value often comes from analyzing the structure of the successes and failures behind it.

Designing the right metric and carrying out the right analysis is only the first step. Once an evaluation reveals these nuances, it naturally raises further questions about what capabilities are missing, which failure modes matter most, and what kinds of systems should be built next. In this way, evaluations not only describe the current state of models, but also

influence the trajectory of future research. This is the final lesson: what we measure shapes what we build.

1.3 Lesson 3: What We Measure Shapes What We Build

Evaluations are most useful when they are forward-looking. Once a benchmark is released, the capabilities and failure modes it makes visible often become focal points for the relevant scientific community, influencing both what researchers measure and what model-building labs optimize for. This does not mean that a benchmark must remain difficult forever. Often, its long-term value is that it names a capability, makes a failure mode concrete, and gives the field a shared target to improve.

CRUXEVAL is one example. By measuring code execution and input-output reasoning directly, it helped make execution reasoning a target for both evaluation and training. Many research groups took our benchmark and explicitly built upon our idea: CRUXEval-X [26] extends execution evaluation to 19 programming languages, REval [27] evaluates runtime state prediction in addition to input-output prediction, and CodeMind [28] introduces additional code-reasoning tasks such as specification reasoning and dynamic semantics reasoning. The same idea also influenced follow-up work probing other aspects of language models. SeqCoBench [29] tested whether code LLMs recognize functionally equivalent code under semantics-preserving transformations. The results from CRUXEval also influenced a follow-up work that we did [23], where we used execution as a probe to show that language models often simulate subtly incorrect programs as if they were correct.

The influence was not limited to evaluation. Many studies inspired by CRUXEval used execution traces and input-output prediction as supervision for stronger code reasoning models: SemCoder [30] teaches models to reason about program semantics, CodeI/O [31] trains on input-output prediction, and Self-Execution Simulation [32] combines execution-trace fine-tuning with reinforcement learning for self-repair. CRUXEVAL also became a standard diagnostic benchmark reported by many new coding model releases, including DeepSeek-Coder V2 [33], Qwen 2.5 [34], Qwen 3 [35], Kimi K2 [36], as well as works I was involved in: StarCoder 2 [37] and Code World Model [38]. In this sense, making execution reasoning measurable helped turn it into a capability that later systems were trained for, reported on, and optimized around.

Chapter 4 develops this lesson through a broader set of examples rather than only through CRUXEVAL. LiveCodeBench shows how contamination-aware, time-indexed evaluation became a standard way to measure coding and reasoning models. LeanDojo [39] illustrates how an evaluation can also become infrastructure for an entire research area. ProofOptimizer [40] shows how measuring proof length turns proof simplification into a concrete optimization objective. Together, these examples support the final lesson of the thesis: what we measure shapes what we build.

1.4 Discussion

The technical chapters of this thesis synthesize a collection of research projects on evaluating language models for coding, mathematics, and reasoning. Chapter 2 introduces CRUXEVAL [17], SlopCodeBench [22], IneqMath [18], the Counterfeit Conundrum [23], and LiveCodeBench [20]. Chapter 3 returns to several of these works and also introduces LINC [24]. Chapter 4 additionally introduces LeanDojo [39] and ProofOptimizer [40]. Chapter 5 closes with a discussion of the roadmap paper *Future Research and Remaining Challenges Toward AI for Software Engineering* [41].

Together, these works have uncovered various features and limitations of language models as they have evolved. Several of these findings later influenced the trajectory of model development, as Section 4 will describe in detail. That being said, because models have improved so rapidly in the past few years, some of the direct findings we describe should be understood in the context of the models available at the time. In a few cases, as we will also see in Section 4, this is because our evaluations led to model development efforts addressing the limitations that they revealed. For example, through CRUXEval, we highlighted that models frequently failed to perform simple code execution. In part due to our benchmark, broader code reasoning became a larger focus in the community. Today, execution reasoning is no longer a foreign concept, and almost all of today’s coding models have a baseline ability to reason about code execution.

More generally, evaluations will increasingly become saturated as models improve and the community optimizes around them. Yet, the three overarching lessons of this thesis will continue to remain relevant. Even though the specific failure modes and targets will change over time, the principles of focusing on the right metrics, looking beyond the score, and understanding that what we measure shapes what we build will continue to guide future progress and evaluation design.

Note: These projects were not done alone. Many of the works discussed in this thesis were collaborations, and the full author lists in the cited papers reflect the people who helped shape the ideas, build the systems, run the experiments, and write the original papers. Throughout the thesis, “we” refers to the research teams behind the corresponding works, and I am grateful to all of these collaborators for their contributions to all the content in this thesis.

Chapter 2

Focusing on the Right Metrics Without Bias Is Crucial

Evaluation metrics play a pivotal role in shaping a scientific community’s collective intuition. They determine which aspects of model behavior become visible and how those behaviors are interpreted. Designing new metrics can surface surprising capabilities and shortcomings, while careful scrutiny of existing metrics can reveal confounding factors that distort evaluation outcomes and lead to misleading conclusions about model performance. It is therefore important to both create fresh metrics and remove sources of bias from existing ones, so that benchmark results faithfully reflect the abilities they are intended to measure. As mentioned in Section 1.1, I use *bias* in this thesis to refer to a systematic confounding factor that makes a score reflect something other than the intended measurement, as opposed to statistical or social bias.

In the first three sections, we introduce metrics that capture desired properties and capabilities missed by existing benchmarks. Section 2.1 introduces CRUXEVAL, a benchmark that measures code reasoning, understanding, and execution. Through this work, we provided evidence that coding language models had just started to develop the ability to execute simple Python programs while showing that top models at the time still had significant gaps. Section 2.2 introduces SlopCodeBench, where we measured two metrics related to code quality: verbosity and structural erosion. By analyzing these metrics throughout iterative coding trajectories, we discovered that even if models can write correct code, the quality of that code degrades over time, which has implications for the maintainability of code generated by language models. In Section 2.3, we discuss IneqMath, a benchmark of inequality proving problems. From measuring both final-answer correctness and step-wise reasoning soundness, we revealed that LLMs frequently arrive at the correct answer but hallucinate unsound reasoning steps.

In the final two sections, we turn our attention toward faithful measurement by isolating confounding factors that change how standard metrics should be interpreted. Section 2.4 introduces the *counterfeit conundrum*, where we found that filtering out trivial incorrect samples in the benchmark can greatly change how we perceive accuracy on tasks like correctness checking, code execution, and program repair. Finally, Section 2.5 discusses LiveCodeBench, where we measured accuracy across time on a suite of code-related tasks like code generation, revealing that accounting for contamination effects can drastically affect

how models compare against each other.

2.1 CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution

In late 2023 and early 2024, the primary way that the community evaluated code LMs was via benchmarks such as HumanEval [2] and MBPP [42], which test the ability to generate short code snippets from natural language specifications. While HumanEval and MBPP capture code generation abilities on simple and fundamental tasks, there was an absence of benchmarks capturing other fundamental dimensions of code LMs such as code understanding and execution. It was important to fill this gap, first because these capabilities are fundamental to programming and software engineering. Second, while humans showed a strong correlation between various programming capabilities, it was not clear whether this was true for language models as well.

Motivated by this, we introduced a new benchmark, CRUXEVAL (Code Reasoning, Understanding, and eXecution Evaluation) with two tasks: 1) output prediction, CRUXEVAL-O to measure code execution following and 2) input prediction, CRUXEVAL-I to measure code reasoning and understanding. An example of a sample in CRUXEVAL is shown in Listings 2.1 and 2.2 (modified for readability). CRUXEVAL examines the abilities of code LMs to reason about the execution behavior of *simple* Python programs. While LMs should not be expected to replace an interpreter on arbitrarily complex problems, we ensured the samples in our benchmark were simple (maximum 13 lines, no complex arithmetic) and solvable by a university-level CS graduate without needing extra memory. CRUXEVAL provides a useful and important probe for better understanding the capabilities of code LMs, as following a few simple steps of code execution should be a basic requirement for these models. The ability to reason about the execution behavior of code also paves the way to tackling more difficult tasks such as code repair with execution feedback and code summarization.

Listing 2.1: Sample problem

```
def f(string):
    string_x = string.rstrip("a")
    string = string_x.rstrip("e")
    return string

# output prediction, CRUXEval-O
assert f("xxxxaaee") == ??
## GPT4: "xxxx", incorrect

# input prediction, CRUXEval-I
assert f(??) == "xxxxaa"
## GPT4: "xxxxaae", correct
```

Listing 2.2: Sample problem

```
def f(nums):
    count = len(nums)
    for i in range(-count+1, 0):
        nums.append(nums[i])
    return nums

# output prediction, CRUXEval-O
assert f([2, 6, 1, 3, 1]) == ??
# GPT4: [2, 6, 1, 3, 1, 6, 1, 3, 1], incorrect

# input prediction, CRUXEval-I
assert f(??) == [2, 6, 1, 3, 1, 6, 3, 6, 6]
# GPT4: [2, 6, 1], incorrect
```

At a high level, our benchmark was constructed as follows. First, we used CODE LLAMA 34B to generate a large set of functions and inputs. The outputs were generated by executing the functions on the inputs. Second, we filtered the set so that our benchmark only consists of short problems with low computation and memory requirements, problems which a good

human programmer should be able to do without extra memory in a minute or so. Third, we randomly selected 800 samples passing the filter, ensuring the benchmark is both small enough to easily run but large enough to reliably see performance differences among various models. We used this approach because while it was difficult to manually come up with examples where the strongest evaluated models failed completely, we observed that they failed quite often on random yet reasonable programs. We also highlight that as models improve, this generate-and-filter approach can be used to create future benchmarks that are more difficult and test different aspects of program execution.

Among the evaluated models, GPT-4 achieved a pass@1 of 67% on CRUXEVAL-I and 63% on CRUXEVAL-O. In contrast, the best evaluated open-source models only achieved 47% on CRUXEVAL-I and 44% on CRUXEVAL-O, failing over half the time at simple execution prediction and code reasoning despite being trained on 100G of Python code and 1T of code data. We also discovered that despite its impressive abilities, GPT-4 consistently failed to understand the execution behavior of some surprisingly simple Python programs.

2.1.1 Benchmark Construction

CRUXEVAL consists of 800 distinct functions, each with an input-output pair such that executing the function on the input deterministically produces the output. Using these functions and input-output pairs, we derive two benchmark tasks. In the *output prediction* task, the goal is to predict the output of executing the function on its associated input. In the *input prediction* task, the goal is to find any input such that executing the function on that input produces the output. For both tasks, we use an execution-based correctness metric. For input prediction, a generated input passes if `assert f(generated_input) == output` passes, and for output prediction, a generated output passes if `assert f(input) == generated_output` passes.

Generating Candidates

We used CODE LLAMA 34B to generate all the candidate functions and inputs of CRUXEVAL. To do so, we provided the name of a function in the Python standard library such as `str.zfill` and asked for a Python function that uses the library function together with five test inputs. We used two varying few-shot examples during generation to improve diversity.

We used a total of 69 different functions from the standard library: 47 from the `str`, 11 from `dict`, and 11 from `list`. Overall, we generated a total of 102000 functions (46% `str`, 27% `dict`, 27% `list`) and 489306 input-output pairs before filtering.

Filtering Candidates

Next, we filtered the generated candidates to ensure that the samples in the dataset were reasonable and of high quality. In order to avoid forcing the model to perform tasks such as arithmetic calculation, we designed filters so that the benchmark only consists of samples that are solvable by a human without extra memory.

Concretely, we filter based on the following criteria.

- Compile time: all arguments of the function must be used in the function, length of code is between 75 and 300 characters, no syntax errors, proper assertion `assert f(input) == output`.
- Runtime: no float point operations, true division, exp, other integer operations must have at least one argument ≤ 3 , string and list operations must have at least one argument with length ≤ 3 , finish running in 2 seconds, no uncaught exceptions.
- Best effort to remove other undesirable code: function cannot have any imports (such as os, random), must be deterministic (random, set ordering), and cannot have side effects such as input, `__builtins__`.

Data size and measuring noise

While larger benchmarks lead to higher statistical power, smaller benchmarks are faster and cheaper to run and iterate on. We therefore aimed to make our benchmark as small as possible while controlling the probability that noise would cause us to reach a different conclusion than we would with a larger benchmark. Since additional examples are easy to generate, we first overgenerated and then measured if the noise was sufficiently small on a smaller dataset.

Out of all the samples, CODE LLAMA 34B outperformed CODE LLAMA 13B as expected, and we wanted to retain this property with high confidence in a smaller dataset. To do this, we took bootstrap samples of size N out of ~ 1700 samples to measure the probability that the performance would be reversed, shown in Fig. 2.1. We found that 800 examples were enough to test that CODE LLAMA 34B > CODE LLAMA 13B, CODE LLAMA COT > CODE LLAMA, and DEEPSEEK 33B > CODE LLAMA 34B on output prediction.

We measured two sources of noise: 1) sampling which data points to include in the benchmark, and 2) sampling candidates from models for each data point (temperature > 0). Of these, 1) dominated 2). For 1) since model A does not always outperform model B on all data points even if $A > B$ in aggregate, the measured performance depends on which data points are included. We measured both noise on each model individually, and also measured type 1) noise on pairs of models using bootstrap. Fortunately, we did not see major differences between models and the most important factor was just the size of dataset. Type 1) noise was generally around 1.5% for each model whereas type 2) was around 0.2% at $N = 800$. Type 1) noise usually became smaller on pairs of models due to correlation, yielding statistically significant results at the $\alpha = 0.05$ level for many model pairs.

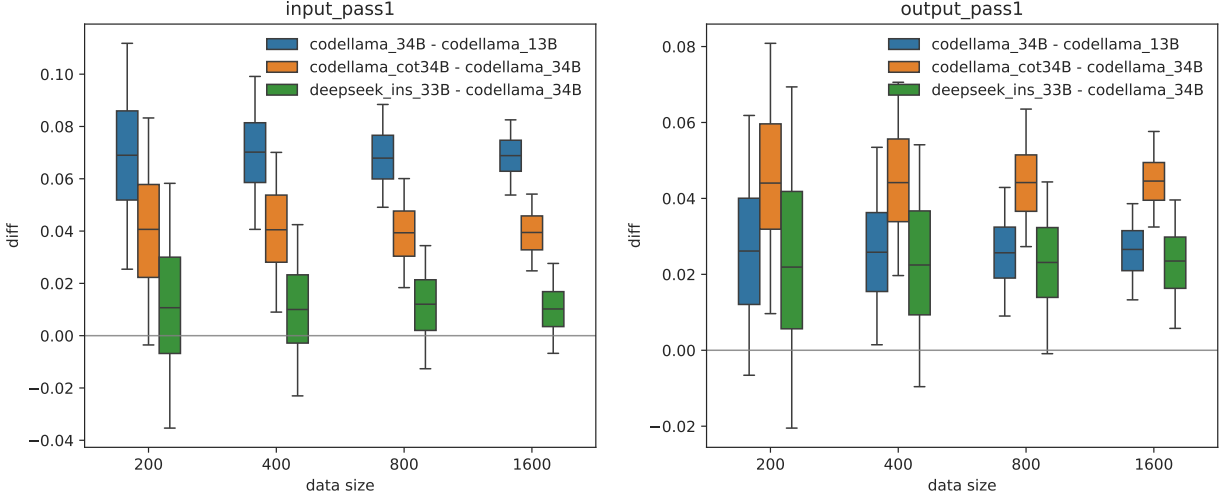


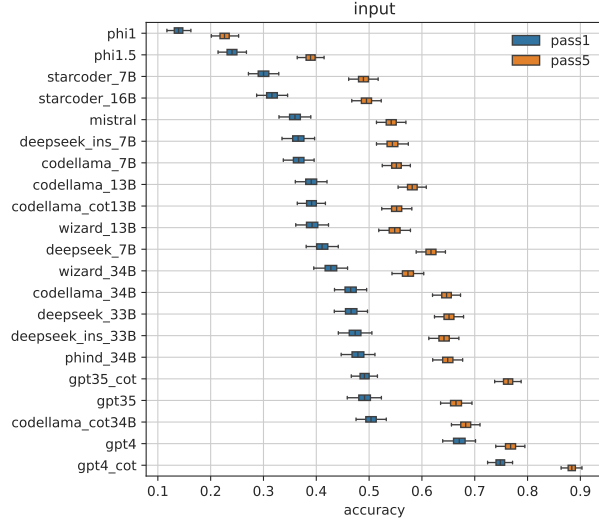
Figure 2.1: Difference between model pairs on bootstrap samples of various sizes. The whiskers show (2.5, 97.5) and boxes show (25, 75) percentiles.

2.1.2 Evaluation

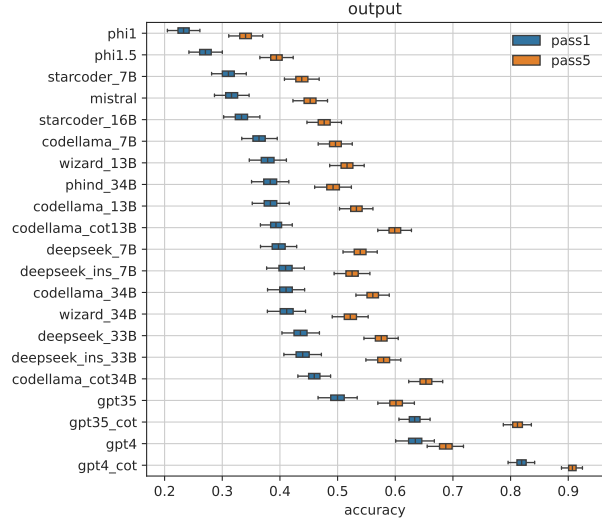
We evaluated a selection of models on CRUXEVAL: StarCoder (Base 7B, 15.5B) [43], Mistral (7B) [44], WizardCoder (13B, 34B) [45], Phi-1 [46] and Phi-1.5 [47] (1.3B), Phind v2 [48] (34B), Code Llama [49] (Base and Python 7B, 13B, 34B), DeepSeek Coder (Base and Instruct 6.7B, 33B), GPT-3.5 [1, 4], and GPT-4 [50].

We used $N = 100$ samples for all non-GPT models and $N = 10$ samples for GPT models. We reported both pass@1 scores ($T = 0.2$) and pass@5 scores ($T = 0.8$). The results are shown in Fig. 2.2. In Fig. 2.2, we show the intervals generated by 10000 bootstrap samples from the dataset, where non-overlapping whiskers would be significant at the 2.5% level. To get more statistical power, we also compared pairs of models on each bootstrapped sample. The intervals generally decreased due to correlations. On all models vs. CODE LLAMA 34B, if the median bar clears the whisker in Fig. 2.2, then the difference actually held with $>97.5\%$ probability under paired bootstrap. For example, CODE LLAMA 34B was better than WIZARD_34B on input and CODE LLAMA 34B was worse than DEEPSEEK_33B on output prediction with $>97.5\%$ probability.

Today, models have improved significantly and saturate the benchmark with relative ease, with open-source models like Qwen3-Coder-Next [51] scoring 96% on CRUXEval. However, as we will discuss in Section 3.1, CRUXEval inspired a family of similar code execution and reasoning benchmarks, several of which still pose a challenge to frontier models today.



(a) CRUXEVAL-I Performance



(b) CRUXEVAL-O Performance

Figure 2.2: Main Results. Boxes show (25, 75) percentiles, whiskers show (2.5, 97.5), and the middle bar shows the median ($\approx$ mean).

2.1.3 Impact of Anonymizing Functions

One potential source of bias is that models may have memorized the functions in our benchmark. While we believe this is not the case due to the synthetically generated nature of the benchmark, we conducted another experiment where we replaced the variable names of functions with synthetic "anonymous" identifiers while keeping function semantics intact. We tested CodeLlama 7B, 13B, and 34B on a subset of our benchmark where variable names are replaced with `x1`, `x2`, ... identifiers. An example of an anonymized function is shown in Listing 2.3. We used the same few-shot prompt without anonymization and reported both pass@1 ($T = 0.2$) and pass@5 ($T = 0.8$) results on the anonymized benchmark with $N = 10$ samples. The results are shown in Table 2.1. This strengthens the case against memorization effects. While it was unclear whether models could perform code execution when we first introduced the benchmark, it is nearly agreed upon by the community that today's models can easily perform code execution and reasoning.

Listing 2.3: Sample of benchmark and anonymized version

```
Original:
def f(s):
    nums = ''.join(filter(lambda c:c.isdecimal(), s))
    if nums == '': return 'none'
    m = max([int(num) for num in nums.split(',')])
    return str(m)
assert f('01,001') == '1001'

Anonymized:
def f(x0):
    x1 = ''.join(filter(lambda x2: x2.isdecimal(), x0))
    if x1 == '':
```



```

    return 'none'
    x3 = max([int(x4) for x4 in x1.split(',')])
    return str(x3)
assert f('01,001') == '1001'

```

Table 2.1: Impact of Anonymization on CRUXEval

Model	Anonymized	Input Prediction		Output Prediction	
		Pass@1	Pass@5	Pass@1	Pass@5
CodeLlama 7B	✗	36.6%	48.0%	36.4%	43.5%
	✓	37.5%	53.3%	34.0%	46.9%
	Δ	+0.9%	+5.3%	-2.4%	+3.4%
CodeLlama 13B	✗	39.0%	50.2%	38.3%	44.7%
	✓	40.0%	55.8%	36.1%	50.6%
	Δ	+1.0%	+5.6%	-2.2%	+5.9%
CodeLlama 34B	✗	46.5%	57.4%	41.1%	47.5%
	✓	48.0%	63.8%	39.1%	54.0%
	Δ	+1.5%	+6.4%	-2.0%	+6.5%

2.1.4 Discussion

When we released CRUXEval, many people in the community were skeptical about whether execution was even a useful metric to measure. After all, a simple way to ace the forward execution subset of CRUXEval is to simply run the code in a Python interpreter. Nevertheless, CRUXEval helped shift attention beyond function-level code generation benchmarks like HumanEval and MBPP. Real-world programming requires a much wider set of skills than writing short interview-style functions, and execution gave the community a concrete way to measure one of those missing capabilities. The benchmark also let us study new phenomena in coding language models. By showing that language models had some ability to perform both forward and reverse execution, we saw early signs that language models had some degree of generalization despite just training on raw code tokens. Overall, CRUXEval worked because it sat just on the boundary of what frontier models could do, stayed simple enough for humans that we would expect models to be able to solve it, and taught the community something new about the state of language models.

2.2 SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks

Most of today’s benchmarks measure the ability of language models to solve a single, static programming task. However, this does not adequately capture the way that software is developed in the real world. Codebases frequently evolve over time as new features are added,

requirements change, and designs are prototyped. The design decisions and abstractions introduced in a codebase frequently have long-term consequences for the maintainability and extensibility of the codebase.

SlopCodeBench was designed to capture this setting and desiderata. It contains 36 language-agnostic problems spanning 196 checkpoints. Each problem is specified solely in terms of observable behavior at a command-line or API boundary. At the first checkpoint, an agent implements the initial specification from scratch; at each later checkpoint, it must modify and extend its own prior solution. Figure 2.3 illustrates this process.

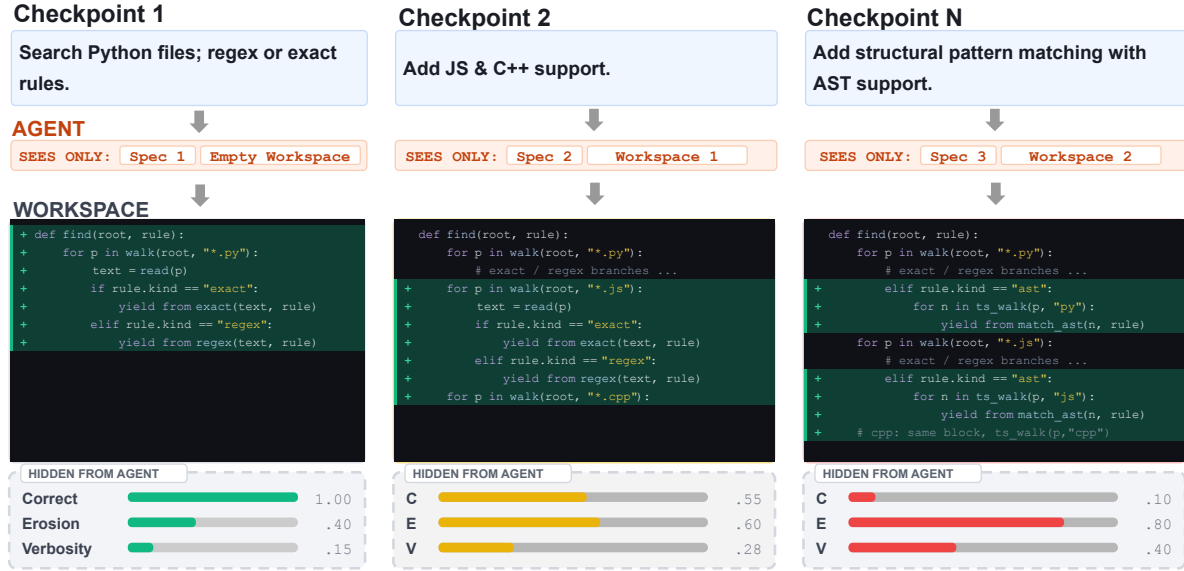


Figure 2.3: Iterative evaluation in SlopCodeBench. A task unfolds over checkpoints. At each step, the agent extends its prior solution to satisfy an updated specification. Correctness, erosion, and verbosity are scored externally and withheld from the agent.

Below is an example of a task from SlopCodeBench:

Running example: code_search. The agent is tasked with building a CLI tool for semantic source-file search, inspired by [ast-grep](#), across five checkpoints:

- C_1 - Python files only exact and regex matching. Establishes the core CLI contract.
- C_2 - Support for Javascript and C++.
- C_3 - AST-based pattern matching with metavariable capture.
- C_4 - Selector rules and auto-fix functionality.
- C_5 - Add support for Go, Rust, and Java.

An agent that hardcodes language-specific logic at C_1 faces cascading rewrites at C_2 and C_5 ; one that builds an extensible parser interface does not.

In addition to an iterative benchmark, SlopCodeBench introduced two trajectory-level metrics to measure the quality of the generated code as it evolves: **structural erosion** and **verbosity**. Structural erosion captures the concentration of complexity in already-complex functions. Verbosity captures how much redundant, duplicated, or unnecessary code is present. These metrics are designed to measure two dimensions of code quality while being independent of functional correctness.

2.2.1 Measuring Code Slop

The central metric contribution is to measure not only whether the code passes tests, but how its quality changes over time. SlopCodeBench tracks two trajectory-level quality signals: **structural erosion** and **verbosity**.

Structural erosion captures the concentration of control-flow complexity in already-complex functions. We first scored each callable f with a *complexity mass*. In this formula, $CC(f)$ is the cyclomatic complexity of f : roughly, the number of independent paths through the callable’s control flow, increasing with branches, loops, and other decision points. $SLOC(f)$ is the number of source lines of code in f . We defined

$$\text{mass}(f) = CC(f) \times \sqrt{SLOC(f)}.$$

Erosion measures the fraction of the codebase’s total complexity mass that sits inside high-complexity functions:

$$\text{Erosion} = \frac{\sum_{f \in \mathcal{F}} \mathbb{I}[CC(f) > 10] \cdot \text{mass}(f)}{\sum_{f \in \mathcal{F}} \text{mass}(f)}. \quad (2.1)$$

Here, $\mathcal{F}$ is the set of callables in the program, and $\mathbb{I}[CC(f) > 10]$ is an indicator that equals 1 only for callables whose cyclomatic complexity exceeds 10. This metric is designed to penalize an anti-pattern where instead of introducing helpful abstractions, an agent repeatedly patches a large dispatch function or main loop until the codebase’s complexity collapses into a few hard-to-maintain functions.

Verbosity captures a different failure mode: redundant, duplicated, or unnecessary code. It combines two sources of evidence. The first is lines flagged by 137 AST-Grep rules for wasteful patterns; the second is lines that belong to structural code clones. We deduplicate overlapping hits and normalize by LOC, the total lines of code in the solution:

$$\text{Verbosity} = \frac{|\{\text{AST-Grep Flagged Lines} \cup \text{Clone Lines}\}|}{\text{LOC}}.$$

2.2.2 Measuring Correctness and Cost Over Time

In Table 2.2, we showed the performance of models on SlopCodeBench. We grouped solutions into three categories: strict, isolated, and core. Core means that a solution passes only a set of core tests, testing behavior explicitly mentioned in the specification. Isolated (Iso) means that the solution passes all tests that only depend on the current checkpoint (it can fail regression tests). Strict means that the solution passes the full test suite, including regression tests from prior checkpoints.

Table 2.2: Per-model performance on SlopCodeBench. For each model we report its best just-solve run, ranked by isolated solve rate with core, strict, and \$/checkpoint as tiebreakers. Solve rates use a fixed 196-checkpoint denominator; missing checkpoints count as unsolved. Other metrics are only for ran checkpoints. **Bold** marks the best value per column.

Model	Solve Rate (%)				Cost & Time			Quality	
	Strict	Iso.	Core	Partial	\$/CKPT	Net \$	Min/CKPT	Erosion	Verbosity
Opus 4.5	9.2	17.3	57.7	25.0	2.53±1.57	492.40	7.2±3.7	0.70±0.20	0.43±0.18
Opus 4.6	9.7	20.9	67.3	27.8	3.17±2.82	620.80	13.1±12.9	0.75±0.15	0.44±0.19
Opus 4.7	8.2	20.9	65.8	36.1	2.17±1.83	425.34	6.8±5.0	0.76±0.17	0.48±0.21
Sonnet 4.6	7.1	16.8	57.7	27.8	1.96±1.97	383.83	14.2±14.2	0.75±0.20	0.44±0.19
GPT 5.2 Codex	9.7	21.9	56.1	36.1	0.85±0.80	167.29	12.3±9.7	0.73±0.18	0.50±0.16
GPT 5.3 Codex	11.2	26.0	60.7	41.7	0.66±0.49	129.65	7.7±4.6	0.64±0.19	0.46±0.18
GPT 5.3 Spark	3.1	8.2	29.1	11.1	0.20±0.41	37.17	2.4±4.6	0.68±0.19	0.48±0.16
GPT 5.4	10.7	23.5	62.8	36.1	0.72±0.51	141.32	8.0±4.7	0.51±0.19	0.33±0.13
GPT 5.4 Mini	5.1	13.8	53.1	22.2	0.45±0.35	89.03	6.2±3.8	0.66±0.17	0.42±0.12
GPT 5.5	14.8	28.1	66.8	50.0	1.51±0.81	295.59	6.0±2.4	0.49±0.20	0.32±0.12
Composer 2	6.1	16.3	52.6	13.9	0.44±0.44	86.31	8.8±8.3	0.72±0.15	0.45±0.17
GLM 5.1	9.7	13.8	40.3	30.6	1.47±1.45	288.85	31.4±20.0	0.71±0.17	0.41±0.18
Kimi K2.5	4.6	9.7	39.8	19.4	0.33±0.25	63.21	14.5±10.0	0.72±0.20	0.44±0.20
Kimi K2.6	10.7	18.9	51.0	33.3	0.74±0.64	133.18	26.9±20.7	0.76±0.20	0.51±0.22
MiniMax M2.7	2.0	4.1	28.1	8.3	0.34±0.23	65.79	11.9±6.8	0.73±0.16	0.47±0.17

Across 15 evaluated coding agents, no agent fully solved any of the 36 problems end-to-end. The best strict solve rate was 14.8%, achieved by GPT 5.5, while the best isolated solve rate reached 28.1% and the best core solve rate reached 67.3%. In Figure 2.4 (left), we plotted the correctness metrics across the trajectory of checkpoints. We found that the core pass rate fell from 64.6% early in the trajectory to 35.5% late, even as mean cost per checkpoint grew by 2.2×. These results highlight something that is not evident from single-task benchmarks: later checkpoints are harder not only because the specification grows, but because the agent must operate inside the codebase it previously created.

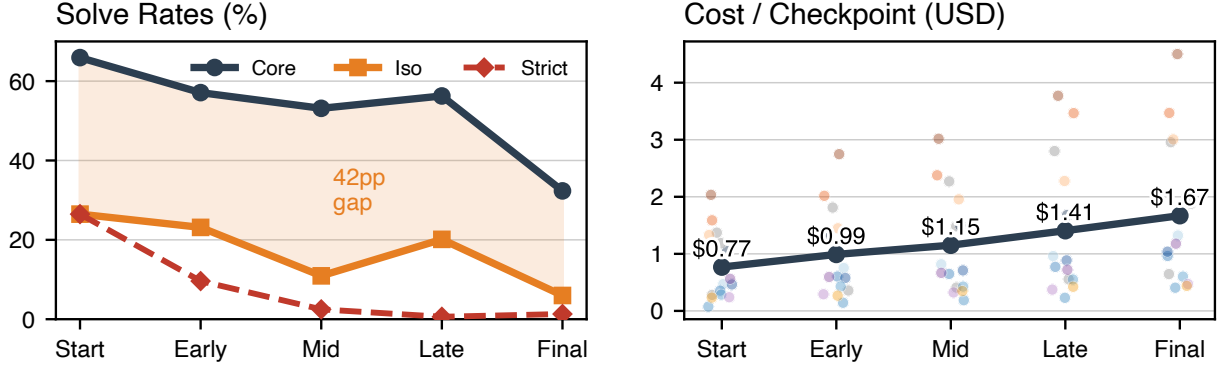


Figure 2.4: SlopCodeBench shows that correctness degrades as iterative problems progress while cost per checkpoint rises. The left panel separates strict, isolated, and core solve rates; the right panel shows mean cost per checkpoint over the trajectory.

2.2.3 Measuring Code Quality Over Time

Next, we examined the quality of code generated by agents throughout the process of completing checkpoints for each task. As shown in Figure 2.5, both structural erosion and verbosity tended to increase as agents progressed through the checkpoints. Across all settings, structural erosion increased in 77% of trajectories and verbosity increased in 75.5%. The average number of functions with cyclomatic complexity at least 10 rose from 3.6 to 23.7, and mean maximum cyclomatic complexity rose from 27.5 to 69.0.

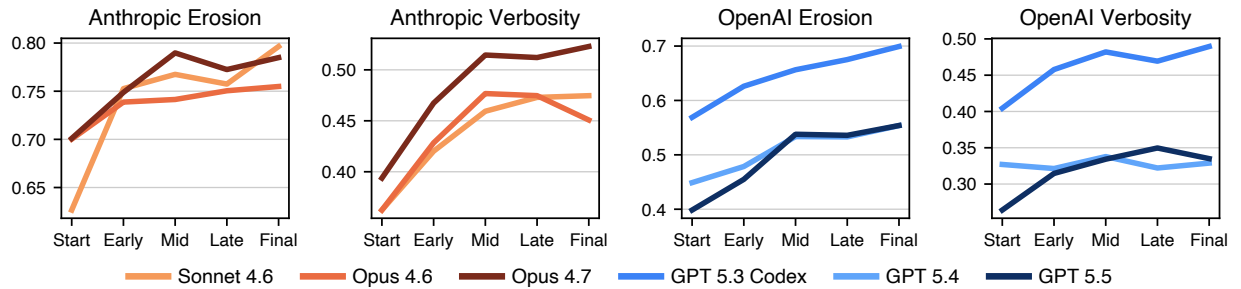


Figure 2.5: Structural erosion and verbosity increase as agents progress through SlopCodeBench tasks. Iterative evaluation makes this accumulation visible because quality is measured at every checkpoint, not only on the final artifact.

2.2.4 Calibration Against Human Repository Histories

Finally, we compared agent trajectories against existing human-maintained Python repositories. We sampled 473 repositories across web frameworks, scientific computing, infrastructure, command-line utilities, and other domains, with up to 30 source-modifying commits per repository for a total of 13667 checkpoints.

Figure 2.6 shows that agent code was $2.3\times$ above the human mean for verbosity and $2.0\times$ above the human mean for erosion. Only 18 of 473 human-written repositories exceeded the agent verbosity mean. When comparing individual checkpoints across trajectories, we found that agent repositories accumulated verbosity $6.6\times$ faster and erosion $5.0\times$ faster than human repositories.

Table 2.3 gives the corresponding HEAD snapshot by GitHub-star tier, showing that the agent gap was larger than the variation across the sampled human repository groups.

Table 2.3: Verbosity and erosion at HEAD commit, by GitHub-star tier and against agent checkpoints.

Group	n	Verbosity	Erosion
Hobby ($< 100\star$)	114	0.21 ± 0.14	0.33 ± 0.25
Niche (100–1k $\star$)	124	0.19 ± 0.11	0.32 ± 0.22
Established (1k–10k $\star$)	120	0.18 ± 0.08	0.33 ± 0.20
Major ($\geq 10k\star$)	115	0.19 ± 0.10	0.37 ± 0.19
Human panel (all)	473	0.19 ± 0.11	0.34 ± 0.22
Agent checkpoints	2869	0.44 ± 0.18	0.68 ± 0.20

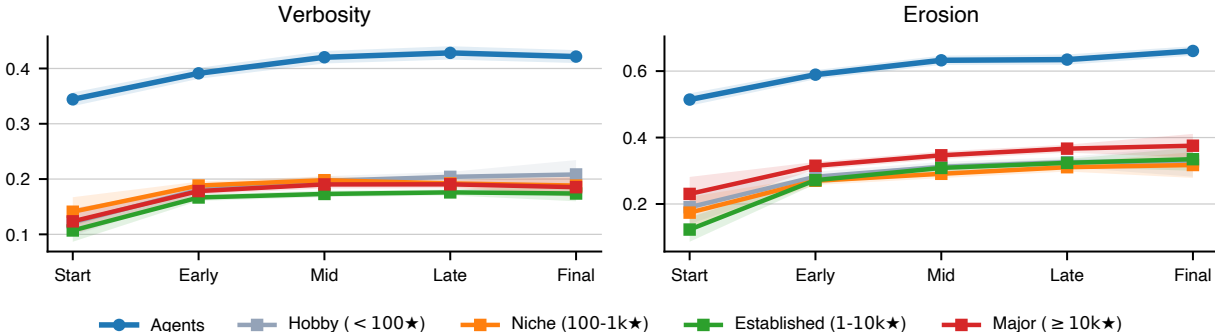


Figure 2.6: Agent trajectories have higher verbosity and erosion than sampled human repository histories, and they degrade faster over normalized trajectory progress.

2.2.5 Discussion

Overall, SlopCodeBench shed light on a model’s ability to write high-quality code in an environment that mirrors real-world software development. Instead of stopping at single-shot tasks, we used an iterative feature-adding setup that required models to plan ahead and write maintainable code. As more feature requirements were added, models struggled to

maintain correctness despite increasing cost. By looking beyond correctness toward verbosity and erosion, we were also able to show how AI-generated code quality decreased over time. SlopCodeBench reinforces the same lesson from another angle: once models are asked to keep extending a codebase, unit-test correctness is no longer enough; iterative progress, verbosity, and erosion expose failures that a one-shot benchmark would miss.

2.3 IneqMath: Solving Inequality Proofs with Large Language Models

In INEQMATH, we studied the ability of language models to solve olympiad-style inequality problems. While previous evaluations in mathematics such as GSM8K [13] and MATH-500 [15] focus on final-answer correctness, we designed INEQMATH to measure step-wise reasoning correctness in addition to final-answer accuracy. By doing so, we discovered what we call a *soundness gap*: models often arrive at the correct final answer while hallucinating bogus reasoning steps, reinforcing the importance of evaluating based on the correct metric.

2.3.1 Task Setup, Dataset, and Metrics

An inequality problem generally involves proving that a specified inequality holds under given conditions, such as *prove $a + b \geq 2\sqrt{ab}$ for all positive real numbers a and b* . In INEQMATH, we reformulate inequality proof problems into two verifiable subtasks: **bound estimation** and **relation prediction**. The bound estimation task asks for an optimal constant in a given inequality. For example, in $a + b \geq C\sqrt{ab}$ for all $a, b > 0$, the objective is to find the largest valid value of C . An example of such a task is shown below. The relation prediction task asks the model to determine the correct relationship between two expressions. For instance, given $f(\mathbf{x}) = a + b$ and $g(\mathbf{x}) = 2\sqrt{ab}$, the goal is to identify the relation, in this case $\geq$, that holds for all $a, b > 0$.

Question: Find the maximal constant C such that for all real numbers a, b, c , the inequality holds:

$$\sqrt{a^2 + (1-b)^2} + \sqrt{b^2 + (1-c)^2} + \sqrt{c^2 + (1-a)^2} \geq C$$

Solution: Applying Minkowsky’s Inequality to the left-hand side we have

$$\sqrt{a^2 + (1-b)^2} + \sqrt{b^2 + (1-c)^2} + \sqrt{c^2 + (1-a)^2} \geq \sqrt{(a+b+c)^2 + (3-a-b-c)^2}$$

By denoting $a + b + c = x$, we get

$$\sqrt{(a+b+c)^2 + (3-a-b-c)^2} = \sqrt{2\left(x - \frac{3}{2}\right)^2 + \frac{9}{2}} \geq \sqrt{\frac{9}{2}} = \boxed{\frac{3\sqrt{2}}{2}}.$$

Minkowsky’s Inequality Theorem: For any real number $r \geq 1$ and any positive real numbers $a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n$

$$\left(\sum_{i=1}^n (a_i + b_i)^r\right)^{\frac{1}{r}} \leq \left(\sum_{i=1}^n a_i^r\right)^{\frac{1}{r}} + \left(\sum_{i=1}^n b_i^r\right)^{\frac{1}{r}}$$

We split inequality proofs into these two subtasks because they simplify the evaluation process while retaining the core challenges inherent in the original problems. In total, INEQMATH contains a 1,252-problem training corpus split evenly between bound-estimation and relation-prediction tasks, as well as a test set containing 200 olympiad-level benchmark problems.

To measure the accuracy of a problem, we used a final-answer judge in conjunction with a suite of step-wise proof soundness judges. The **final-answer judge** is similar to that used in previous math benchmarks: it extracts the final answer and checks mathematical equivalence to the reference answer, as done in MATH-500 [52]. The step-wise proof soundness judge checks for four common forms of unsound reasoning. The **toy-case judge** flags solutions that infer a universal claim from a few numerical examples. The **logical-gap judge** flags missing derivation steps, unjustified transformations, and unsupported optimality claims. The **numerical-approximation judge** flags cases where decimal approximations replace exact symbolic reasoning in a way that affects the proof. The **numerical-computation judge** checks arithmetic and substitution errors once concrete quantities appear.

We reported three different metrics. *Answer accuracy* only requires the final answer to be correct. *Step accuracy* requires the reasoning trace to pass all four step-wise checks. *Overall accuracy* requires both, and is the strongest check ensuring that both the answer and the proof reasoning steps are correct.

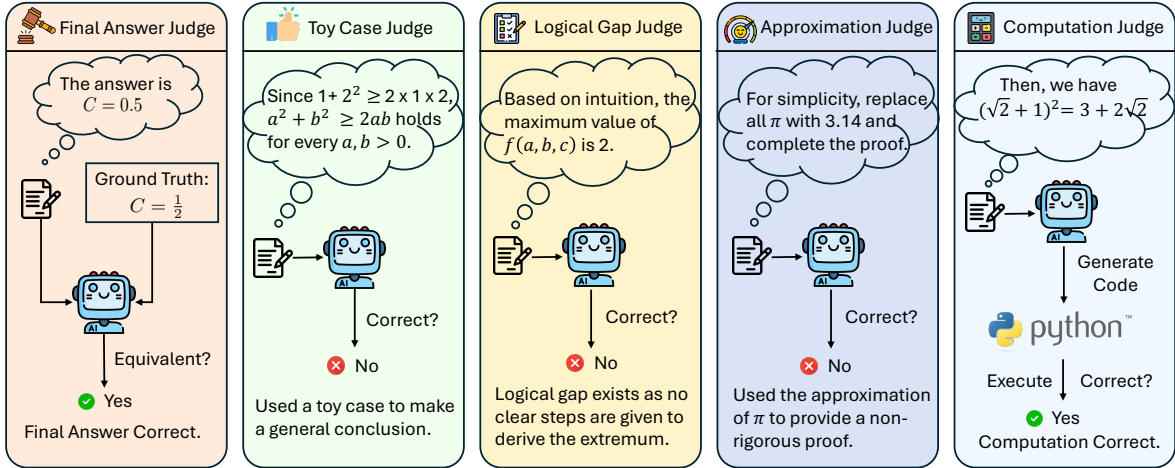


Figure 2.7: IneqMath evaluates both the final answer and the reasoning trace. A solution is counted as fully correct only when it passes the final-answer judge and all step-wise soundness judges.

2.3.2 The Soundness Gap

In Table 2.4, we showed the performance of 24 models on INEQMATH, revealing several insights into LLM capabilities for inequality proving:

1) **Reasoning LLMs achieve higher final-answer accuracy.** First, we compared reasoning models with their general-purpose chat counterparts. We saw that o1 (62.5% *Answer Acc*) and Grok 3 mini (71.5% *Answer Acc*) significantly outperformed GPT-4o

Models	Overall Acc (↑)			Answer Acc (↑)			Step Acc (↑)											
							No Toy Case			No Logic. Gap			No Approx. Error			No Comp. Error		
	All	Bnd.	Rel.	All	Bnd.	Rel.	All	Bnd.	Rel.	All	Bnd.	Rel.	All	Bnd.	Rel.	All	Bnd.	Rel.
<i>Heuristic Methods</i>																		
Random Guess	-	-	-	8.5	0.0	16.3	-	-	-	-	-	-	-	-	-	-	-	-
Frequent Guess	-	-	-	18.0	9.4	26.0	-	-	-	-	-	-	-	-	-	-	-	-
<i>Open-source Chat LLMs</i>																		
Qwen2.5-Coder-32B [53]	1.5 _[39.0]	1.0 _[50.0]	1.9 _[28.9]	40.5	51.0	30.8	36.0	27.1	44.2	3.0	2.1	3.8	90.5	96.9	84.6	88.5	89.6	87.5
Llama-4-Scout [54]	1.5 _[32.0]	2.1 _[44.8]	1.0 _[20.2]	33.5	46.9	21.2	30.5	15.6	44.2	3.5	4.2	2.9	93.0	94.8	91.3	92.5	92.7	92.3
Qwen2.5-72B [55]	2.5 _[39.5]	3.1 _[47.9]	1.9 _[31.8]	42.0	51.0	33.7	54.5	53.1	55.8	5.0	4.2	5.8	91.0	94.8	87.5	95.0	94.8	95.2
Llama-4-Maverick [56]	2.5 _[38.0]	2.1 _[43.7]	2.9 _[32.7]	40.5	45.8	35.6	42.5	28.1	55.8	4.0	4.2	3.8	89.0	91.7	86.5	95.0	92.7	97.1
Qwen2.5-7B [57]	3.0 _[32.0]	2.1 _[38.5]	3.8 _[26.0]	35.0	40.6	29.8	44.5	32.3	55.8	4.5	3.1	5.8	92.5	96.9	88.5	93.0	92.7	93.3
<i>Proprietary Chat LLMs</i>																		
Gemini 2.0 Flash-Lite [58]	1.5 _[31.5]	2.1 _[41.7]	1.0 _[22.1]	33.0	43.8	23.1	11.5	11.5	11.5	3.5	3.1	3.8	73.0	77.1	69.2	90.5	87.5	93.3
GPT-4o mini [59]	2.0 _[37.5]	1.0 _[41.7]	2.9 _[33.6]	39.5	42.7	36.5	29.0	11.5	45.2	2.5	2.1	2.9	90.0	91.7	88.5	93.0	92.7	93.3
GPT-4.1 [60]	2.5 _[38.0]	0.0 _[31.3]	4.8 _[44.2]	40.5	31.3	49.0	16.0	12.0	19.0	10.0	8.3	11.5	59.5	66.7	52.9	93.5	92.7	94.2
GPT-4o [61]	3.0 _[34.5]	2.1 _[38.5]	3.8 _[30.8]	37.5	40.6	34.6	32.0	21.9	43.0	3.5	3.1	3.8	92.5	93.8	91.4	94.0	93.8	94.2
Gemini 2.0 Flash [62]	3.0 _[46.0]	3.1 _[56.3]	2.9 _[36.5]	49.0	59.4	39.4	15.5	13.5	17.3	13.5	7.3	19.2	55.5	60.4	51.0	94.5	94.8	94.2
Grok 3 [63]	3.5 _[51.0]	4.2 _[62.5]	2.9 _[40.4]	54.5	66.7	43.3	17.0	13.7	20.2	16.0	11.6	20.2	36.0	42.1	30.8	93.0	96.8	90.4
<i>Open-source Reasoning LLMs</i>																		
QwQ-32B [64]	2.0 _[47.5]	2.1 _[52.1]	1.9 _[43.3]	49.5	54.2	45.2	26.0	25.0	26.9	29.5	20.1	37.5	21.0	20.8	21.2	87.0	82.3	91.3
Deepseek-R1 (Llama-70B) [65]	3.5 _[50.0]	5.2 _[53.1]	1.9 _[47.1]	53.5	58.3	49.0	23.0	24.0	22.1	26.0	20.9	30.8	35.5	38.5	32.7	87.0	89.6	84.6
Deepseek-R1 (Qwen-14B) [66]	5.0 _[35.5]	6.3 _[36.4]	3.8 _[34.7]	40.5	42.7	38.5	21.0	18.8	23.1	21.0	19.8	22.1	35.5	38.5	32.7	85.0	91.7	78.8
Deepseek-R1 [67]	5.0 _[44.5]	4.2 _[63.5]	5.8 _[26.9]	49.5	67.7	32.7	57.0	53.1	60.9	17.5	6.3	27.9	81.0	95.8	67.3	95.0	99.0	91.3
Qwen3-235B-A22B [68]	6.0 _[35.0]	3.1 _[32.3]	8.7 _[37.5]	41.0	35.4	46.2	35.0	30.2	39.4	36.0	26.0	45.2	31.0	28.1	33.7	92.5	93.8	91.3
<i>Proprietary Reasoning LLMs</i>																		
Claude 3.7 Sonnet [69]	2.0 _[40.0]	2.1 _[44.8]	1.9 _[35.6]	42.0	46.9	37.5	49.0	36.5	60.6	4.0	3.1	4.8	93.5	95.8	91.3	93.0	90.6	95.2
Gemini 2.5 Flash [70]	4.5 _[11.0]	3.1 _[11.1]	5.8 _[0.9]	5.5	4.2	6.7	88.0	84.4	91.3	13.5	7.3	19.2	100.0	100.0	100.0	100.0	100.0	100.0
Grok 3 mini [19]	6.0 _[65.5]	4.2 _[68.7]	7.7 _[62.5]	71.5	72.9	70.2	24.0	16.7	30.8	19.5	11.5	26.9	53.5	63.5	44.2	91.0	94.8	87.5
Gemini 2.5 Pro [71]	6.0 _[11.0]	7.3 _[11.0]	4.8 _[11.0]	7.0	8.3	5.8	88.5	83.3	93.3	19.0	12.5	25.0	100.0	100.0	100.0	99.5	100.0	99.0
o1 [72]	8.0 _[54.5]	7.3 _[55.2]	8.7 _[53.8]	62.5	62.5	62.5	34.5	37.5	31.7	17.5	12.5	22.1	86.5	99.0	75.0	99.5	100.0	99.0
o3-mini [73]	9.5 _[53.0]	7.3 _[62.5]	11.5 _[44.3]	62.5	69.8	55.8	37.0	34.4	39.4	22.0	17.7	26.0	77.5	92.7	63.5	95.0	96.9	93.3
o4-mini [74]	15.5 _[49.5]	14.6 _[48.9]	16.3 _[50.0]	65.0	63.5	66.3	62.0	58.3	65.4	26.0	25.0	26.9	86.5	90.6	82.7	93.0	92.7	93.3
o3 [74]	21.0 _[16.0]	18.8 _[11.4]	23.1 _[20.2]	37.0	30.2	43.3	93.5	91.7	95.2	39.5	28.1	50.0	91.5	99.0	84.6	97.0	96.9	97.1
Average Accuracy (↑)	5.0 _[38.0]	4.5 _[42.9]	5.5 _[33.5]	43.0	47.4	39.0	40.3	34.8	45.5	15.0	11.0	18.7	73.1	77.9	68.8	93.2	93.7	92.8
Average Error Rate (↓)	95.0 _[38.0]	95.5 _[42.9]	94.5 _[33.5]	57.0	52.6	61.0	59.7	65.2	54.5	85.0	89.0	81.3	26.9	22.1	31.2	6.8	6.3	7.2

Table 2.4: Evaluation performance of chat and reasoning LLMs on the INEQMATH benchmark (the test set). *Bnd.* denotes bound problems and *Rel.* denotes relation ones. We report: (1) *Overall Acc*, which reflects the correctness of both the final answer and intermediate steps; (2) *Answer Acc*, which measures final answer correctness alone; and (3) *Step Acc*, which evaluates the accuracy of intermediate steps across four error categories—*Toy Case*, *Logical Gap*, *Numerical Approximation*, and *Numerical Computation*. Blue superscripts ↓ indicate accuracy drop (*Overall Acc* - *Answer Acc*) from step-wise errors. Underlining denotes best result within each model category; **boldface** highlights best overall performance. Default max token limit for reasoning LLMs is 10K.

(37.5%) and Grok 3 (54.5%) in final-answer accuracy.

2) Step-wise scrutiny reveals a dramatic performance drop. Second, we discovered a large soundness gap across all models. As shown in Figure 2.8, the *Overall Acc* plummeted when steps were evaluated. For instance, Grok 3 mini’s accuracy dropped from 71.5% *Answer Acc* to 6.0% *Overall Acc*.

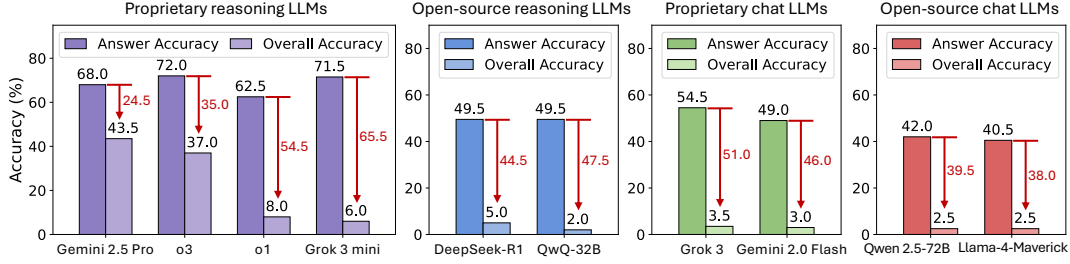


Figure 2.8: IneqMath highlights the gap between final-answer accuracy and full reasoning soundness. The chapter-level lesson is that final-answer success is not enough evidence of sound mathematical reasoning.

3) Robust proof construction remains a major challenge. Third, we found that providing a mathematically rigorous proof remains a significant challenge for all models. Even the best model, Gemini 2.5 Pro, only achieved 43.5% *Overall Acc*, highlighting that solving challenging inequality proofs remains a challenge for these LLMs.

4) Scaling does not reliably close the soundness gap. Finally, we investigated whether scaling up model size or test-time computation can help close the soundness gap. The answer was largely no, as seen in Figure 2.9. Larger models did tend to improve final-answer accuracy but did not help with overall accuracy. Similarly, increasing the maximum token budget of models could initially help, but quickly plateaued, as shown in Figure 2.10.

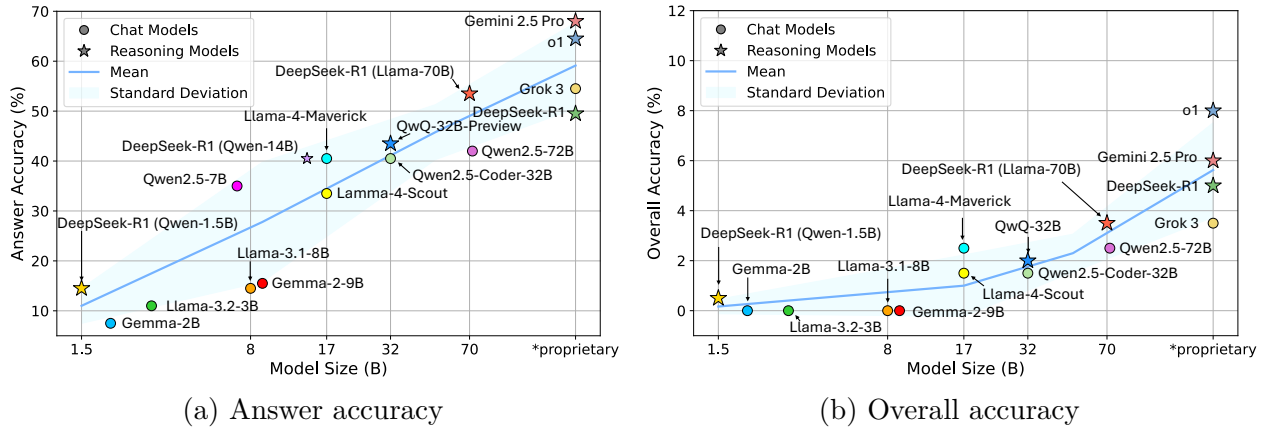


Figure 2.9: Scaling improves final-answer accuracy more reliably than it improves full reasoning soundness.

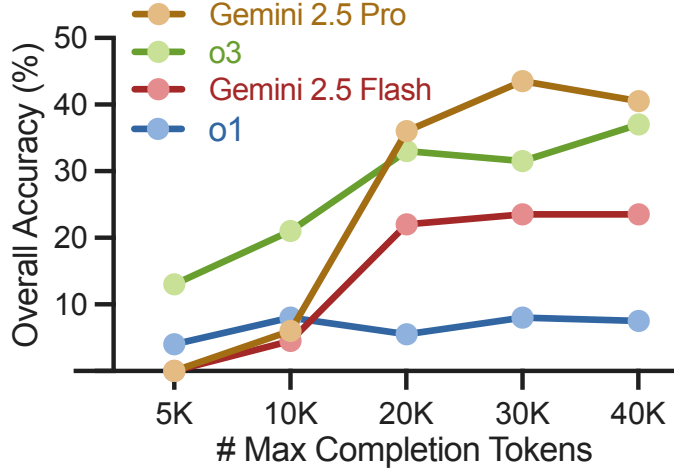


Figure 2.10: Scaling law in test-time computation for reasoning LLMs.

2.3.3 Discussion

This soundness gap shows that benchmarks are only as informative as the capabilities they measure. Because evaluating soundness is more difficult than checking a final answer, many math benchmarks had focused on the latter. Through INEQMATH, measuring overall accuracy alongside final-answer accuracy revealed a critical blind spot in our understanding of LLM problem-solving capabilities.

2.4 The Counterfeit Conundrum: Can Code Language Models Grasp the Nuances of Their Incorrect Generations?

In *The Counterfeit Conundrum*, we studied how measurements of program understanding are distorted by the kinds of incorrect programs being evaluated. For example, when building a benchmark to evaluate a model’s ability to distinguish correct programs from incorrect ones, we may find that models perform well on the benchmark only because the incorrect programs have egregious mistakes that are easy to detect. The resulting bias is that the negative samples becomes too unbalanced: high accuracy may reflect a model’s ability to reject obvious failures rather than its ability to recognize subtle semantic errors in plausible programs. In this work, we focused our attention on other incorrect programs that fail in more subtle ways, which we call **counterfeit samples**.

We define a counterfeit sample to be a program sampled from a code language model which is 1) good enough to be generated by the language model at a moderate temperature, 2) incorrect, and 3) passes weak but nontrivial correctness checks. In this work, we studied the extent to which models could understand these counterfeit programs. This final criterion of passing nontrivial correctness checks distinguishes programs with more subtle errors from those that trivially fail and are likely uninteresting. Fig. 2.11 shows an example of an incorrect, counterfeit, and correct program. Because we used relatively weak correctness

checks, many counterfeit programs could still be easily detected as wrong by a human.

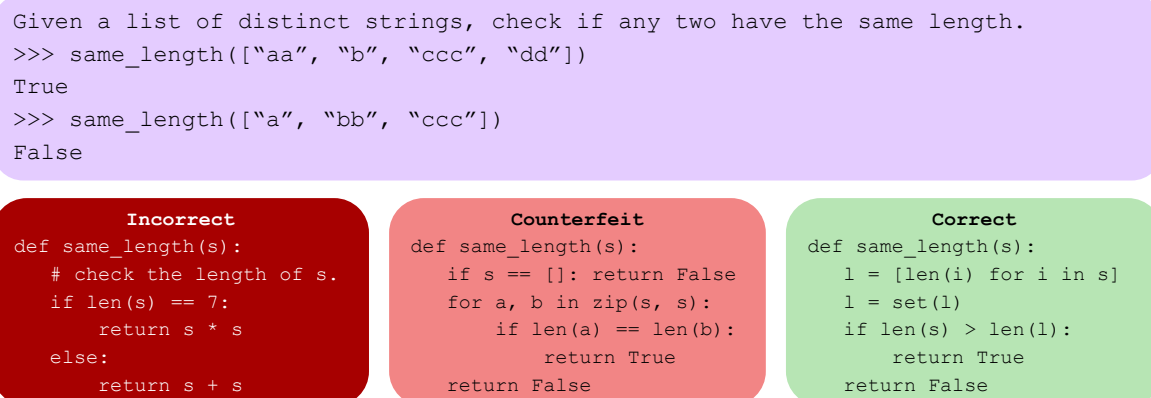


Figure 2.11: Example of a problem specification with incorrect, counterfeit, and correct programs.

We provided empirical evidence that code language models had a shallow understanding of these counterfeit samples (Sec. 2.4.2) via three evaluations: correctness checking, execution prediction, and program repair. For correctness checking, the model is asked to assess whether a short piece of code correctly implements a natural language specification (sometimes with test cases). For execution prediction, the model is given a program-input pair and asked to predict the output of executing the program on the given input. For fairness, we ensured the programs were generally short and that execution did not require complex calculations. For repair, the model is given the counterfeit program alongside its original specification and is asked to correct it. First, we found that models frequently misjudged counterfeit samples as correct. Second, models were much worse at reasoning about the execution of counterfeits than their correct companions, often executing counterfeits as if their semantics matched those of a correct program. Third, models faltered at repair: the likelihood of a model successfully repairing a counterfeit example was often even lower than that of generating a correct program when sampling from scratch. As a caveat, our experiments focused on open-source models, primarily CodeLlama 34B and DeepSeek Instruct 33B. We also presented limited results on GPT-3.5 and GPT-4 which suggested that GPT-3.5 behaved similarly to the open-source models while GPT-4 had a much better understanding of counterfeits. Nevertheless, we still found that GPT-4 exhibited some of these misunderstandings.

2.4.1 Experimental Setup

Generating Counterfeit Examples

We used three datasets: HumanEval, LeetCode, and ODEX [75]. HumanEval evaluates code generation of simple natural language descriptions, LeetCode is a harder dataset of the same flavor using programming interview practice problems, and ODEX tests knowledge of diverse Python libraries. To generate counterfeit examples, we first sampled programs from CodeLlama (CL), DeepSeek-Coder Instruct (DS-I), and StarCoder (SC) at temperature $T = 0.6$. Of the incorrect programs, we designed a dataset-specific filter to remove incorrect

programs that did not pass mild correctness criteria. For HumanEval, counterfeits are programs passing at least 10% of EvalPlus [76] tests. For LeetCode, counterfeits are programs that received a “Wrong Answer” verdict, which filters out programs that crashed during runtime or took too long to finish. For ODEX, counterfeits are programs that can be successfully parsed by `ast.parse` and are under 500 characters. Fig. 2.12 shows the number of counterfeits generated by different models (left), benchmarks (middle), and problem difficulty levels (right), showing that counterfeits are widespread and occur in each setting. In particular, for HumanEval, most models generated sizeable numbers of both ordinary incorrect samples and counterfeit samples, with the two categories often appearing in comparable amounts. If an evaluation pools these together as a single negative class, it can create the illusion that models recognize wrong programs, even if much of the apparent success comes from identifying the more blatant failures.

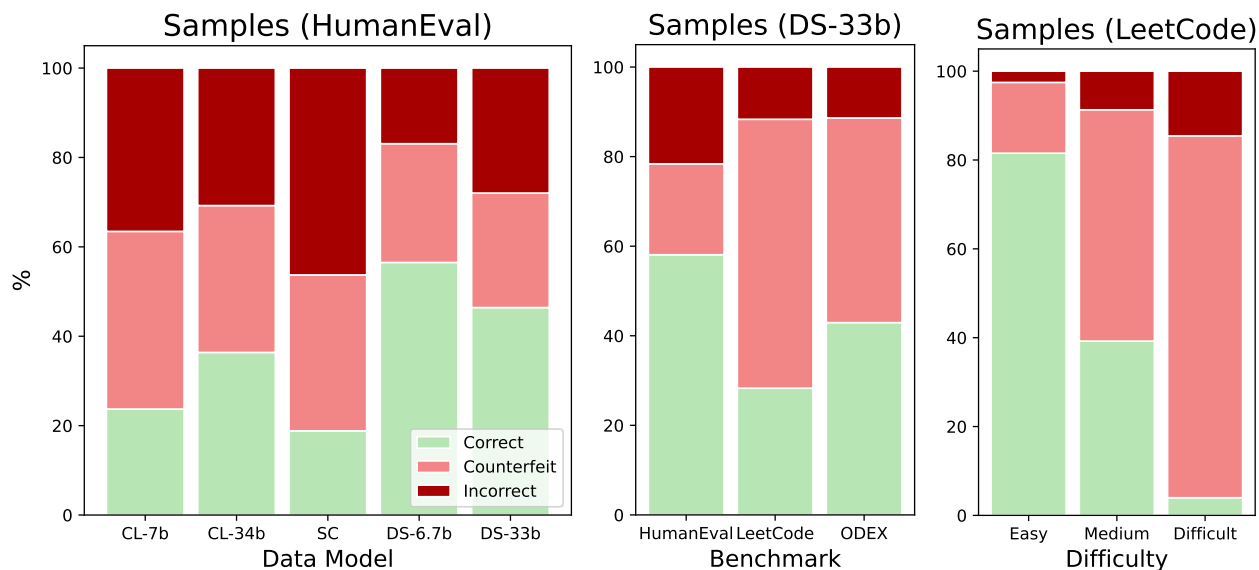


Figure 2.12: Distribution of correct, counterfeit, and incorrect samples by model, benchmark, and problem difficulty, showing their prevalence across all settings.

Code Understanding Tasks

We evaluated counterfeits on the following tasks:

Correctness Checking: The goal is to check whether a model-generated Python program (either correct or counterfeit) correctly implements a natural language (NL) specification. These specifications often include input-output examples. We used CoT with majority voting [77, 78] and reported accuracy.

Execution Prediction: The goal is to predict the execution output of a given model-generated Python program on a specific input. We used an execution-based metric for correctness and reported pass@1, the fraction of samples that are correct.

Repair: The goal is to repair a given incorrect model-generated counterfeit program to correctly implement a given natural language specification. The model is not given any execution feedback other than the fact that the program is incorrect.

All three tasks were given to the language model (LM) in a few-shot setting. For GPT-3.5 and GPT-4, all tasks were prompted with chain-of-thought (CoT) [77]. For other models, we used CoT with majority voting ($N = 10$) for correctness checking but not execution prediction, where we found it did not help. These tasks isolate semantic understanding of generated programs rather than merely asking for another unconditional code sample.

Dataset Creation

Each set of samples was curated using a single dataset (such as HumanEval) and model (such as CodeLlama 34B). Each set was balanced and consisted of 5 correct and 5 counterfeit programs for each problem (problems that did not have enough programs were discarded). Overall, across HumanEval, LeetCode, and ODEX, we generated 12 different sets of samples, each consisting of 360 to 1190 programs. For correctness prediction, these datasets were used directly. For execution prediction, we randomly selected input-output examples, removing pairs that required complex arithmetic or execution. We also removed the problem statement and example input-output pairs so the model focused on executing the code. For repair, we used the counterfeit samples in each set and discarded the correct samples.

2.4.2 Can code language models understand counterfeit samples?

In this section, we showed that models struggled to understand counterfeit samples. The figures highlight representative datasets and models; the qualitative conclusions were robust across the evaluated benchmark/model combinations. Unlike prior correctness-checking, execution-prediction, and self-repair evaluations, our evaluation tests a more challenging scenario. Because the negative examples are counterfeit rather than trivially broken, success requires recognizing subtle semantic failures rather than spotting simple syntax errors or obvious test failures.

Correctness Checking

We began by examining whether language models could correctly identify whether a program was correct or counterfeit given the natural language specification. In Fig. 2.13, we plotted the accuracy of CodeLlama 34B, DeepSeek-Coder 33B, GPT-3.5, and GPT-4 on balanced datasets of correct and counterfeit programs for HumanEval and ODEX. For the first three models, the blue bars indicate that correctness checking accuracy was at about 60% for both of these datasets, which was only slightly better than the 50% random-guessing baseline. This indicated that models generally failed to distinguish between correct and counterfeit samples. In addition, by comparing the green and red bars, we observed that the performance of these three models on correct samples was much higher than their performance on counterfeit samples, showing that models were biased towards thinking that counterfeit samples were actually correct. On the other hand, GPT-4 was much better (but not perfect) at this task with an accuracy at around 80% for both datasets. We also observed that, unlike the rest of the models, GPT-4 was *not* biased towards predicting that these samples were correct. However, GPT-4 still faltered around 20% of the time, and we qualitatively analyzed some of these remaining GPT-4 failures in Sec. 3.2.2.

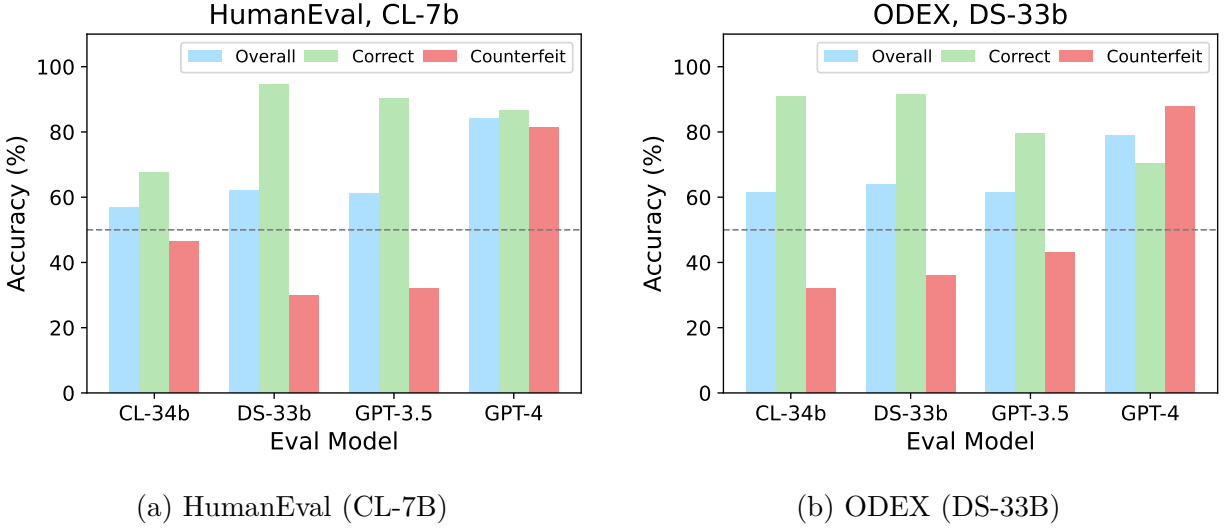


Figure 2.13: Models other than GPT-4 struggle to classify samples as correct or counterfeit and are much better at assessing the correctness of correct samples than counterfeit samples.

Execution Prediction

Next, we assessed the ability of models to predict the execution behavior of counterfeit samples. In Fig. 2.14, we plotted the execution accuracy of the previous four models on two datasets, LeetCode generated by DS-33B and HumanEval generated by CL-34B.

In this task, each sample included a program (correct or counterfeit) P and an input I . The accuracy of the correct samples is shown in the green bars. Because counterfeit programs still passed a subset of tests, we distinguished their execution samples into two groups. We called samples where P passed test I *test-passing* counterfeit samples and the rest *test-failing* counterfeit samples. The execution prediction accuracies of these samples are shown in blue and red, respectively. The purple bars show the proportion of test-failing counterfeit samples where the model actually predicted the output of the correct program. Note that samples counting towards the red accuracy are disjoint from those counting towards the purple accuracy.

Overall, we observed that models had a difficult time distinguishing the semantics of a counterfeit program from their correct counterparts, suggesting they may have had a shallow understanding of program semantics. By comparing the green and blue bars with the red bar, we saw that models failed much more at executing counterfeit programs when the semantics were incorrect. The purple bars provide further evidence of this: models other than GPT-4 frequently executed counterfeit programs as if they had the semantics of a correct program, sometimes even more often than their true semantics (red). For GPT-4, the effect was much less pronounced but still present, as GPT-4 still performed much better on correct and test-passing counterfeits than test-failing counterfeits. Despite having such high performance, it was still confused for a sizable number of test-failing counterfeit samples and predicted the output of the correct program rather than the correct execution result. Overall, because models only saw the programs and not the problem statements, this suggested that they might have been hallucinating the semantics of incorrect programs. This provided further

evidence that models were poor at distinguishing correct programs from counterfeit programs.

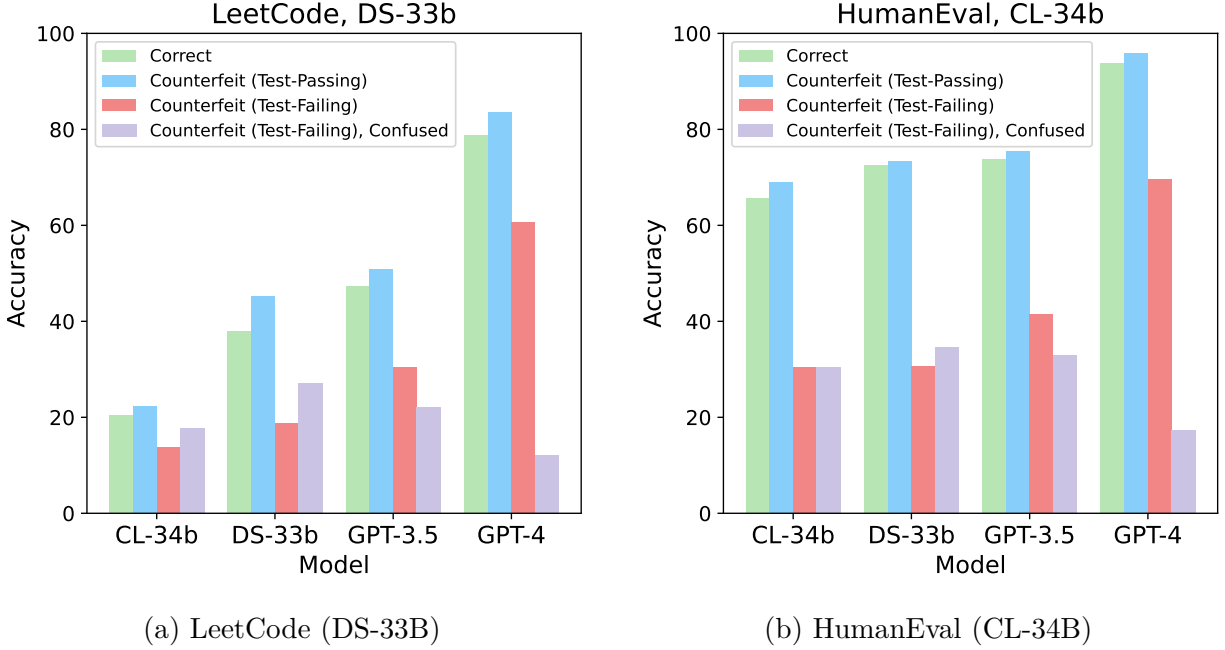


Figure 2.14: Models are much better at executing correct samples than counterfeit samples, and even often execute counterfeit samples as if they were correct.

Repair

Finally, we probed the model’s ability to repair the counterfeit samples it had generated. Although this task may appear to simply boil down to code generation, prior work has highlighted that code understanding forms an integral part of the repair pipeline since achieving good performance hinges on the model’s (in)ability to generate accurate textual explanations of *what* is wrong with the code [79]; as such, self-repair may give us further insight into the model’s capabilities.

Prior work has shown that when given information about *which* unit test failed, many models are capable of repairing incorrect Python programs at rates that exceed their baseline pass rates [79, 80]. In this section, we pressed the model even harder by not giving any execution signal whatsoever, instead simply informing it that the program did not pass; thus, successful repair depended entirely on the model’s own ability to understand the program and its relationship to the specification. Importantly, the success rate of repair must be compared to the baseline pass@1 rate, since a sample can also be “repaired” simply by drawing another unconditional sample from the model.

Fig. 2.15 shows the results for CodeLlama 34B (specifically the instruction-tuned CodeLlama 34B-Instruct, since repair depends heavily on following instructions rather than simply regenerating code from scratch) and DeepSeek 33B when repairing their own programs on HumanEval and LeetCode (respectively). In these figures, each point is the mean success rate of repair for a particular problem; points above the line $y = x$ (which corresponds to a pass

rate equal to that of the simple resampling strategy) thus indicate successful repair, while points below it indicate that the model could not reliably debug and repair the programs. We note that although repair appears somewhat successful with DeepSeek-33B on HumanEval (Fig.), beating out the baseline for 35/81 problems, in all other settings a strong majority of the points lie below the line $y = x$. In other words, the success rate of repair is—for most tasks—significantly below what one would achieve with the simple resampling strategy. This evidence showed that models could not reliably repair counterfeit samples, which suggested that they could not understand why these programs were deemed incorrect.

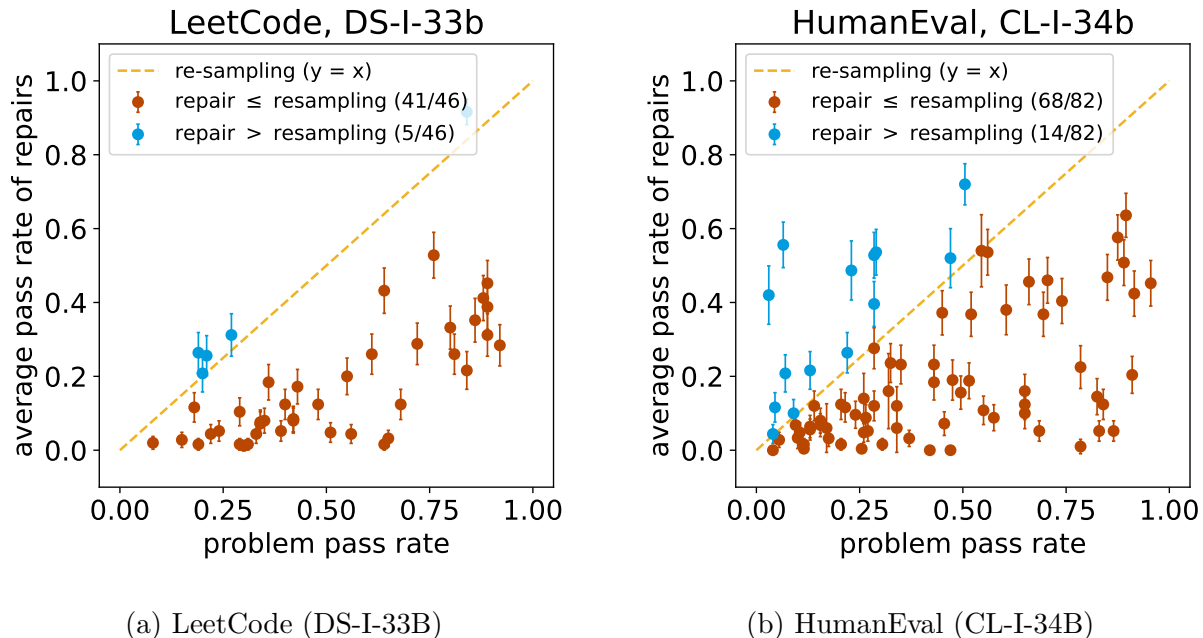


Figure 2.15: In the absence of execution information, repair underperformed resampling in almost all settings. Samples above the $y = x$ resampling baseline have been coloured in blue for clarity. Vertical lines indicate 95% confidence intervals over repair samples.

2.4.3 Discussion

In this work, we brought attention to the counterfeit samples of a code language model: incorrect programs that a model thinks are correct and can pass surface-level correctness checks. While previous works had suggested that code language models could understand and self-correct code, we saw that part of this was because these errors included the blatantly incorrect programs that we saw earlier. When limiting the evaluation to counterfeits, we saw a different picture emerge: models struggled to assess correctness, reason about execution, and repair these programs. Here, we uncovered the confounding factor of distinguishing obviously incorrect programs and counterfeit programs, which led to a blind spot in our understanding of model capabilities.

2.5 LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code

In LIVECODEBENCH, we addressed two problems that made existing code benchmarks increasingly unreliable as code models improved. The first problem is contamination. Benchmarks such as HUMANEVAL, MBPP, and APPS are static: once their problems are public, later models may see them during training. Therefore, data contamination and programming ability can easily be conflated in model evaluation.

The second problem is that these coding benchmarks only measure natural language to code generation. While this task is important, it is only one of many capabilities that programming agents need to exhibit. Coding workflows involve repairing failed solutions, reasoning about the execution of programs, and constructing input-output pairs for testing. To address these two gaps, LIVECODEBENCH combines a live, time-indexed benchmark with a holistic multi-scenario design.

2.5.1 Benchmark Construction

LIVECODEBENCH was designed to continuously collect contest problems from LEETCODE, ATCODER, and CODEFORCES. These platforms periodically release problems with natural-language statements, example input-output pairs, hidden tests, and public contest dates. LIVECODEBENCH recorded the contest date for each problem, which makes the release date part of the benchmark instance rather than external metadata. This allowed us to scroll over different time windows by filtering problems between any pair of dates.

The benchmark also emphasizes problem and test quality. Existing competitive programming benchmarks can include problems that are difficult to evaluate by standard input-output auto-grading, such as problems that permit multiple correct outputs for a single input. Such problems can incorrectly penalize valid solutions. Other benchmarks can contain insufficient tests, adding noise to reported pass rates. LIVECODEBENCH used automated scrapers and filtering heuristics to collect problem statements, tests, ground-truth solutions, and metadata, while excluding problems that are not suitable for reliable input-output evaluation.

Test collection is also an important aspect of the benchmark construction across all four scenarios. When tests are natively available through the platform, LIVECODEBENCH used them directly. Otherwise, it used a language model to synthesize test input generators rather than directly generating individual inputs. In addition, it included adversarial or stress tests where possible.

2.5.2 Effects of Contamination

Contamination in DeepSeek and GPT-4-O. LIVECODEBENCH curates problems released since May 2023. However, DEEPSEEK was released Sep 2023 and GPT-4-O’s official cutoff date is Nov 2023. In Figure 2.16, we showed the performance of these models on LEETCODE problems released in different months from May 2023 and Aug 2024. First, we noticed significant drops in model performance: DS-INS-33B after Aug. 2023 and GPT-4-O after Nov. 2023. These drops aligned with the release/cutoff dates of the models, suggesting

that earlier problems may indeed be contaminated. This trend was consistent across other LIVECODEBENCH scenarios like repair and code execution. We noted that this performance drop occurred primarily for LEETCODE problems, while model performance was relatively smooth across the months for problems from other platforms (Figure 2.17), which suggested different levels of contamination across different sources.

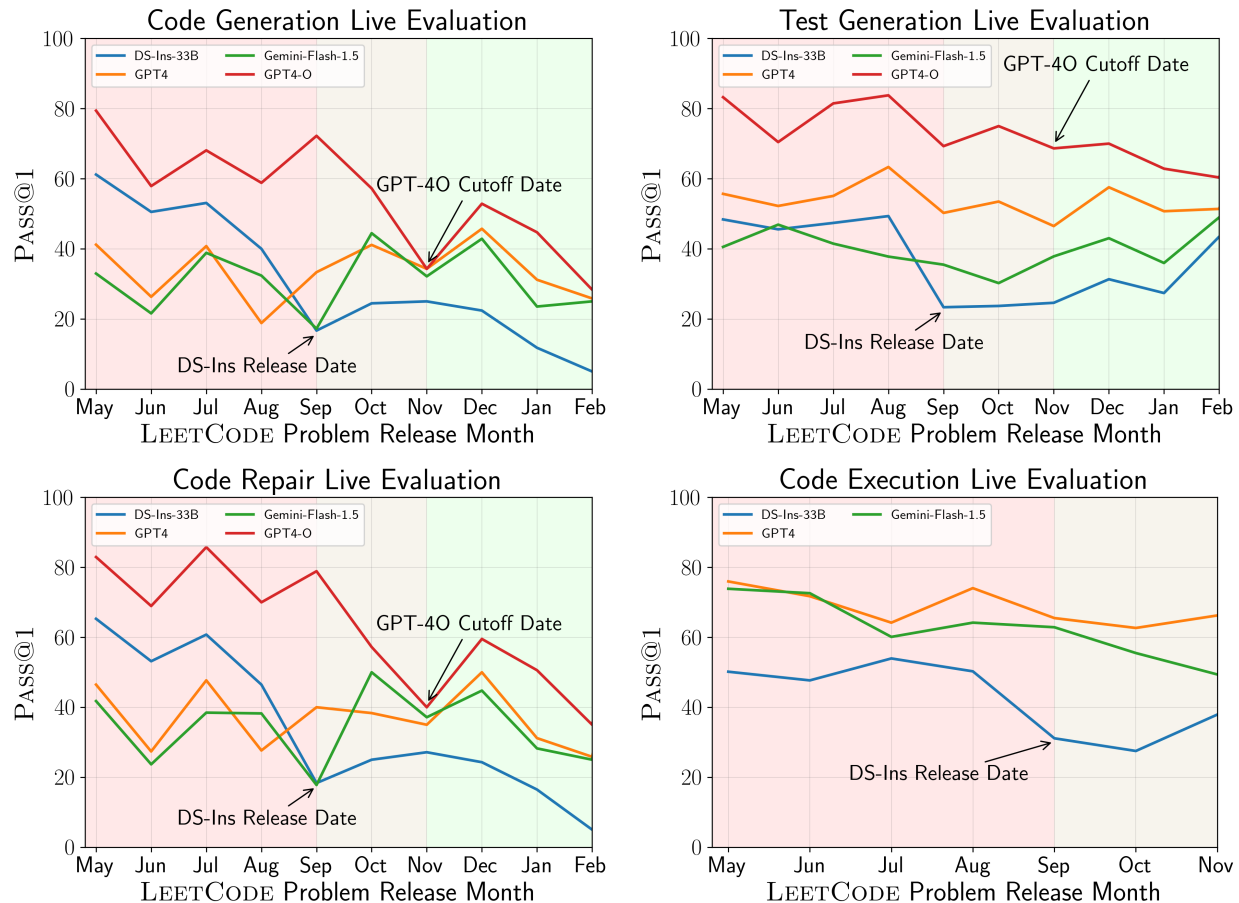


Figure 2.16: Contamination across code generation, test-output prediction, self-repair, and code execution scenarios over time.

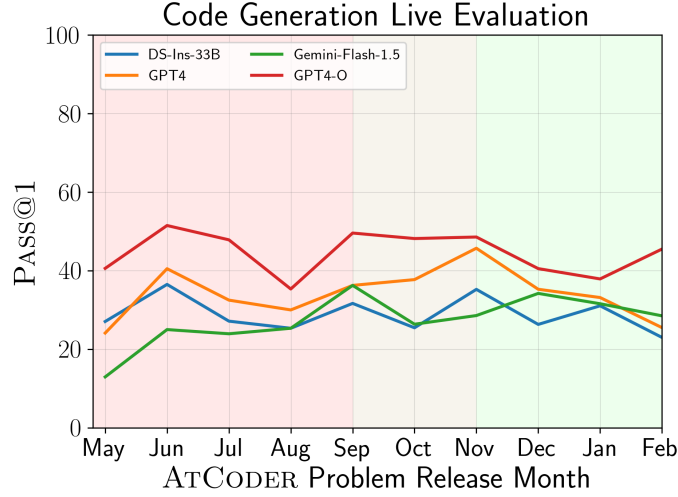


Figure 2.17: Performance on problems released over different months for ATCODER.

Contamination Effects of Other Models: In Figure 2.18, we showed the performance of six models for each month between May 2023 and Aug 2024. GPT-4-TURBO, MISTRAL-L, and CLAUDE-3s were released in November 2023, February 2024, and March 2024, respectively. For these three models, we did not observe significant performance variations across months. On the other hand, we observed that DS-BASE-33B dropped from PASS@1 ~ 60 in May problems to PASS@1 ~ 0 in September LEETCODE problems, suggesting significant contamination. Finally, CODESTRAL achieved PASS@1 36.5 on problems released between May'23 and Jan'24 and PASS@1 28.3 on problems post Jan'24.

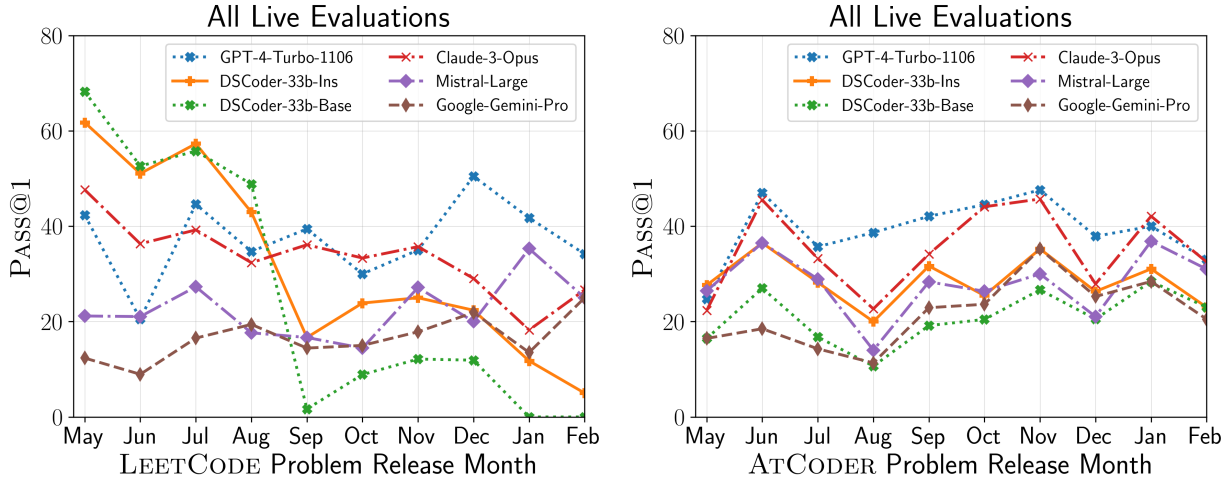


Figure 2.18: Live evaluation over time for various models on code generation in LIVE-CODEBENCH.

2.5.3 Holistic Evaluation

LIVECODEBENCH also evaluated a broader set of coding capabilities. Instead of measuring only code generation, it evaluated four scenarios: code generation, self-repair with execution feedback, code execution, and test-output prediction. These scenarios are intended to isolate capabilities that appear in real world software engineering workflows.

The four tasks, shown in Figure 2.19, are as follows:

- **Code Generation:** This is the standard setup used in previous benchmarks like HumanEval, MBPP, and APPS. The model is given a problem statement consisting of a natural language description and example tests (input-output pairs). The task is to generate a correct solution passing all the tests.
- **Self Repair:** The self-repair scenario is based on previous works that tested the self-repair capabilities of LLMs [81–83]. Here, the model is given a problem statement from which it generates a candidate program. If the program is incorrect, the model is additionally provided with error feedback, either the exception message or a failing test case, and is tasked with generating the fixed solution.
- **Code Execution:** The code execution scenario is based on the output prediction setup used in CRUXEVAL [17]. The model is provided a program snippet consisting of a function (f) along with a test input to the program and is tasked with predicting the output of the program on the input test case.
- **Test Case Output Prediction:** The test case output prediction task is designed to study natural language reasoning and test generation. In this task, the model is given the problem statement along with a test input, and it is asked to generate the expected output of that input.

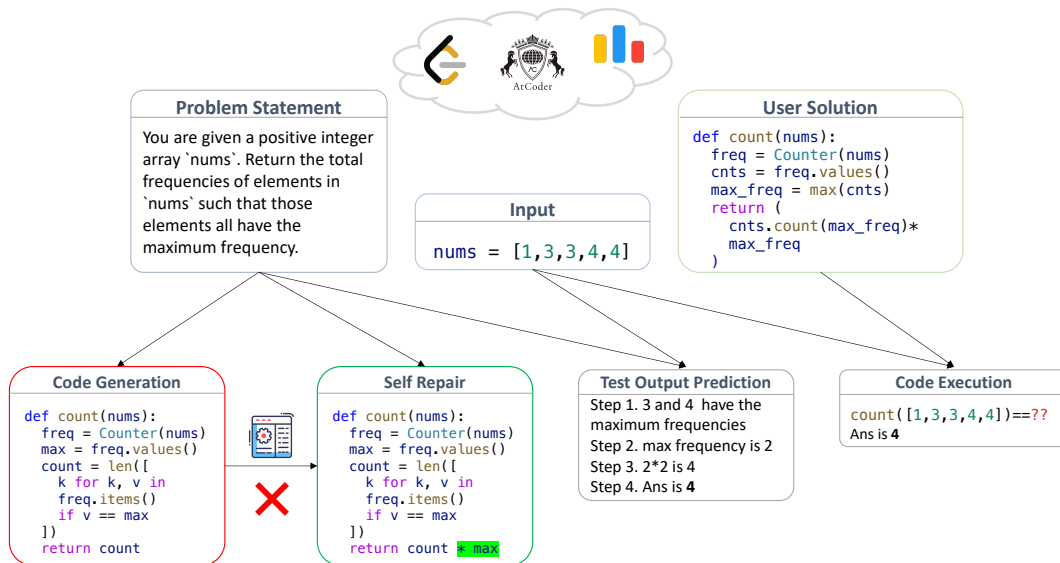


Figure 2.19: Overview of the four scenarios present in LIVECODEBENCH.

In [Figure 2.20](#), we showed a high-level view of how various models performed across the four scenarios. We observed that relative model differences changed across code generation, self-repair, code execution, and test-output prediction.

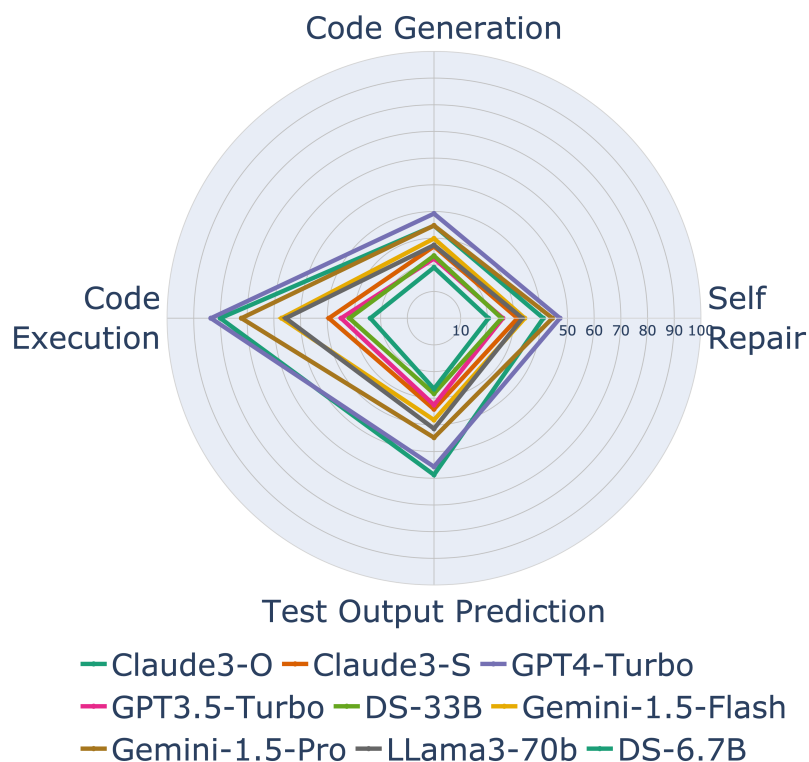


Figure 2.20: Scenario-level performance profiles in LIVECODEBENCH.

Next, in [Figure 2.21](#), we showed the full results of all models across each scenario. This revealed that relative model performance was related across scenarios, with high correlations between PASS@1 scores. However, there were changes in relative gaps. After self-repair, GPT-4-TURBO increased its lead over GPT-4. CLAUDE-3-OPUS and MISTRAL-L performed well on the more reasoning and CoT-based scenarios, code execution and test-output prediction.

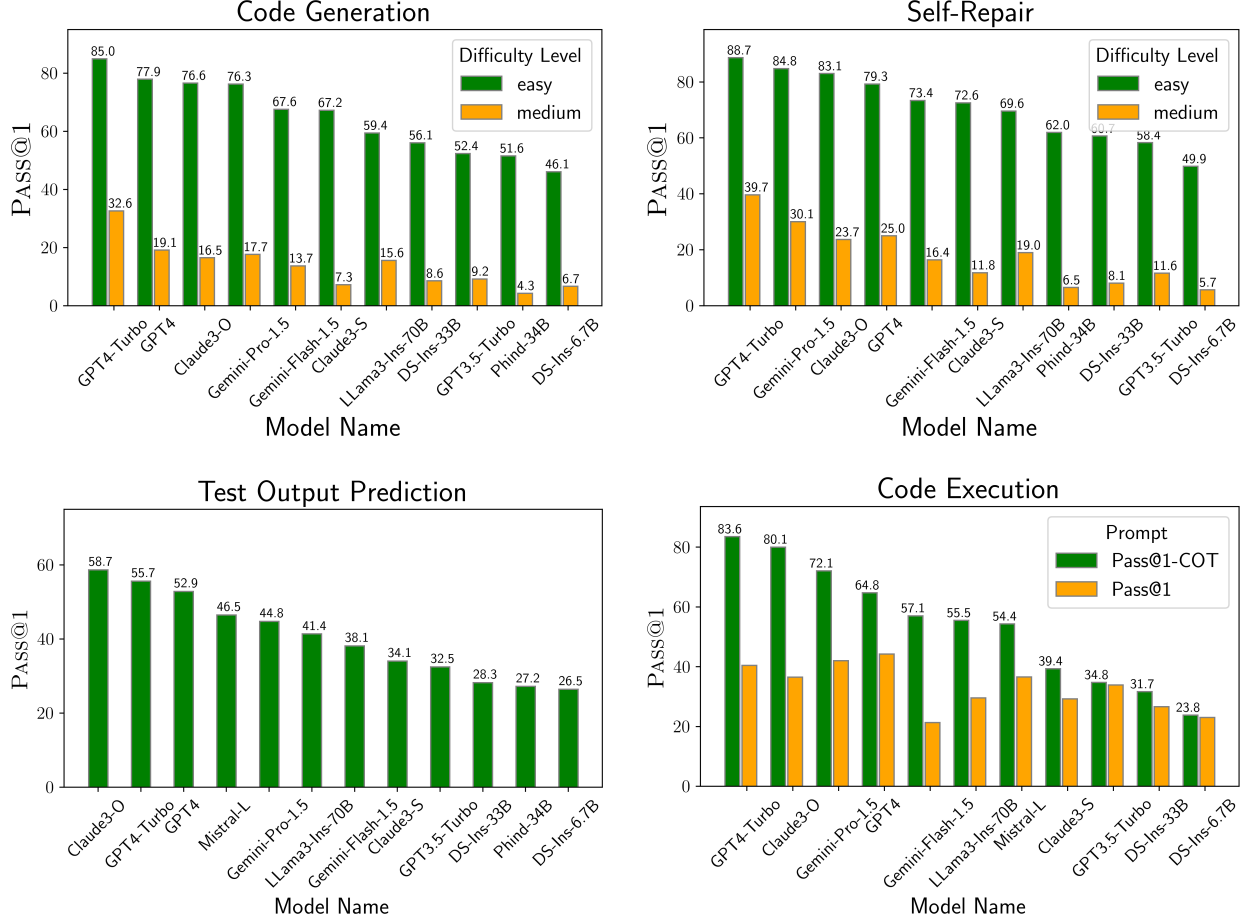


Figure 2.21: Model performances across the four scenarios available in LIVECODEBENCH.

2.5.4 Discussion

In LIVECODEBENCH, we showed how crucial it is to account for data contamination when evaluating code models. As we saw, model performance often drops sharply on problems released after a model’s training cutoff date. Failing to take this bias into account can lead to misleading comparisons between models, as one model may have seen problems that another model has not. Through our holistic scenario design, we also saw that there were important differences in the ability of frontier language models to perform code generation, repair, execution, and test-output reasoning. Overall, LIVECODEBENCH provided a clearer and more holistic picture of the algorithmic coding capabilities of language models while setting a standard for future benchmarks to control for contamination biases, reinforcing our lesson: focusing on the right metrics without bias is crucial.

2.6 Conclusion

An important aspect to highlight is that the metrics we focused on needed to change over time to adapt to the changing landscape of frontier model development. When we developed

CRUXEval, the standard coding evaluations were single-function static benchmarks. The community had not thought deeply about capabilities beyond code generation, and CRUXEval helped signal the importance of broader capabilities such as code execution. As discussed later in Section 4.1, this mattered not only because it revealed a missing ability at the time, but also because it helped make execution behavior a target for later benchmarks and training recipes. In particular, CRUXEval helped motivate work on training models with execution traces, one of the ways code LLMs improved in the years that followed.

When we later saw that there were several biases distorting how we interpreted accuracy, we turned towards eliminating biases through works like the Counterfeit Conundrum and LiveCodeBench. The contamination-free evaluation methodology from LiveCodeBench quickly gained wide adoption, as discussed in Section 4.2. Accurate model comparisons became more important as capabilities scaled up. In the following year, models quickly improved and were increasingly adopted by software engineers, making it clear that single-function static benchmarks were no longer reflective of real-world use. This shift motivated SlopCodeBench, where we focused on iterative evaluation and code-quality metrics that better align with long-horizon developer workflows and the priorities of software engineers. Overall, the metrics that we designed evolved alongside the various shifts at the frontier.

In each of the five preceding sections, we saw how important it is to focus on the right metric and eliminate confounding sources of bias. By deciding on a metric, we control what we can learn. The first three evaluations showed the presence and absence of particular abilities: models were beginning to execute simple code snippets, could not maintain code quality over long iterative trajectories, and were often better at finding the right answer to inequalities problems than at producing sound stepwise proofs. The final two evaluations highlighted biases that changed how we perceived the results of our measurements. The next chapter looks at subtle model behaviors exposed by deeper analysis into these metrics.

Chapter 3

Looking Beyond the Score Reveals Hidden Nuances

In Chapter 2, we saw the importance of defining metrics that capture the properties we actually care about while controlling for confounding biases. In this chapter, we focus on the importance of looking beyond the score and analyzing the underlying success and failure modes of models. We present four examples showing how deeper analysis revealed hidden nuances in the behavior of language models and advance our scientific understanding.

This chapter first discusses insights and analyses derived from three of the benchmarks introduced in the previous chapter. In Section 3.1, we dive deeper into the results from CRUXEval. For example, by comparing scores on CRUXEval with code generation benchmarks like HumanEval, we discovered that a cluster of distilled models (e.g. WizardCoder) performed much better on HumanEval compared to CRUXEval than their base models, suggesting distillation may not fully transfer general programming abilities. In Section 3.2, we analyzed whether counterfeit samples have different properties depending on the difficulty of the problem or the strength of the model generating the samples. In Section 3.3, we returned to the LiveCodeBench paper to analyze overfitting to HumanEval, the role of post-training, and the gap between open and closed models. Finally, in Section 3.4, we introduce LINC, a neuro-symbolic approach to performing logical reasoning. By analyzing the behavior of LINC compared to traditional chain-of-thought, we discovered that LINC and chain-of-thought had distinct failure modes, suggesting opportunities to synthesize the two approaches.

3.1 CRUXEval: Generalization, Chain of Thought, Fine-Tuning, and Qualitative Failures

3.1.1 Quantitative Analysis

Correlation between scores on HumanEval and CRUXEval: After the release of Code Llama’s model and GPT-3.5 and GPT-4’s APIs, there have been many creative efforts to take data distilled from GPT models and use them to train more powerful code models such as WizardCoder [45], Phi-1 [46], Phi-1.5 [46], and Phind [48]. For example, WizardCoder 34B started with the Code Llama 34B base model and improved the HumanEval pass@1 score

from 53.7% to 73.2%, a significant and impressive achievement. There remains curiosity about whether these models show more general improvements in other aspects of programming or code understanding [84]. We measured this through CRUXEVAL.

In Fig. 3.1, we plotted reported HumanEval scores (we did not reproduce them ourselves) against scores on CRUXEVAL. Indeed, we spotted some interesting outliers: when comparing the distilled models WizardCoder 34B and Phind 34B to CODE LLAMA 34B, we saw that the distilled models scored over 20% more than Code Llama on HumanEval but did not show this drastic improvement when evaluated on both input and output predictions. In addition, the Phi-1 model outperformed most of the bigger models on HumanEval, but performed among the worst of all our evaluated models on CRUXEVAL. Overall, this suggested that models optimized for the HumanEval task by distilling data from GPT-3.5 and GPT-4 (WizardCoder, Phind, Phi) may not have learned other code reasoning capabilities along the way. On the other hand, for models such as StarCoder, Mistral, CodeLlama, and DeepSeek-Base, we still saw a positive trend between HumanEval score and CRUXEVAL score, suggesting that code generation and execution/understanding abilities are correlated.

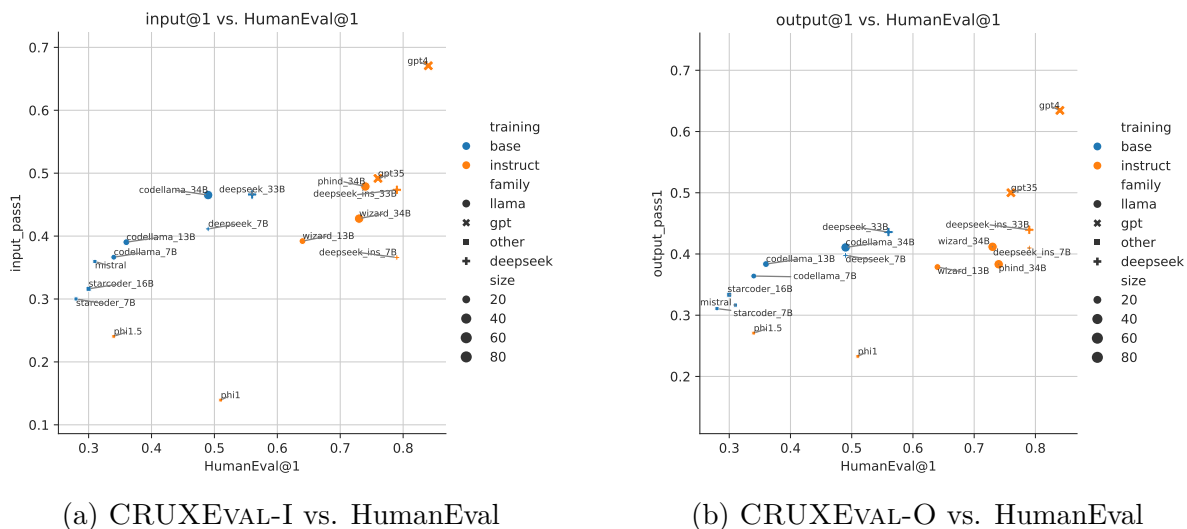


Figure 3.1: Correlation between HumanEval pass@1 scores and CRUXEVAL-O pass@1 scores

Base models show a weak correlation between HumanEval and CRUXEVAL. For HumanEval, distilled models (WizardCoder, Phind, Phi) significantly beat their base models, but for CRUXEVAL, no distilled model performs significantly better than CODE LLAMA 34B.

Relationship between input prediction and output prediction: In Fig. 3.2a, we compared the input prediction and output prediction pass@1 scores with each other. Conceptually, the two tasks seem relatively different: output prediction is directly testing code execution ability, while input prediction requires a higher-level understanding of the code’s functionality. However, we discovered that there was a strong correlation between their performance. This suggests the hypothesis that performance on relatively distinct

coding-related tasks may be closely correlated. In addition, we saw a relatively clear impact of scaling the model size on our two tasks.

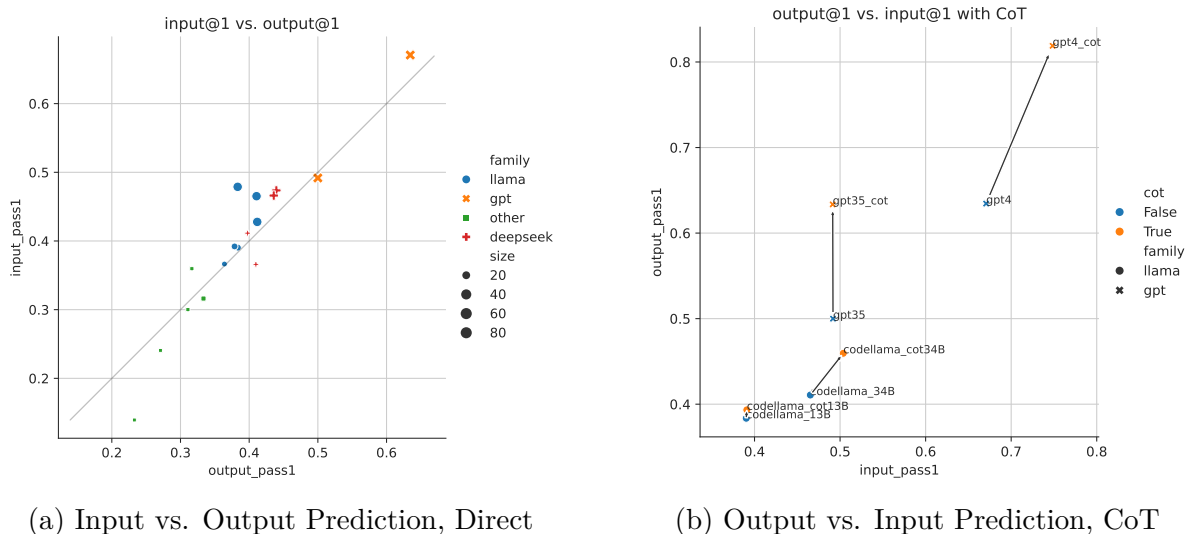


Figure 3.2: Correlation between Input and Output Prediction Scores, with and without CoT

With the potential exception of GPT models, performance on CRUXEVAL-I and CRUXEVAL-O seem to be very correlated. As the tasks seem relatively different, this suggests that the code reasoning capabilities of models may generalize from task to task.

Confusion matrix/error correlation for different models. Fig. 3.3 shows the pairwise correlation of pass@1 scores for each pair of models. The correlation is chosen based on its highest signal among cosine distance, Spearman and Kendall. The middle section of “open”-ish models (StarCoder, Code Llama, DeepSeek, etc.) are strongly correlated with each other. Strong correlations are seen between sizes of the same model, between models of the same size, and between instruct and base (PHIND 34B, WIZARD 34B vs. CODE LLAMA 34B). CoT results also tend to have strong correlations with other CoT results, even GPT-4 vs Llama 13B. For the output task, DEEPSEEK forms a small sub-cluster of especially strong associations.

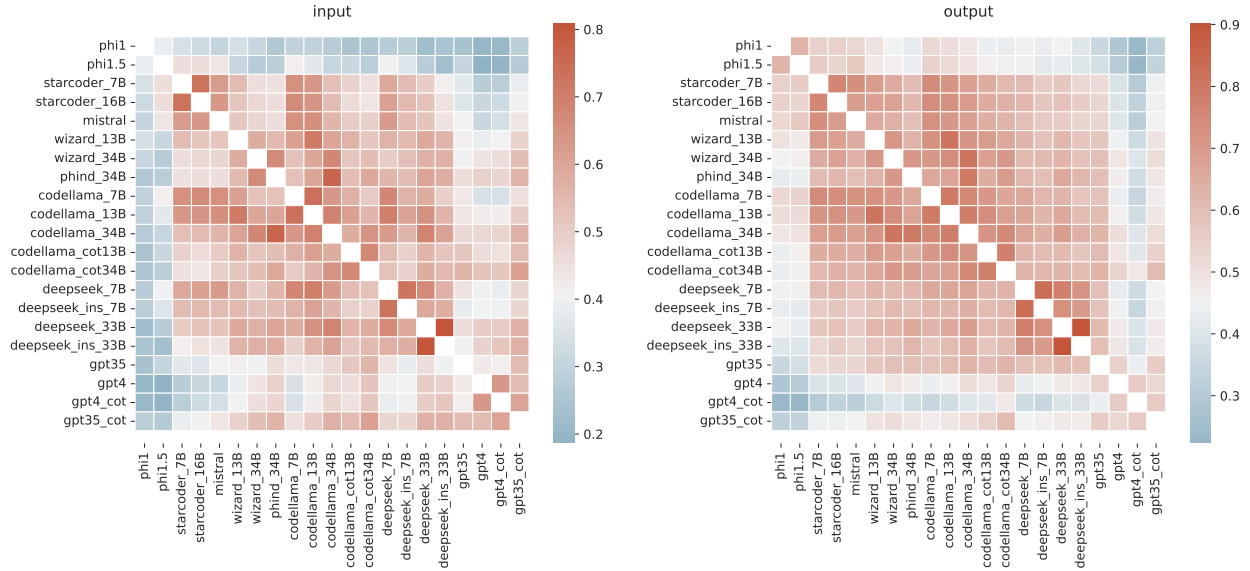


Figure 3.3: Correlation between predictions on input (left) and output (right)

Looking at model predictions, strong correlations are seen between sizes of the same model, between models of the same size, and between instruct and base models. Although what is hard for a better model tend to be hard for worse models on average, worse models succeeded on some examples where the better models fail completely.

3.1.2 Chain of Thought Prompting

Next, we evaluated how the popular chain-of-thought (CoT) prompting method [77] affected the performance of Code Llama, GPT-3.5, and GPT-4 models on CRUXEVAL. All results were reported using $N = 10$ samples other than CodeLlama 13B and 34B without CoT, which were reported with $N = 100$ samples. As before, pass@1 was reported with $T = 0.2$ and pass@5 with $T = 0.8$.

Impact of CoT: We began by focusing our attention on the pass@1 scores of models with and without CoT. In Fig. 3.2b, we plotted the input and output prediction scores of each model with and without CoT. First, GPT-4 benefited significantly more than other models. Second, output prediction boosts were generally larger than input prediction. In fact, CoT did not seem to improve Code Llama 13B and GPT-3.5 performance on input prediction. This is intuitive, as input prediction involves a more difficult reasoning task, while output prediction only requires executing the program step by step.

CoT helps Code Llama 34B and GPT-4 on both input and output prediction, GPT-3.5 on only output prediction, and Code Llama 13B on neither task. CoT also leads to larger boosts on output prediction than input prediction. GPT-4 benefits significantly more from CoT than other models, achieving the highest pass@1 of 74.8% on input prediction and 81.9% on output prediction but still far from acing the benchmark.

CoT widened the gap between pass@5 and pass@1 scores: In Fig. 3.4, we plotted the pass@5 scores against the pass@1 scores for all models. For models without CoT (shown in blue), there was a positive correlation between pass@1 and pass@5 scores. For models with CoT (shown in orange), we saw an increase in the gap between pass@5 and pass@1 scores. This suggests that CoT increases generation diversity, which can improve the best-of-several score even when the first sample remains unreliable.

Because CoT increases the diversity of generated inputs and outputs, models with CoT see a larger gap between pass@1 and pass@5 score compared to models without.

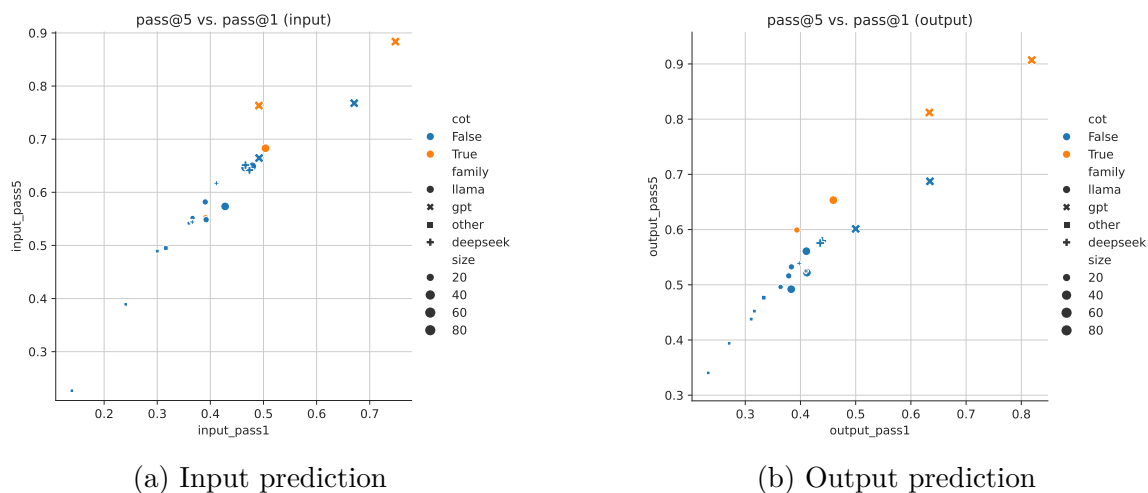


Figure 3.4: pass@5 score vs. pass@1 score with and without CoT

Predictions of CoT vs. Base Model: In Fig. 3.5, we showed a confusion matrix over samples to better understand the correlations between direct output predictions and CoT predictions. For CodeLlama 13B, 34B, and GPT-3.5, we observed a large number of samples where direct prediction succeeded but CoT failed. However, with GPT-4, we observed that there were relatively few samples where this was the case.

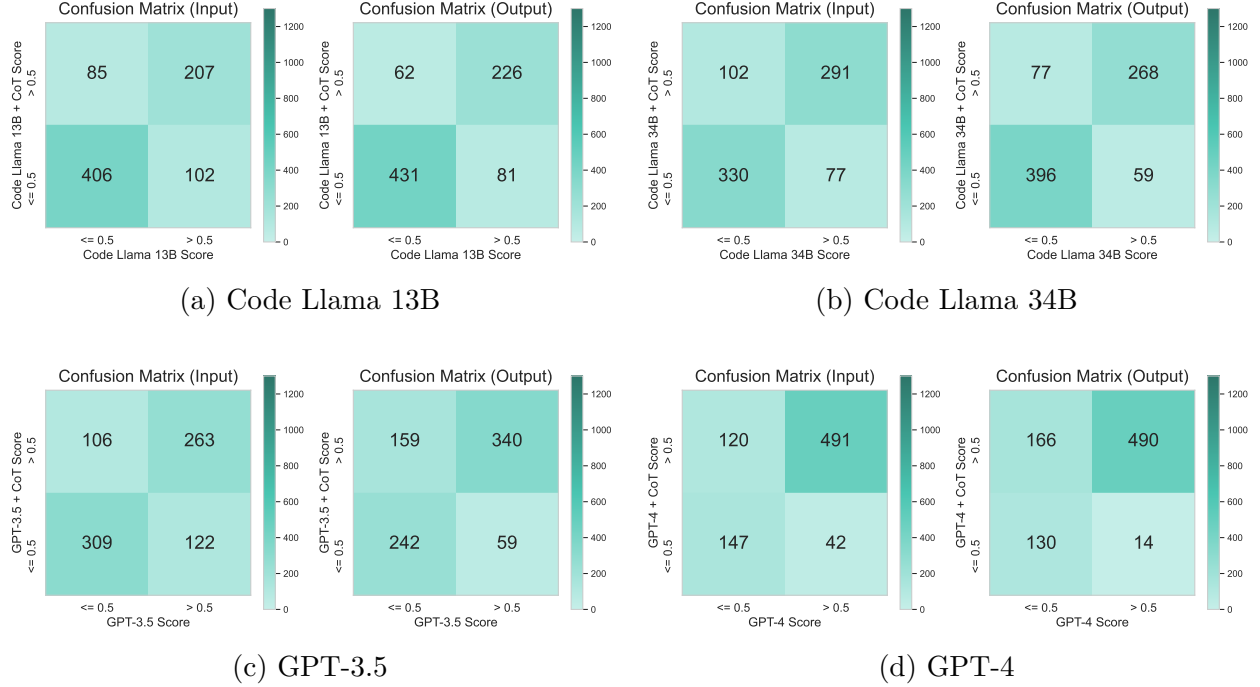


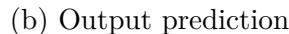
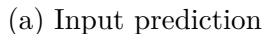
Figure 3.5: Confusion Matrix of Direct Prediction vs. CoT Prediction ($T = 0.2$)

While CoT generally improves performance overall, there are many individual samples where CoT actually hurts the prediction accuracy for Code Llama 13B/34B and GPT-3.5 on both input and output prediction. For GPT-4, CoT generally improves individual sample accuracy, more so for output prediction than for input prediction.

3.1.3 Fine-tuning Experiments

Next, we did a preliminary analysis to understand the effect of simple fine-tuning schemes on CRUXEVAL performance. We fine-tuned CODE LLAMA 34B on nearly 140K samples of Python functions distilled with the procedure outlined in Sec. 2.1.1, without filtering. We performed weak decontamination, only removing samples where both the function and input-output pairs match samples in the benchmark.

In particular, we finetuned on a mixture of 50% samples where the function was not in the benchmark and 50% samples where the function was in the benchmark but input-output pairs were not, a very liberal setup. The training and testing accuracy over time is shown in Fig. 3.6. Despite finetuning on programs very similar to the benchmark, we still observed a plateauing effect in the test accuracy, suggesting that our execution tasks may be too difficult to learn from this simple fine-tuning scheme.



After fine-tuning on samples very similar to those in our benchmark, Code Llama 34B can match the performance of GPT-4 on both input and output prediction. However, accuracy plateaus at under 70% for both tasks, so simple finetuning is far from solving the benchmark.

All models except GPT4 had over 50% failure rates, showing they could not do simple executions. In this section, we focused on GPT4 with CoT and verified that the remaining 20% failures were due to the model, were consistent, and were indeed on *simple* programs.

```
def f(string, sep):
    cnt = string.count(sep)
    return((string+sep) * cnt)[::-1]
assert f('caabcfcabcf', 'ab') == '
bacfbacfbcaabcbacfbacfbcaac'
# GPT4+CoT preds:
# f('baa', 'c')
# f('cba', 'f')
# 'bacfbacbfcaabcbacfbacbfcaac'
# 'bacfbacfbcaabcbacfbacfbcaabc'
```

```
def f(prefix, s):
    return str.removeprefix(prefix, s)
assert f('hymi', 'hymifulhxxhznpyihyf') == 'hymi'
# GPT4+CoT preds:
# input
# f('p', 'phymi')
# f('', 'hymi')
# output
# 'fulhxxhznpyihyf'
# 'fulhxxhznpyihyf'
```

63

```
def f(text, search_string):
    indexes = []
    while search_string in text:
        indexes.append(text.rindex(search_string))
        text = text[:text.rindex(search_string)]
    return indexes
assert f('ONBPICJOHRHDJOSNCPNJ9ONTHBQCJ', 'J') ==
[28, 19, 12, 6]

# GPT4+CoT preds:
# input
# f("0000123000012300001230000123", "123")
# f('bbbbbbabbbbbabbbbbabbbbbab', 'a')
# f("abcxxxxxabcxxxxxabcxxxxxabc", "abc")
# output
# [23, 13, 8, 5]
# [25, 18, 15, 11, 6]
# [7, 10, 14, 18]
```

Listing 3.3: GPT-4 CoT failures where solutions requires counting to 30

```
def f(L):
    N = len(L)
    for k in range(1, N//2 + 1):
        i = k - 1
        j = N - k
        while i < j:
            # swap elements:
            L[i], L[j] = L[j], L[i]
            # update i, j:
            i += 1
            j -= 1
    return L
assert f([16, 14, 12, 7, 9, 11]) == [11, 14, 7, 12, 9, 16]

# GPT4+CoT preds:
# f([16, 9, 12, 7, 14, 11])
# f([16, 9, 7, 12, 14, 11])
# [11, 9, 7, 12, 14, 16]
# [11, 9, 7, 12, 14, 16]
```

Listing 3.4: GPT-4 CoT failure, cannot easily tell the answer without running the loops

Listings 3.5 and 3.6 show GPT-4 CoT failures for output prediction only. In Listing 3.6, the model fails because it concludes that 6173 is not less than 1000. Small perturbations like changing to $0 < \text{num}$ and $\text{num} < 1000$ or changing the strings also failed. Both problems only have 2 possible answers and other models sometimes get them correctly whereas GPT4 CoT is consistently wrong. We manually tested scratchpad [85] style prompts, which failed in the same way as regular CoT.

```
def f(text, suffix):
    if suffix == '':
        suffix = None
    return text.endswith(suffix)
assert f('uMeGndkGh', 'kG') == ??
# GPT-4 CoT: True
# should be False
```

Listing 3.5: GPT-4 CoT output

```
def f(num):
    if 0 < num < 1000 and num != 6174:
        return 'Half Life'
    return 'Not found'
assert f(6173) == ??
# GPT-4 CoT: 'Half Life'
# should be 'Not found'
```

Listing 3.6: GPT-4 CoT output

Failures of GPT-4, Input Prediction: Here are two simple failures on input prediction. Listings 3.7 and 3.8 show input prediction failures for concise and simple Python programs with and without CoT, respectively.

```
def f(text, repl):
    trans = str.maketrans(text.lower(), repl.lower())
    return text.translate(trans)
assert f('lower case') == 'lower case'

# GPT4 CoT: 'lower case', 'ow'
# could be 'lower case', 'lower case'
```

Listing 3.7: GPT-4 CoT input

```
def f(text):
    string = ''
    for char in text:
        string += char + char.lower()
    return string
assert f('LAK') == 'llaallaakk'
# GPT-4 CoT: 'LAK'
# should be 'lalak'
```

Listing 3.8: GPT-4 CoT input

Other GPT-4 Failures: Finally, we concluded with a set of six relatively simple string manipulation tasks that we discovered that GPT-4 failed on. We suspect the errors could

be partially due to tokenization. These examples are enough for the thesis point: even the strongest model we tested can fail consistently on small execution problems whose semantics are unambiguous.

- What is a string containing 'a' three times, 'b' three times, 'c' twice, 'd' $\hookrightarrow$ three times, and 'z' twice?
- In Python, what is " BaB ".rfind(" B ")?
- In Python, if I have a string s = 'iabnm~~~~~', what is s[1::2]?
- In Python, what is "+".join(['*', '+', 'n', 'z', 'o', 'h'])?
- In Python, if text = "!123Leap and the net will appear" and res = 123, what is $\hookrightarrow$ text[len(str(res)):]?
- What is "pomodoro".replace("or", "pomodoro")?

3.1.5 Discussion

By looking deeper at CRUXEval, we saw that high scores on familiar benchmarks can be ambiguous. Sometimes they indicate general programming abilities, but other times they may be due to task- or benchmark-specific adaptation. In particular, distilled models substantially improved on HumanEval without showing the same gains on CRUXEval, suggesting that distillation may not fully lead to transfer between different tasks. We also saw the potential and limitations of chain-of-thought prompting and fine-tuning. CoT helped GPT-4 more than the other models, but it sometimes hurt individual predictions for weaker models. Fine-tuning substantially improved Code Llama, but still did not solve the benchmark. These results made CRUXEval useful not just as a score, but as a way to understand which interventions improved general code reasoning.

3.2 Counterfeit Conundrum: The Nature of Counterfeit Samples

3.2.1 Do counterfeit samples from different models or problems differ significantly?

Is it easier for models to understand counterfeit samples from problems it finds easier?

Intuitively, if a given programming problem is easy for a model to solve (meaning it has a high pass@1 rate), we might believe models understand how to solve that problem. If that is the case, then models should be able to better understand both correct and counterfeit samples for that problem. To test if this was true, we bucketed problem difficulties into easy, medium, and hard by a model's pass@1 on that problem. We then calculated the average correctness checking, verification, and repair accuracy for each of the tasks for each problem difficulty bucket. Fig. 3.7 shows a representative subset of these results on HumanEval.

Surprisingly, we found that 1) correctness checking accuracies were relatively uncorrelated with problem difficulty, while 2) execution ability and the success rate of repair exhibited

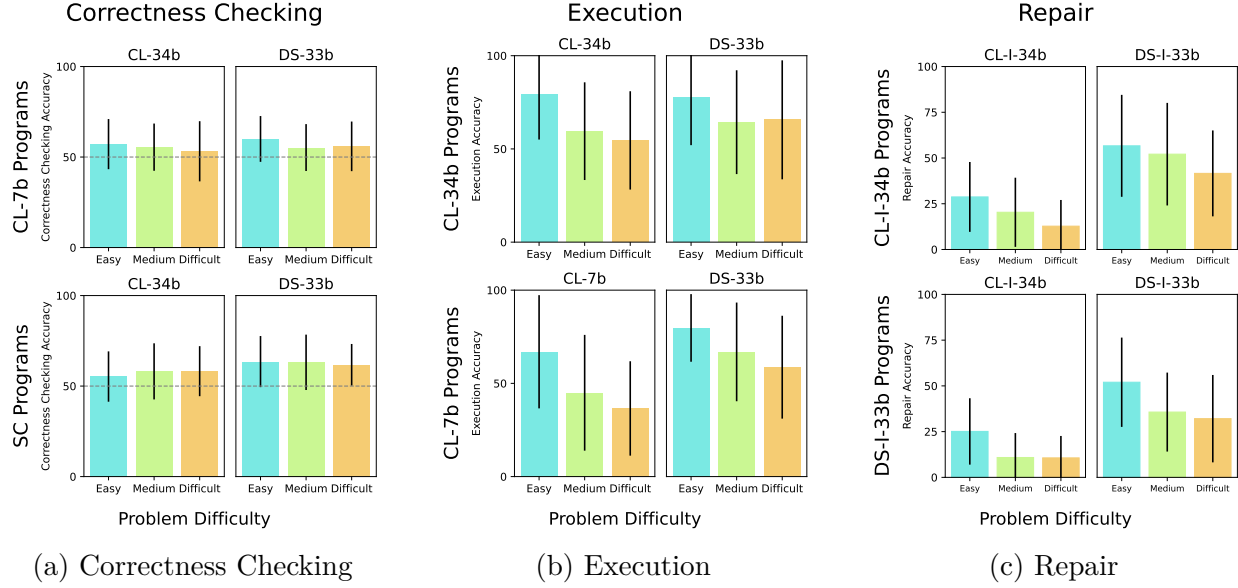


Figure 3.7: Mean accuracy of correctness checking, execution, and repair on HumanEval (error bars are 1 std). A problem’s difficulty is determined by the pass@1 of the evaluation model. In general, correctness checking accuracy is uncorrelated with problem difficulty, while both execution and repair accuracies are only very weakly correlated.

a modest amount of correlation with problem difficulty. We found that these trends were generally robust across HumanEval, LeetCode, and ODEX, although the amount of correlation exhibited in the repair task varied (but was, at best, modest).

To get a more precise measurement of these relationships, we calculated the Pearson correlation between generation performance and each of correctness checking, execution prediction, and repair performance across problems. The resulting histogram showing the distribution of correlations for each task is shown in Fig. 3.8.

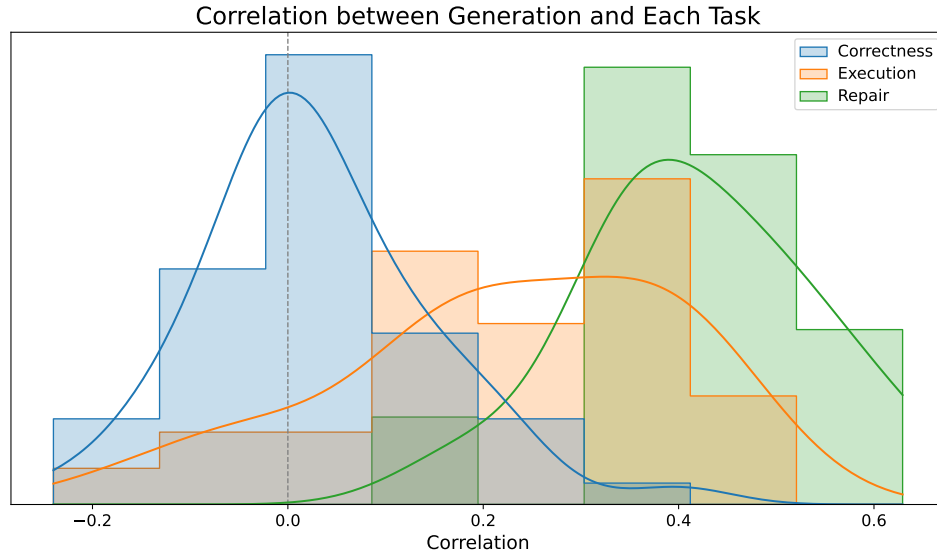
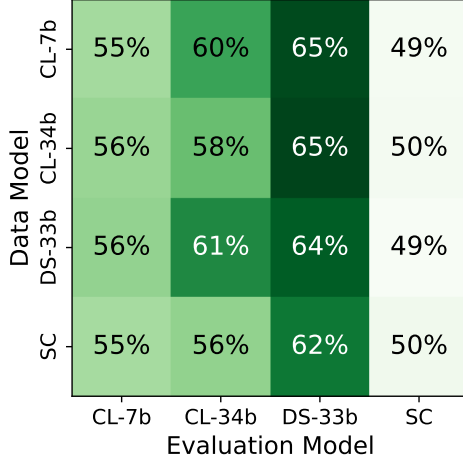


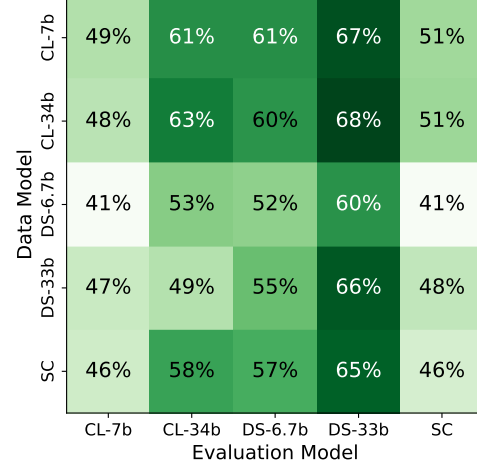
Figure 3.8: Correlation between generation and each of our three tasks

Do models perceive their own samples differently?

For a given model, its counterfeit samples had a high enough log-likelihood to be generated by the model, so one may hypothesize that models might have a harder time than other models at distinguishing their own counterfeit samples. In Fig. 3.9, we plotted heatmaps showing the performance of various models on datasets generated by other models for the correctness checking task (left) and execution prediction task for HumanEval (right). For both tasks, the relative performance of different models was similar across datasets, and we found no evidence that models faltered more on their own samples. This suggests that counterfeit samples may be general: those from one model are generally difficult for other models to understand as well.



(a) Correctness (ODEX)

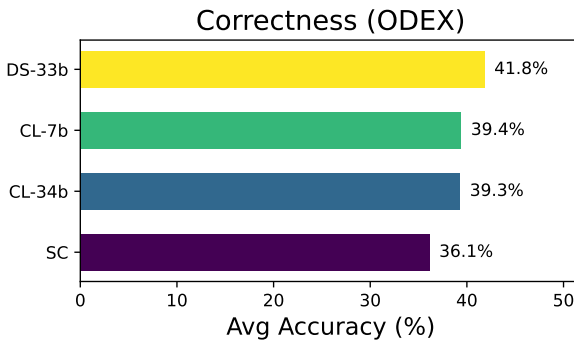


(b) Execution (HumanEval)

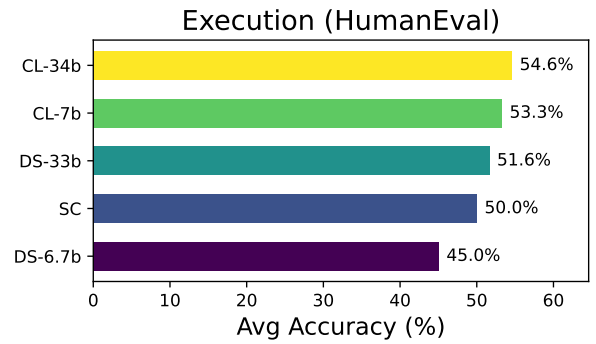
Figure 3.9: Heatmap of accuracies for correctness checking and execution prediction. Models do not seem to perceive their own generations differently from those of other models.

Do stronger models generate harder counterfeit samples?

One might also expect that counterfeit samples of stronger models are harder to verify than those of weaker models, as stronger models are less likely to generate obvious mistakes. In Fig. 3.10, we compared the average scores of two tasks on counterfeit samples for datasets generated by stronger (DS-33B, CL-34B) and weaker (CL-7B, SC) models. Note that this is the same as Fig. 3.9 with each row aggregated and filtered to only include counterfeit samples. Since there did not seem to be a significant difference between the difficulties, models of all strengths could be used to generate counterfeit samples that were challenging for models to understand.



(a) Correctness (ODEX)



(b) Execution (HumanEval)

Figure 3.10: Accuracy of correctness and execution tasks on counterfeit samples. Both strong and weak models are able to generate difficult counterfeit samples.

3.2.2 Qualitative Analysis

In general, most counterfeit programs fall into one of three broad categories: (1) error in algorithmic design or implementation, (2) incorrectly understanding or completely ignoring details in the specification, and (3) failing to address corner cases in the input space.

Failure Modes on Correctness Checking

In Sec. 2.4.2, we saw that GPT-4 was significantly more performant than other models at both correctness checking and execution prediction. Yet, we still found a significant fraction of failures. In this section, we took a closer look at the performance of GPT-3.5 and GPT-4 to understand where these models still fell short. Through a manual inspection of examples, we uncovered three main failure modes for verifying the correctness of counterfeit samples and present one example of each. For conciseness and readability, some of the problem statements have been altered from their original form.

1) The model does not catch misunderstood or ignored details in the specification: Sometimes, the verification model doesn't catch important specification details that are misunderstood or ignored by the counterfeit sample.

For example, GPT-3.5 failed to find the reasoning error in the below counterfeit sample because it appeared to have misunderstood the definition of “corresponding closing bracket”. However, when asked to implement a function for the specification, it correctly understood that the corresponding closing bracket must come after the opening bracket, highlighting an inconsistency between generation and correctness checking.

```
def correct_bracketing(brackets: str):
    """ brackets is a string of "<" and ">". return True if every opening bracket
    ↪ has a corresponding closing bracket.
    >>> correct_bracketing("<<>>")
    True
    >>> correct_bracketing("><<>")
    False
    """
    opening = brackets.count("<")
    closing = brackets.count(">")
    return opening == closing
```

GPT-3.5: The code appears to be correct. It counts the number of opening brackets ("<") and closing ↪ brackets (">") in the input string and returns True if the counts are equal, indicating that every ↪ opening bracket has a corresponding closing bracket.

2) The model fails to catch subtle implementation mistakes: Often, counterfeit samples follow a generally correct algorithm design but contain a subtle implementation mistake that models do not catch. Below, the `if num == target` check should consider whether the remaining digits in `square[index:]` sum to 0 for this implementation to be correct, but GPT-4 missed this subtle detail.

```
"""Given a positive integer n, return the punishment number of n, defined as the
    ↪ sum of the squares of all integers i such that 1) 1 <= i <= n and 2) The
```

↪ decimal representation of $i * i$ can be partitioned into contiguous
 ↪ substrings such that the sum of the integer values of these substrings
 ↪ equals i ."""

```

def punishmentNumber(n: int) -> int:
    def check(num, target, index, square):
        if num == target:
            return True
        if index == len(square):
            return False
        cur = 0
        for i in range(index, len(square)):
            cur = cur * 10 + int(square[i])
            if cur > target:
                break
            if check(num + cur, target, i + 1, square):
                return True
        return False

    res = 0
    for i in range(1, n + 1):
        square = str(i * i)
        if check(0, i, 0, square):
            res += i * i
    return res

# GPT-4 CoT verdict: Correct

```

3) The model forgives an identified error: A third failure mode occurs when the model correctly reasons that the code is incorrect, but then makes up a justification for the error and judges the code as correct. In the counterfeit sample below, the code fails to consider capital consonants. While GPT-4 correctly identifies this issue, it proceeds to state that assuming that concern is not a real issue, the code is correct. However, the implementation fails one of the examples.

```

def select_words(s, n):
    """Given a string s and a natural number n, return a list of all words from
    ↪ string s that contain exactly n consonants, in the order these words appear
    ↪ in the string s.
    Examples:
    select_words("Mary had a little lamb", 4) ==> ["little"]
    select_words("Mary had a little lamb", 3) ==> ["Mary", "lamb"]
    """

    # convert string to list
    word_list = s.split()
    consonants = "bcdghjklmnpqrstvwxyz"
    # filter out all words that contain n consonants
    return [word for word in word_list if sum(c in consonants for c in word) == n]

```

↪]

GPT-4: ... However, there is a potential issue with case sensitivity. The ‘consonants’ string only contains ↪ lowercase consonants, and the program does not explicitly convert the input string ‘s’ or the words to ↪ lowercase before checking if a character is a consonant. This means that if the input string contains ↪ uppercase letters, the program might not count them as consonants, leading to incorrect results. [...] ↪ the program correctly implements the functionality described in the docstring under the assumption ↪ that the input string contains only lowercase letters or that case sensitivity is not a concern...

Execution Prediction: To conclude this section, we highlighted an error in execution prediction. In the example below, even though the statement `if ")" in brackets` is clearly true, GPT-4 was biased by the intended functionality of the program and did not follow its actual execution semantics.

```
def correct_bracketing(brackets: str):
    if ")" in brackets:
        return False
    open_brackets = 0
    for bracket in brackets:
        if bracket == "(":
            open_brackets += 1
        else:
            open_brackets -= 1
            if open_brackets < 0:
                return False
    return open_brackets == 0
assert correct_bracketing('()') == ??
# Correct Answer: False
```

*GPT-4 Execution: [...] 2. The function is called with the argument "()".
3. The first if condition checks if ")" is in brackets. Since it is, but only after "(", this condition does not ↪ lead to a return of False at this point [...]*

3.2.3 Discussion

Once again, we saw two surprising nuances that were not clear from benchmark scores. First, we saw a low correlation between problem difficulty and counterfeit understanding ability. Even for problems that models pass often, we still found many samples they failed to understand. Second, we discovered that weaker models could generate counterfeits that were as difficult to understand as stronger models. Overall, counterfeits were a relatively general phenomenon, not just a failure mode of weaker models or harder problems.

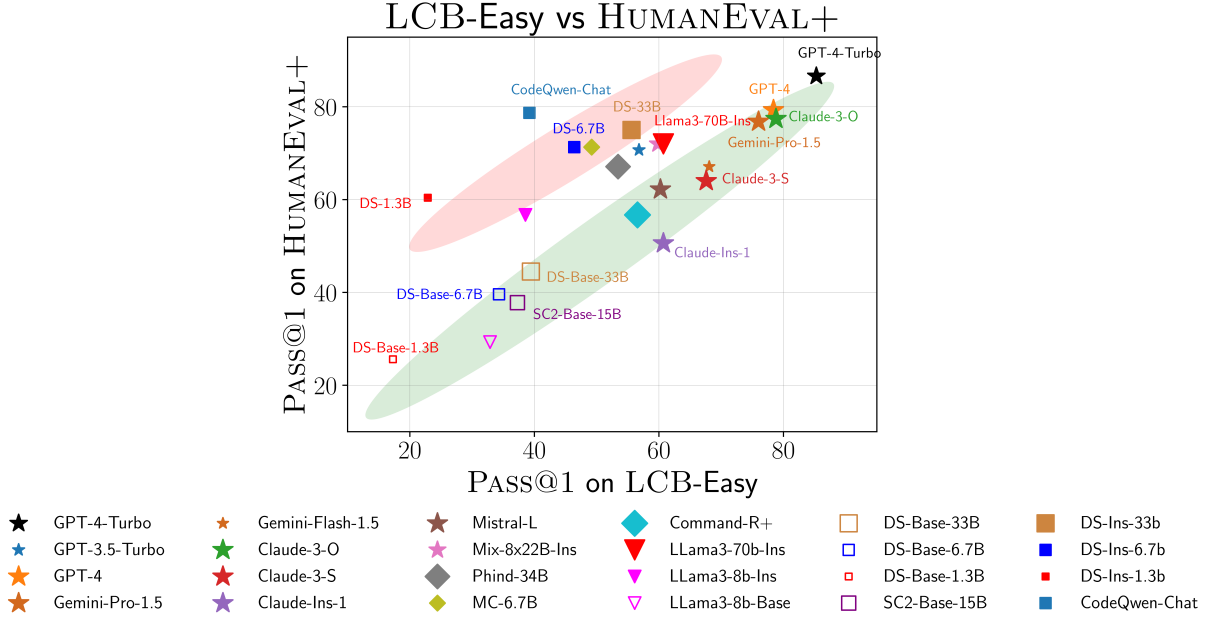


Figure 3.11: Scatter plot comparing PASS@1 of models on HUMANEval+ versus LCB-Easy (time-window Sep 2023 to May 2024). The green-shaded region marks models with aligned performance across benchmarks, while the red-shaded region marks models with much stronger performance on HUMANEval+.

3.3 LiveCodeBench: Overfitting, Post-Training, and Open vs. Closed Models

3.3.1 Overfitting to HumanEval+

We first compared the code generation performance of models on HUMANEval+ vs. LIVECODEBENCH, shown in Figure 3.11. We observed two distinct clusters of models, shown in red and green. The green cluster shows models that perform similarly on both benchmarks, consisting mostly of base and closed-access models. The red cluster shows models that perform significantly better on HUMANEval+ than LIVECODEBENCH. Careful analysis reveals that this cluster consists primarily of fine-tuned variants of open-access models, strongly indicative of overfitting or data contamination. In particular, we drew attention to DS-Ins-1.3B, which achieved 60% on HUMANEval+ but only 26% on LCB-Easy. These results mirror what we found with CRUXEVAL in Section 3.1: a class of models with a high HumanEval (HUMANEval+ in this case) score with no corresponding gains on a different benchmark.

Overall, the lesson is that high scores on familiar benchmarks can be ambiguous. Sometimes, it may indicate general programming abilities, but other times it may be due to benchmark-specific adaptation. It is crucial to compare across a varied suite of different benchmarks to understand the full picture, and we should be careful of techniques that promote overfitting rather than developing general abilities.

3.3.2 Role of Post-Training

Next, we analyzed the performance of base models vs. post-trained models. In terms of base models, we saw that L3-BASE and DEEPSEEK significantly outperformed CODELLAMA and STARCORDER2. However, these differences may partially be due to the training data distribution. For example, STARCORDER2 uses competition coding problems in its pre-training corpus.

Our benchmark also showed that post-training significantly improved performance, as L3-INS-70B, DS-INS-33B, and PHIND-34B achieved 28.3, 23.6, and 21.0 Pass@1 on LCB, improving over their base models by 8.2, 7.3, and 9.5 points respectively.

On the other hand, while base models generally fall into the green cluster of models, post-trained models generally end up in the red cluster. We hypothesize that compared to data used in closed-access models, the fine-tuning datasets for open models are lacking in diversity and quality, which may lead to poorer generalization.

3.3.3 Gap Between Closed and Open Models

In Figure 3.12, we investigated the gap between open-access and closed-access models. We already started to see this in Section 3.1, where we found that GPT-4 had already distinguished itself from other models at CRUXEval. It showed good performance on both CRUXEval and HumanEval seemingly without overfitting to either, and benefited significantly more from chain-of-thought than any other models. Overall, on LIVECODEBENCH, closed models generally outperformed open models. However, the gap is not uniform, as models like L3-INS-70B, MIXTRAL, and DS-INS-33B performed on-par with the weaker closed models like GEMINI-PRO and GPT-3.5-TURBO, and CLAUDE-3-SONNET. However, this varied across scenarios, as closed-access models performed better in the test output prediction and code execution scenarios. This suggests that strong base models and high-quality post-training data can partially close the gap, but that there are several capabilities where closed models still have a significant lead.

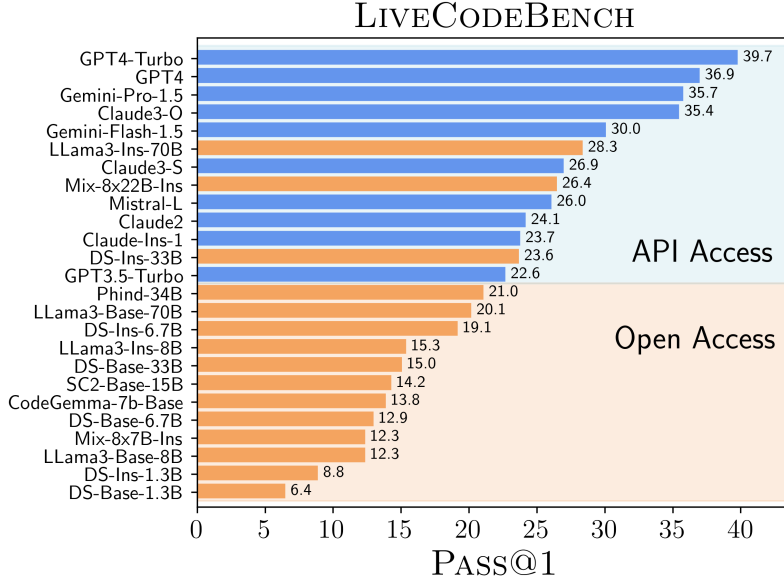


Figure 3.12: Comparison of open-access and closed-access models on the LIVECODEBENCH-Easy code-generation scenario.

3.3.4 Discussion

By looking deeper at CRUXEval and LiveCodeBench, we discovered two shared nuances. First, high scores on familiar benchmarks can be ambiguous. Sometimes, they indicate general programming abilities, but other times they may be due to task- or benchmark-specific adaptation. We should be careful of techniques that promote overfitting rather than developing general abilities. Second, people felt that closed models were better than open models, but existing benchmarks had not made this clear. Through these two benchmarks, however, we saw a quantitative difference. Our results suggest that strong base models and high-quality post-training data can partially close the gap, but that there are still capabilities like code execution where closed models had a significant lead.

3.4 LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers

In this section, we shifted our focus towards logical reasoning: deductively inferring the truth value of a conclusion from a collection of premises. Prior to our work, there had been many prompting and chain-of-thought based strategies to improve the logical reasoning capabilities of language models. However, we found that they failed in subtle and unpredictable ways. To mitigate this, we introduced **LINC: Logical Inference via Neurosymbolic Computation**.

In **LINC**, logical reasoning is tackled through a two-step neurosymbolic process. In the first step, a language model takes the natural language premises and desired conclusion and

converts them into first-order logic (FOL) expressions [86, 87]. Once everything is represented in first-order logic, we then apply a symbolic theorem prover to determine the truth value of the conclusion. As the translation from natural language to first-order logic can be imperfect, we use a majority voting scheme to improve the reliability of the approach.

The key advantage of LINC is that the reasoning burden is offloaded from the language model to the theorem prover. On the other hand, the challenge shifts to faithful and accurate formalization. The FOL translation must capture all the relevant information from the premises without adding or removing any information. Since it is not clear how the difficulties of formalization and natural language reasoning compare, the main focus of our work was to compare and contrast LINC with traditional **Chain-of-Thought** approaches. Through thorough experimentation and analysis on two challenging datasets, we provided insights into the strengths and weaknesses of both approaches. Overall, we found that they had distinct behaviors and failure modes, suggesting a strong synergy between theorem provers and language models.

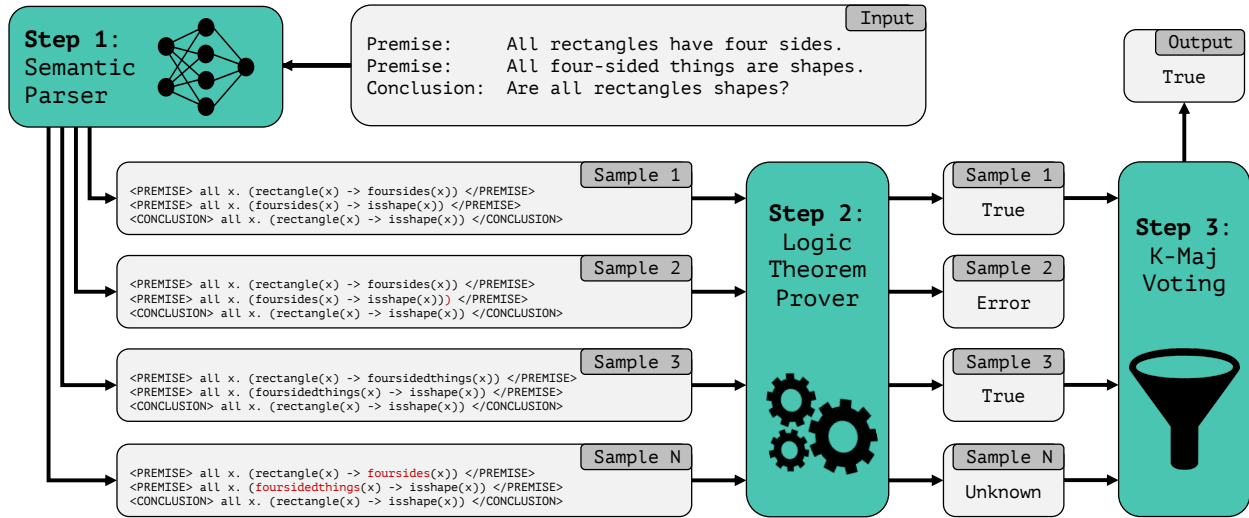


Figure 3.13: Overview of the LINC approach. Starting from a problem in natural language, the language model first samples first-order logic translations of the premises and conclusion. The translated formulas are then passed to an automated theorem prover, which filters invalid syntax and predicts labels for the valid samples. Finally, a majority-vote step aggregates the remaining candidate labels.

3.4.1 LINC: Logical Inference via Neurosymbolic Computation

Fig. 3.13 provides an overview of LINC’s two-stage approach. First, the LLM acts as a semantic parser to generate first-order logic expressions to be sent to the automated theorem prover. Second, **Prover9** [88] executes the expressions to determine whether the conclusion is **True**, **False**, or **Uncertain** under the translated premises, and raises an exception if the FOL syntax is invalid. The *K*-Maj voting scheme then aggregates predictions across multiple samples to determine the final prediction.

The core tradeoff of LINC is that the difficulty is shifted from faithful reasoning to faithful formalization. Using LINC, a language model no longer needs to reason in natural language. This removes any reasoning errors that may arise from the model’s inability to properly keep of logical structure. On the other hand, with LINC, the model must now faithfully formalize each of the natural language statements into FOL. This requires the FOL expressions to both be 1) *syntactically* valid, so that they are accepted by the prover, and 2) *semantically* valid, so that they contain the same semantic information as the natural language statements. As shown later in the error analysis, this translation can be difficult when statements are more complex, because the FOL expressions may fail to capture both explicit and implicit information.

3.4.2 Experiments

In this section, we present our experimental setup, the models we used, and the three baselines to which we compared LINC.

Datasets: We used two challenging datasets for logical reasoning in natural language: **FOLIO** [25] and **ProofWriter** [89]. FOLIO is an expert-written, open-domain, logically complex and diverse dataset for natural language reasoning with first-order logic. We used the validation set but discovered that 22 of the 204 samples contain errors (details in the original paper appendix), so we used the other 182 samples. ProofWriter is a synthetically generated dataset for logical reasoning over natural language. We randomly selected 360 samples from the OWA (Open-World Assumption) subset of ProofWriter in a way that is balanced across the number of reasoning steps in the shortest ground truth proof (depth 0-5; 50 samples each) and across the three labels (120 samples each; 20 each per depth).

In-context learning examples: We hand-picked eight diverse samples from the FOLIO training set to be used as few-shot in-context examples. Because ProofWriter does not come with ground truth FOL statements, we used these eight samples for both evaluations. Questions in ProofWriter generally have more premises per question than in FOLIO (5.3 in FOLIO vs. 18.8 in ProofWriter). Thus, our evaluation on FOLIO is an *in-distribution* task, whereas ProofWriter requires generalizing *out-of-distribution* to reasoning over considerably larger sets of premises than are given in the prompt.

Majority voting: We used K -way majority voting [90], where K samples are taken i.i.d. from the model and the mode used as the final prediction. We reported $K=10$ -way majority voting unless otherwise stated, with ties broken arbitrarily. We reported the effect of K on performance across our conditions and briefly discussed trends in the original paper appendix.

Models: We used three models pre-trained on both natural language and code: GPT-3.5 [4], GPT-4 (gpt-3.5-turbo-16k-0613 and gpt-4-0613) [5], and StarCoder+ (<https://huggingface.co/bigcode/starcoderplus>) [43] with a decoding temperature of $T = 0.8$ for all experiments. We defer model, hyperparameter, and hardware details to the original paper appendix. We opted for StarCoder+ for three reasons: firstly, unlike the other models we considered, it is a free, open-access model. Secondly, it has a dataset search functionality (see <https://huggingface.co/spaces/bigcode/in-the-stack> and <https://huggingface.co/spaces/bigcode/search>), with which we verified that FOLIO and ProofWriter were not in StarCoder+’s training set, giving further assurance in the validity of our findings. Thirdly, with its 15.5B parameters it is likely considerably smaller than GPT-3.5 and GPT-4; although

the exact size of these models has not been made public, their common predecessor GPT-3 was known to have 175B parameters [1]. This allowed us to compare performance at different model scales.

Controlled baselines:

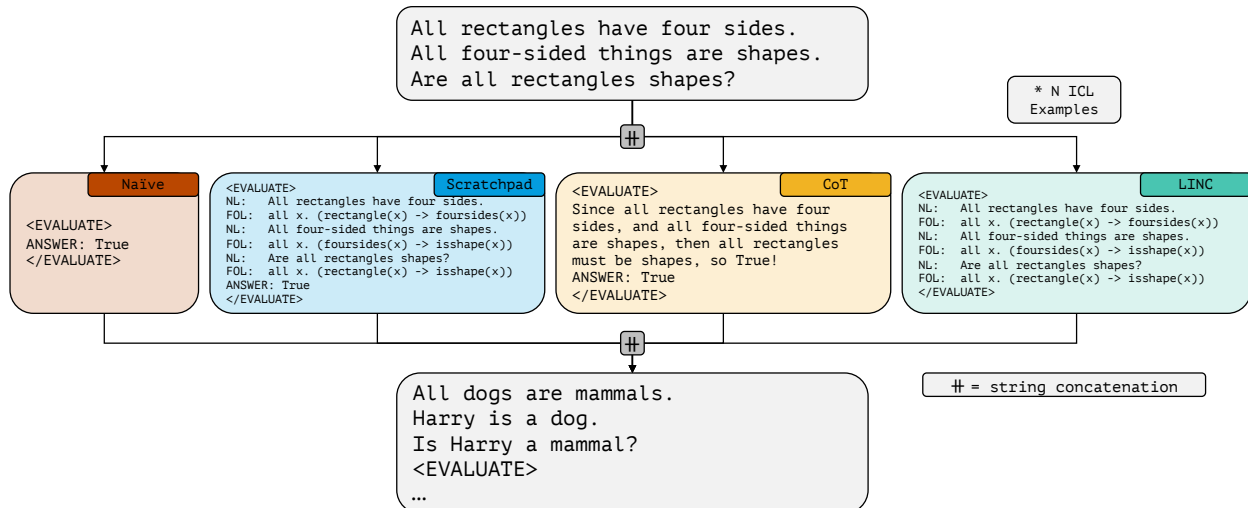


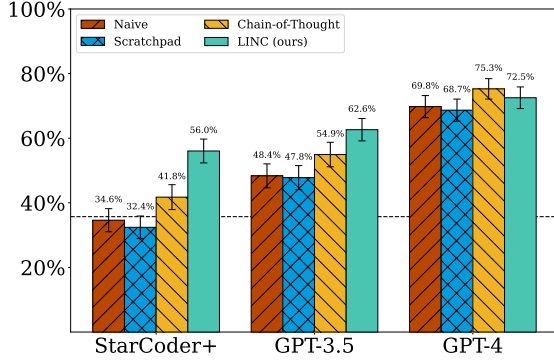
Figure 3.14: This figure outlines the string concatenation workflow for each of our conditions. We start with the original problem, provide ICL examples through an intermediate markup language, and finally append the problem to evaluate. At this stage, we allow the model to autoregressively sample until producing a stop token.

As shown in Fig. 3.14, we compared LINC with three baseline approaches: **Naïve**, **Scratchpad**, and **Chain-of-Thought (CoT)**. In **Naïve** (red), the model is given the natural language premises and asked to directly generate the label (**True/False/Uncertain**). In **Scratchpad** (blue) [85], we ablated the effectiveness of using a symbolic prover instead of a LLM in the second stage. The model is asked to generate FOL expressions just like in LINC, but instead of passing the FOL expressions to a symbolic prover, we ask the model to directly predict the label based on its own FOL translation. In **CoT** (yellow), we used standard CoT prompting [77, 90, 91], where the model is asked to generate step-by-step natural language reasoning to arrive at the conclusion. The prompts we used for all approaches can be found in the original paper appendix.

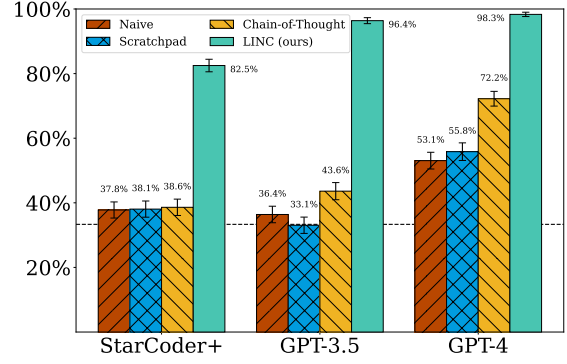
3.4.3 Results & Discussion

Our main results are shown in Figure 3.15. Each bar represents either LINC or one of the three baselines, while each group of bars indicates the language model used (in {**StarCoder+**, **GPT-3.5**, **GPT-4**}).

On FOLIO (Figure 3.15a), StarCoder+—the smallest model we experimented with—benefited the most from LINC, achieving a mean accuracy that was 14.2 points higher than the closest controlled baseline (56.0% vs. 41.8% with CoT). We found that **Scratchpad**, where the intermediate logical formulae are still generated but the call to the symbolic solver is ablated and replaced by the model’s own prediction, did not appear to benefit performance;



(a) FOLIO.



(b) ProofWriter.

Figure 3.15: Results of each model on the FOLIO and ProofWriter datasets. Accuracies are for bootstrapped 10-way majority vote for all models. Error bars are ± 1 bootstrapped standard deviation. Dotted, black line is the accuracy obtained by always guessing the most common label in the dataset.

for StarCoder+, neither **Scratchpad** nor the **Naïve** baseline performed better than simply deterministically predicting the most common label (“Uncertain”). For GPT-3.5, the trend is similar, although the gap between **LINC** and the closest baseline shrinks (62.6% vs. 54.9% average accuracies). For GPT-4, the trend reversed: **LINC** underperformed **CoT**. However, we performed a McNemar’s test [92] to get the p-value on this GPT-4 **LINC** vs. **CoT** comparison, and we found that the difference was not significant ($p = 0.58$). Meanwhile, for our balanced subset of ProofWriter, we saw significant performance gains across the board (Figure 3.15b); particularly so for GPT-3.5 and GPT-4, which achieved mean accuracies of 96.4% and 98.3% when paired with **LINC**.

We attribute the superior performance of **LINC** on ProofWriter to two causes. First, ProofWriter is much easier to formalize than FOLIO, which is the bottleneck for **LINC**. While FOLIO contains complex sentences with linguistic variety, all sentences in ProofWriter are synthetically generated and easily formalizable. Second, ProofWriter is an *out-of-distribution* setting, where the in-context examples contain fewer premises and shorter deductive chains than the reasoning task. This distribution shift makes it harder for the model to ignore irrelevant premises and keep track of the long deductive chains. By removing the reasoning burden, **LINC** is able to ignore this difficulty and perform well.

Finally, in Figure 3.16 we analyzed each model’s performance on ProofWriter as a function of the necessary proof depth. First, we found that StarCoder+’s performance was close to random chance across all three baselines, suggesting it did not have reasoning capabilities out-of-the-box. With **LINC**, it attains almost perfect accuracy at depths 0 and 1, but drops as the necessary proof length increases, likely due to some formalization inaccuracies. As StarCoder+ is a coding model, it makes sense that its formalization abilities far exceed its reasoning abilities. For GPT-3.5, all baselines perform above chance at depth 0, but quickly drop down for higher proof depths. In contrast, **LINC** allows it to achieve near-perfect performance for all proof depths, showing our approach scales even for longer reasoning chains. We saw a similar trend for GPT-4: although the baselines were significantly stronger, reasoning difficulty still remained at larger proof depths.

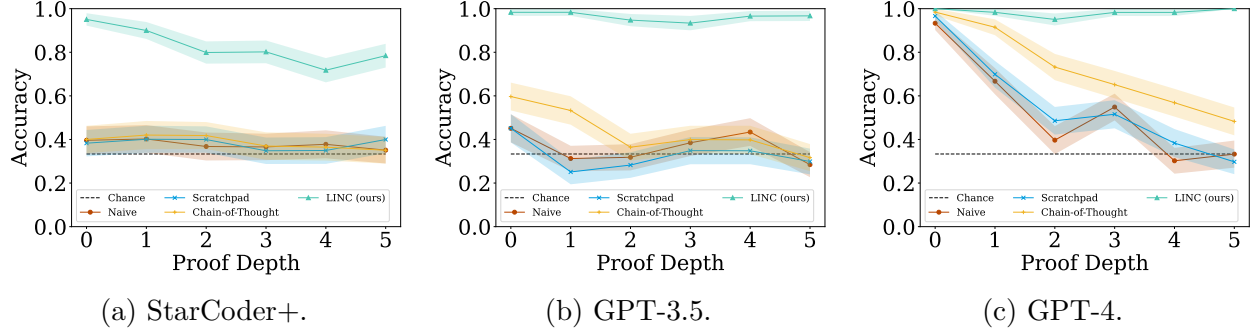


Figure 3.16: Accuracy per necessary proof depth in ProofWriter. Accuracies reported are for bootstrapped 10-way majority vote, and shaded areas cover ± 1 bootstrapped standard deviation. Black, dotted lines reflect the expected success rate of guessing a random label, which is $1/3$ in all subsets per our experiment design.

3.4.4 Error Analysis

Having established that LINC can improve performance in many settings, we now move on to our final research question: *How do the failure modes of LINC compare to those of in-context reasoning methods?* We focused on comparing GPT-4+CoT vs. GPT-4+LINC on FOLIO, since their overall performance was very similar (75.3% vs. 72.5% average accuracy). We leave an analysis of StarCoder+’s predictions on FOLIO to the original paper appendix.

Qualitative Analysis

Qualitatively, we found that LINC and CoT had completely different failure modes. We give a high-level overview and abbreviated examples of each failure mode here, leaving full detailed examples to the original paper appendix.

First, we detail the failure modes for LINC:

L1: FOL fails to capture implicit information not mentioned in the premises.

Often, there is obvious information not explicitly listed in the premises that is necessary to explicitly encode in FOL in order to successfully make a desired deduction. For example, in the snippet below one must encode in FOL the implicit assumption that Harry is a person (`Person(Harry)`).

Premise 1: When a person reads a book, that person gains knowledge.

FOL: all x. all y. (`Person(x) & Reads(x, y) & Book(y) -> Gains(x, Knowledge)`)

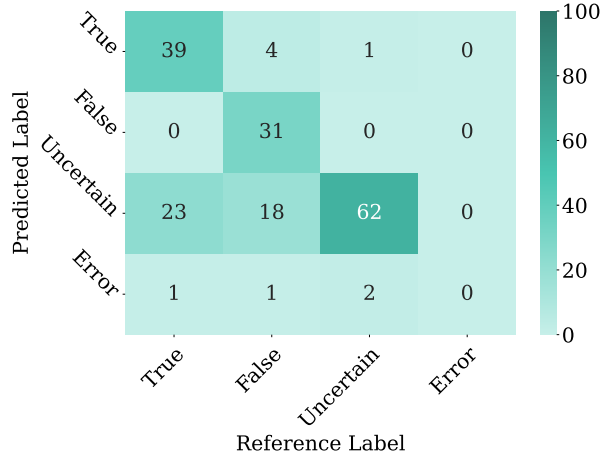
Premise 2: Harry read the book "Walden" by Henry Thoreau.

FOL: `Reads(Harry, Walden)`

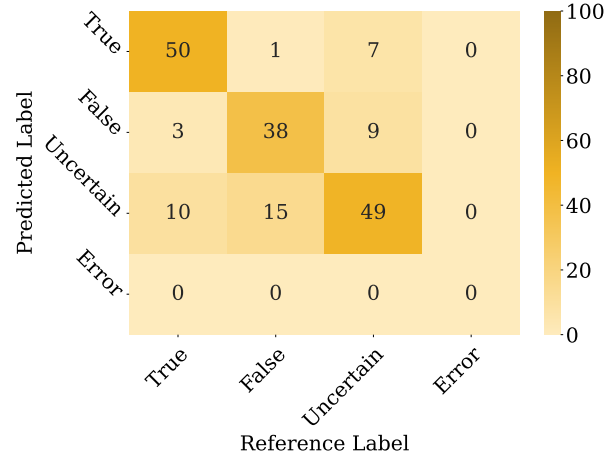
Conclusion (Prover9: Uncertain): Harry gains knowledge.

FOL: `Gains(Harry, Knowledge)`

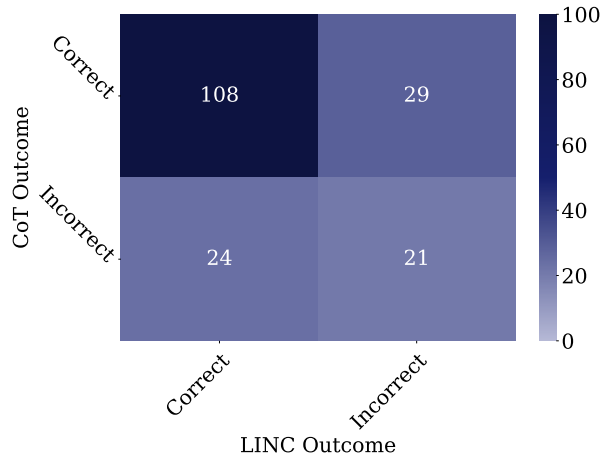
L2: FOL fails to capture information explicitly mentioned in the premises due to the choice of representation. Even when information is explicitly written in the



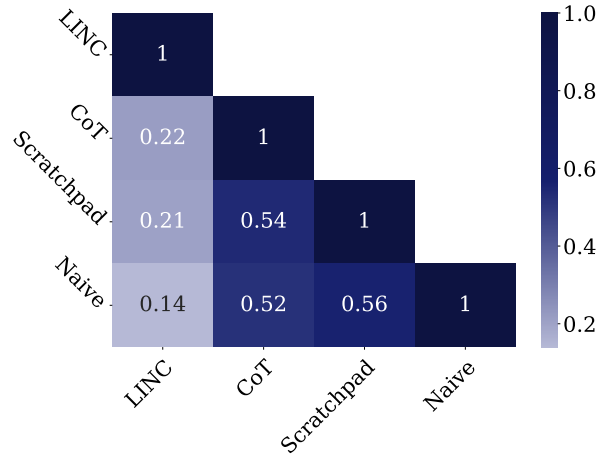
(a) Confusion matrix for LINC.



(b) Confusion matrix for Chain-of-Thought.



(c) Comparing the consistency of LINC vs. Chain-of-Thought.



(d) Similarity between incorrect predictions of each method, i.e., $(A \text{ wrong} == B \text{ wrong}) / (A \text{ wrong or } B \text{ wrong})$.

Figure 3.17: Analyzing and comparing the mistakes made by GPT-4 on the FOLIO dataset.

premises, the choice of how the NL is represented in FOL can lead to lost information. In the example below, the fact that Heinrich was a Nazi German politician is captured by one symbol `NaziGermanPolitician`, causing the information that he was independently Nazi, German, or a politician to be lost. As a result, LINC predicted `Uncertain` instead of the ground truth label `True`.

Premise: Heinrich Schmidt was a Nazi German politician.

FOL: `NaziGermanPolitician (HeinrichSchmidt)`

Conclusion (Prover9: Uncertain): Heinrich Schmidt was German.

FOL: `German(HeinrichSchmidt)`

L3: FOL contains syntax errors. Across all generations, we found that the FOL expressions sometimes contained syntax errors: 38% for StarCoderPlus, 24% for GPT-3.5,

and 13% for GPT-4. The most common error was that the same symbol was used with multiple arities. As an example, if `Summer(July4)` and `Loves(Alex, Summer)` were both present in a FOL translation, `Summer` would have a multiple arity violation.

Next, we detail three failure modes for CoT:

C1: CoT concludes something different than it suggests. One common mistake we often saw with the CoT approach was that it generated a chain of reasoning indicating uncertainty, but then asserted that the conclusion was false. An example of this is shown below, where GPT-4 suggested there was “no evidence” for the conclusion but then deduced it false.

Premises:
 Some mammals have teeth.
 Platypus have no teeth.
 Platypus are mammals.
 Humans have teeth.

Conclusion:
 Platypus are reptiles.

(GPT-4 Incorrect CoT): Let’s think step by step. We want to evaluate if platypus
 ↪ are reptiles. We know from the premises that platypus are mammals and that
 ↪ they have no teeth. The fact that some mammals have teeth does not
 ↪ contradict the fact that platypus, although mammals, do not have teeth.
 ↪ Moreover, the statement about humans having teeth is unrelated to the
 ↪ classification of platypus. There’s no evidence in the premises that would
 ↪ suggest platypus are reptiles. Therefore, the statement is false.

C2: CoT makes incorrect logical deductions. While there were many instances where CoT made incorrect logical deductions, one common deduction mistake was the structure “if B is true and $A \rightarrow B$, then A is true”. In the example shown below, we saw this incorrect deduction where A is “Mike has an F1 visa” and B is “Mike needs to apply for CPT or OPT if he wants to work in the US”. We note that making this type of logical mistake is often called the *fallacy of the converse*, and that humans are not immune to it, either.

Premises:
 International students in US have either an F1 visa or a J1 visa.
 An international student in US with an F1 visa needs to apply for CPT or OPT if
 ↪ the student wants to work in the US.
 Mike is an international student.
 Mike needs to apply for CPT if he wants to work in the US.

Conclusion (Ground Truth: Uncertain):
 Mike has an F1 visa.

(GPT-4 Incorrect CoT): Let’s think step by step. We want to evaluate if Mike has
 ↪ an F1 visa. From the premises, we know that Mike is an international
 ↪ student and needs to apply for CPT if he wants to work in the US. We also
 ↪ know that an international student in the US with an F1 visa needs to apply

$\hookrightarrow$ for CPT or OPT if the student wants to work in the US. So, it is clear
 $\hookrightarrow$ that Mike has an F1 visa since he fits the criteria. Therefore, the
 $\hookrightarrow$ conclusion is true.

C3: CoT fails to find complex paths of reasoning. We found that with all three models, CoT failed when the path of reasoning necessary to make the deduction was complex. Sometimes, CoT had difficulty getting started, and other times, it got stuck in the middle of a reasoning chain.

Quantitative Analysis

1. Compared to CoT, LINC had worse recall but better precision on True/False predictions. To see this, we plotted the confusion matrices in Figure 3.17b (CoT) and Figure 3.17a (LINC). Looking just at the distributions of predicted labels of the two methods, we saw that CoT predicted 32% True, 27% False, and 41% Uncertain, while LINC predicted 24% True, 17% False, and 57% Uncertain (with 2% of predictions throwing an error). Notably, we observed that LINC predicted Uncertain much more frequently than CoT (57% vs. 41%). To understand why, note that the translation from natural language to FOL is a lossy process: recall that in L1 and L2, we saw that the information conveyed through the FOL is sometimes a subset of the information in the original premises. Removing pieces of crucial information that were on the critical path to deducing True/False may then leave an uncertain conclusion. At the same time, while the FOL translations sometimes do not retain all of the information in the NL, they rarely contain false information that was not provided in the original premises. Therefore, LINC’s precision when predicting True or False was very high (93%) compared to that of CoT (81%), but this came at the cost of lower recall on True/False predictions (60% for LINC vs. 75% for CoT).

2. LINC and CoT mispredict on different examples. Earlier in Sec. 3.4.4, we saw that LINC and CoT exhibited different failure modes, which suggested they should fail on different examples. Indeed, we found that this was the case in our experiments on FOLIO: Figure 3.17c shows a 2×2 confusion matrix which compares whether or not each method’s prediction was correct. We observed that out of the $24 + 29 + 21 = 74$ samples where at least one method made an incorrect prediction, only 21 were shared. On a closer examination of these 21 samples, we found that 16 were ambiguous or incorrect in their specification (details in the original paper appendix), so *the two methods only agreed on 5 well-formed samples*. This suggests that LINC and CoT are complementary methods which fail under distinct circumstances.

3. Mispredictions of in-context reasoning baselines are more similar to each other than they are with mispredictions of LINC. As an extension of the previous analysis, we next investigated the correlation between the mispredictions of each pair of methods. To do so, we define a similarity score between two methods A and B as follows: Given a dataset D with N rows and ground truth labels $\{A_i\}_{i=1}^N$ and $\{B_i\}_{i=1}^N$ from two methods A and B , we define

$$\text{sim}_D(A, B) \triangleq \frac{\sum_{i=1}^N \mathbf{1}[A_i = B_i \neq R_i]}{\sum_{i=1}^N \mathbf{1}[A_i \neq R_i \text{ or } B_i \neq R_i]}$$

In words, $\text{sim}_D(A, B)$ measures the number of instances where A and B are wrong in identical ways vs. the number of instances where *at least one* of them is wrong.

Figure 3.17d shows the pairwise similarity between our four methods, highlighting that the similarity between LINC’s mispredictions and the other methods’ mispredictions (0.14, 0.21, 0.22) is much lower than the similarity between any pair of the in-context reasoning methods (0.52, 0.54, 0.56). These results suggest that for GPT-4 on FOLIO, LINC was the only method we evaluated which significantly altered the ways in which the model *fails* to reason.

3.4.5 Discussion

Through this work, we first saw by looking at the score that LINC outperformed CoT on ProofWriter but performed similarly on FOLIO. However, through deeper analysis, we learned that the failure modes of the two were completely different, making them complementary. CoT failed through natural-language reasoning errors, such as invalid deductions or final labels that did not match the preceding reasoning. LINC avoided many of these reasoning errors by using a theorem prover, but introduced a different bottleneck: lossy or invalid formalization into first-order logic. Looking only at the score would have missed this distinction.

3.5 Discussion

The four examples in this chapter highlight how evaluations can go beyond ranking models to reveal deeper insights about model behavior. In each case, analyzing the underlying behavior of models revealed nuances hidden by aggregate scores. Through CRUXEval and LiveCodeBench, we learned that distillation may not fully lead to transfer between different tasks and datasets. In the Counterfeit Conundrum, we saw several counterintuitive trends about the properties of counterfeit samples: they are a relatively general phenomenon, not just a failure mode of weaker models or harder problems. Finally, through LINC, we initially saw that LINC generally outperformed chain-of-thought prompting, but later discovered that the two approaches were actually complementary and failed in distinct ways. By analyzing the underlying successes and failures in each example, we gained a better high-level understanding of where methodologies shine and fall short, identifying opportunities for future scientific inquiry.

Chapter 4

What We Measure Shapes What We Build

Chapters 2 and 3 argued for the importance of using the right metrics and analyzing beyond surface-level results to gain deeper insights. Once a benchmark is established, it can become a goalpost for future research and innovation. This chapter explores how the benchmarks we designed and metrics we chose influenced the trajectory of work that comes afterwards.

4.1 CRUXEval

When we released CRUXEval in January 2024, coding benchmarks primarily measured the ability to write code. The release of our benchmark introduced the community to the idea of measuring code reasoning. In the two years since, the benchmark has shaped the field in three different ways. First, it inspired a family of code reasoning benchmarks. Second, it became a testbed for many execution-aware training and fine-tuning advancements. Finally, many frontier model developers adopted it as a standard evaluation.

4.1.1 Inspiring a Family of Code Reasoning Benchmarks

CRUXEval was the first benchmark to isolate code reasoning as an independent capability of interest through input and output prediction. Despite being simple and easy to run, it revealed many surprising insights about the behavior of language models. Soon, a wave of benchmarks embraced the idea of code reasoning and extended CRUXEval along many different axes. CRUXEval-X [26] translated the benchmark into nineteen programming languages. REval [27] increased the granularity of the prediction target by asking the model to reason about intermediate variable states, coverage, and execution paths. Others broadened the tested behaviors and semantics: ThrowBench [93] targeted runtime exceptions, CodeMind [28] added reasoning over a model’s own generated code and specifications, and EquiBench [94] and SeqCoBench [29] probed functional equivalence between programs. CodeCrash [95] tested an adversarial version of CRUXEval through misleading perturbations of the original programs. Recently, the idea of code reasoning has also been scaled beyond simple Python programs to real-world functions [96], larger-scale repositories [97], and more complicated

control flows including parallelism [98].

4.1.2 Explicitly Improving Code Execution

Since becoming a measurable target, code execution has frequently become part of the training data for coding models. Several works directly explored training on execution traces. SemCoder [30] trained on “monologue reasoning” with execution traces, showing that approximating an interpreter in natural language improved both CRUXEVAL and code generation. Similarly, CodeI/O [99] trained on a large dataset of input-output pairs, showing that it improved not only code reasoning but also general reasoning across math, logic, and commonsense tasks. Execution tuning [100] trained directly on execution traces of different granularities, and TracePile [101] collected a large-scale dataset of 2.6 million execution trace rationales. More recently, Meta’s Code World Model [38] included execution traces in its pre-training data to build an internal world model of program semantics.

Since the first wave of works training on execution traces, more creative execution-aware training strategies have emerged. CodeBoost [102] incorporated both input and output prediction on raw code snippets into the reward function of a reinforcement learning loop. Maimon et al. [32] trained models to simulate their own generated code, improving both CRUXEVAL and downstream competitive programming. Absolute Zero [103] used a single model to self-generate CRUXEVAL-style tasks to train reasoning capabilities with no external data.

4.1.3 Adoption by Frontier Model Developers

Finally, CRUXEVAL has been included as a standard evaluation in the release reports of many frontier models. Just two months after CRUXEVAL’s release, it was included in the release report of StarCoder2 [104]. Since then, it has been included in many other models, such as IBM’s Granite Code [105], Mistral’s Codestral [106], 01.AI’s Yi-Coder [107], ByteDance’s Seed-Coder [108], Ant Group’s Ling-Coder [109], Moonshot’s Kimi K2 [36], Meta’s Code World Model [38], and successive generations of DeepSeek [33, 110, 111] and Qwen [34, 35] models.

Beyond simply reporting scores, a few of these release reports directly discuss how they optimized for CRUXEVAL performance, showing that the benchmark played a role in shaping model development. For example, Qwen2.5-Coder explicitly mentioned that prioritizing executable code during data cleaning improved their CRUXEVAL performance [34]. Ling-Coder-Lite also generated synthetic training data in CRUXEVAL’s execution prediction style, which they said improved their score on the benchmark [109]. Finally, Yi-Coder advertised being the first open code model to surpass 50% accuracy on CRUXEVAL-O [107], showing their attention to the benchmark.

4.2 LiveCodeBench

Around the time of LIVECODEBENCH’s release, models were being trained on larger and larger crawls of the internet, making static benchmarks increasingly vulnerable to contamination.

It was the first benchmark to challenge the status quo of static coding benchmarks, instead focusing on *liveness*, continuously collecting new problems and updating the evaluation set every few months. First, this design inspired a wave of contamination-aware benchmarks across code, math, and general reasoning, helping to shift the community’s mindset to treat contamination as a first-class concern. Second, with over 1500 citations, LIVECODEBENCH has been adopted as a standard evaluation by researchers and frontier model developers alike, utilizing appropriate time windows to avoid contamination.

4.2.1 Contamination-Aware Benchmarks

LIVECODEBENCH popularized the recipe of using problems from a scrolling time window drawn from a continuously updated source of new tasks. LiveBench [112] extended LIVECODEBENCH beyond code to a larger suite of tasks including math, reasoning, language, and data analysis. The recipe also propagated into adjacent domains: LiveMathBench [113] applied the same idea to competition mathematics, assembling fresh problem sets from annual contests such as the AMC and the Putnam. LiveAoPSBench [114] timestamped olympiad-level problems mined from the Art of Problem Solving forum, similarly observing a steady performance decline on more recent problems and showing a similar phenomenon in the math domain. LiveMathematicianBench [115] pushed the idea to research-level mathematics, drawing problems from arXiv papers posted after model training cutoffs. LiveCodeBench Pro [116] extended the benchmark’s idea to harder olympiad-level problems, and in software engineering, SWE-bench-Live [117] applied the same monthly-update discipline to real GitHub issues created after a fixed date. The idea has even been applied to multimodal evaluation: LiveXiv [118] built a continuously refreshed visual-question-answering benchmark from newly posted arXiv manuscripts.

Beyond the specific benchmarks it inspired, LIVECODEBENCH helped shift how the community reasons about contamination. Contamination itself was not a new concern: prior studies had developed methods to detect benchmark data in training corpora [119], found that performance on coding problems correlates with their prevalence on GitHub [120, 121], and caught coding benchmarks leaking into training data shortly after release [122]. However, these works treated contamination as something to detect after the fact. Instead, LIVECODEBENCH treated contamination as a first-class design concern. By showing a scrolling benchmark with time-adjustable windows, it directly exposed sharp performance drops on newer problems. As evident by the number of live benchmarks following LIVECODEBENCH, contamination-free benchmarks have become a standard expectation.

4.2.2 Adoption by Frontier Model Developers

Finally, LIVECODEBENCH became part of the standard evaluation suite for frontier model releases, particularly among open-weight and reasoning-oriented systems. It appeared in the release reports of successive DeepSeek generations [111, 123], the Qwen2.5-Coder and Qwen3 reports [34, 35], Google’s Gemini 2.5 [124], Moonshot’s Kimi K2 [36], ByteDance’s Seed-Coder [108], Zhipu’s GLM-4.5 [125], MiniMax-M1 [126], Mistral’s Magistral [127], and NVIDIA’s Llama-Nemotron [128], among others.

In addition, many of these reports referenced LIVECODEBENCH’s contamination methodology. The DeepSeek-V3 report [111] restricted evaluation to a recent window (problems from August to November 2024) precisely to avoid leakage. Similarly, the Qwen2.5-Coder report [34] evaluated only on the most recent months, describing them as questions that “could not have leaked into the training set” and thus reflect out-of-distribution ability. Gemini 2.5 [124] reported LIVECODEBENCH with its exact problem date range (October 2024 through January 2025) stated alongside the score.

4.3 LeanDojo: Theorem Proving with Retrieval-Augmented Language Models

LeanDojo was released in 2023, a time when there was limited tooling and infrastructure for doing research in machine learning for theorem proving. Most existing works relied on private training datasets, kept model checkpoints and training codebases hidden, and did not come with easy-to-use interaction tools for working with Lean. Through LeanDojo, we addressed all these concerns, providing an open-source toolkit for extracting data from and interacting with Lean, a large benchmark built from Lean’s mathematical library, and ReProver, a retrieval-augmented prover that served as a reproducible baseline. As a result, LeanDojo had a significant influence on the field, accumulating over 600 citations and 800 GitHub stars, inspiring many theorem-proving systems built directly on its infrastructure.

4.3.1 LeanDojo Benchmark

Figure 4.1 provides an overview of how the different components of LeanDojo come together. Here, we focus on the benchmark and tooling that facilitated progress in the field. First, LeanDojo provided infrastructure to extract proof meta-data from arbitrary Lean repositories. Second, applying this tooling on Mathlib produced the LeanDojo Benchmark, a collection of theorems, proofs, tactics, and premises. Third, we used this benchmark to train our ReProver model. Finally, the trained model proved theorems by interacting with Lean. It could initialize a theorem, observe the current proof state, try running a tactic, and see the feedback.

Data Extraction

Repositories on GitHub (e.g., `mathlib` or `lean-liquid`) contain Lean source code for theorems and proofs. However, they lack important runtime information such as intermediate proof states. Therefore, LeanDojo extracted and augmented Lean code with information such as file dependencies, ASTs, states and tactics, and premises. One design choice was that LeanDojo’s data extraction pipeline was not specific to Mathlib: it ran on arbitrary Lean repositories, allowing researchers to construct their own benchmarks from any Lean project.

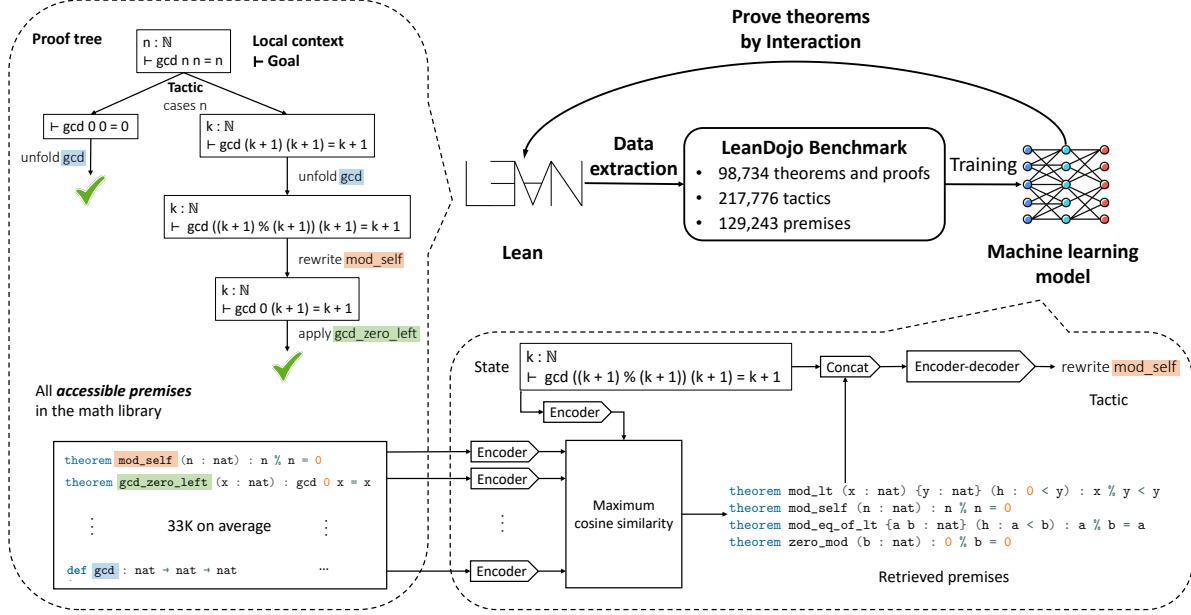


Figure 4.1: LeanDojo connects data extraction, proof-state interaction, premise retrieval, and tactic generation into a reproducible theorem-proving environment.

LeanDojo Benchmark

We created our benchmark by running this data extraction pipeline on Mathlib. LeanDojo Benchmark contained 98,734 theorems and proofs extracted from 3,384 files, split into 94,734 theorems for training, 2,000 for validation, and 2,000 for testing. One unique feature of our benchmark was that it introduced a new challenging data split to mitigate memorization effects. Mathlib contains several instances of nearly identical theorems with similar proofs (Figure 4.2). Using a random train/test split, a model can appear to solve a hard theorem by reusing a memorized proof pattern from its near-duplicate in training. In our `novel_premises` split, we require every test proof to use at least one premise that never appears in training set proofs.

Interacting with Lean

LeanDojo turned Lean into a gym-like environment with a deliberately minimal interface: `initialize(theorem)` returns the theorem’s initial proof state, a string representing the current goals and local contexts, and `run_tac(state, tactic)` executes a tactic on a state and returns either the next state or an error (e.g., from a timeout or an inapplicable tactic). Because the prover observes the outcome of every step, it can react to feedback as it goes, allowing iterative and feedback-driven proof search strategies to be easily implemented. By providing this interaction environment, we allowed future researchers to implement a variety of proof search strategies that could be seamlessly tested on our benchmark.

```

src/algebra/quaternion.lean
lemma conj_mul : (a * b).conj = b.conj * a.conj := begin
  ext; simp; ring_exp
end

lemma conj_conj_mul : (a.conj * b).conj = b.conj * a := begin
  rw [conj_mul, conj_conj]
end

lemma conj_mul_conj : (a * b.conj).conj = b * a.conj := begin
  rw [conj_mul, conj_conj]
end

```

Figure 4.2: Lean libraries contain clusters of similar theorems and proofs. Benchmark splits must account for this or they risk measuring memorization.

4.3.2 ReProver: A Retrieval-Augmented Model for Lean

In addition to facilitate future progress in the field, LeanDojo also released an open-source model: ReProver, a retrieval-augmented model. Before LeanDojo, tactic generators were typically language models that map a proof state directly to the next tactic. A model trained this way can only apply premises it has seen during training and memorized: if a proof requires a lemma the model has not internalized, there is no mechanism for finding it. This is a serious limitation, since Mathlib contains 130,262 premises—far more than any model can be expected to memorize. ReProver, our baseline prover, addresses this with two steps: *retrieve* premises relevant to the current proof state, then *augment* the tactic generator’s input with the retrieved premises.

Retrieve: Premise Selection

Our retriever was based on Dense Passage Retriever (DPR) [129]: a Transformer encoder that embeds the proof state and all candidate premises into a vector space. At each step, we retrieved by finding the candidate premise with the highest cosine similarity with the current proof state. The retriever was trained by minimizing a standard contrastive loss function between positive premises and in-batch negative premises.

We incorporated two insights specific to premise selection. First, instead of retrieving from all premises in the library, we restricted the retrieval set to premises accessible to the current theorem: those defined in the same file before it or imported from other files. This made the candidate premise set much smaller: LeanDojo Benchmark contains 130,262 premises in total, but the average number of accessible premises is only 33,160. Second, DPR’s performance depends critically on the quality of negative examples. In early experiments, we sampled all negative premises randomly, and the model often mistakenly retrieved other premises from the same file as the positive one. Therefore, we trained on a mixture of *in-file negatives* and random negatives.

Table 4.1: Theorem proving Pass@1 (%) on the LeanDojo Benchmark test sets. ReProver outperforms tidy, GPT-4, and an ablation that generates tactics without retrieval. All methods degrade substantially on the `novel_premises` split.

Method	random	novel_premises
tidy	23.8	5.3
GPT-4	29.0	7.4
ReProver	51.2	26.3
w/o retrieval	47.6	23.2

Augment: Tactic Generation

As in Figure 4.1 (*bottom*), retrieved premises are concatenated with the state.¹ Then an encoder-decoder Transformer, ByT5 [130], took them as input and generated the tactic. The model was trained to minimize the cross entropy loss w.r.t. human-written tactics. When proving theorems, the model generated multiple tactic candidates at each step, which were used in a standard best-first search algorithm to find proofs [131].

4.3.3 Results

Table 4.1 shows the results on the testing data of LeanDojo Benchmark. ReProver outperformed all baselines on two different data splits, demonstrating the effectiveness of retrieval-augmented theorem proving. GPT-4 performed substantially worse than our method. On the `novel_premises`, all methods performed substantially worse, but we observed that retrieval still provided a significant boost.

Finally, we ran ReProver on theorems from MiniF2F [132] and ProofNet [133]. The distribution of theorems in these datasets is different from Mathlib, making it an out-of-distribution dataset. On MiniF2F’s test set, ReProver achieved a Pass@1 of 26.5%, competitive with then state-of-the-art methods without RL (25.9% [134]). On ProofNet, our Pass@1 is 13.8%, which is the first reported theorem proving result on this dataset. Further, our prover discovered 33 proofs in MiniF2F and 39 proofs in ProofNet for theorems that did not previously have Lean proofs.

ReProver was able to make an impact in the theorem proving community because it was a strong baseline that was comparable to state-of-the-art models, while being simple and relatively fast to train. In the years to come, researchers built upon it in order to develop new algorithms and training schemes in theorem proving.

4.3.4 Opening Up Future Theorem-Proving Research

LeanDojo’s broader influence came from providing an open-source Lean playground consisting of toolkits, data, models, and benchmarks. This lowered the barrier to entry for research in neural theorem proving, as anyone could now train their own theorem provers. Lean

¹We retrieve 100 premises, concatenate them with the state, and truncate the concatenation to a fixed length.

Copilot [135] built on LeanDojo and ReProver to run language models in an IDE, suggesting tactics and searching for proofs as users write them. Lean-STaR [136] leveraged LeanDojo’s data and interaction environment to performing expert iteration, leading to a new state-of-the-art at the time.

In the years since, the field has grown far beyond what these early systems could reach. Benchmarks such as PutnamBench [137] continued to push the boundaries of neural theorem proving. After large-scale reinforcement learning became mainstream, systems like AlphaProof [138], Goedel-Prover-V2 [139], and Seed-Prover [140] quickly advanced the capabilities of theorem proving models, reaching near gold-medal performance on the International Mathematical Olympiad. Once provers could reliably produce correct proofs, a new issue became apparent. Optimizing for correctness alone yields proofs that are correct but inscrutable to human readers, far from what a mathematician would want to read or what a library like Mathlib would want to maintain. Echoing the theme of Chapter 2, it was now important to focus more on metrics beyond correctness. Inspired by this, we developed ProofOptimizer (Section 4.4), the first end-to-end fine-tuned model for optimizing Lean proofs.

4.4 ProofOptimizer: Training Language Models to Simplify Proofs without Human Demonstrations

ProofOptimizer is an example showing how focusing on a different metric led us to develop future work to address it. When analyzing AI-generated proofs, we discovered that they were often *correct* yet *suboptimal*: convoluted, bloated with redundant steps, or reliant on unnecessarily strong automation where a simple step would suffice. For example, Seed-Prover [140]’s [Lean proof of IMO 2025 P1](#) consists of 4,357 lines of code, 16x longer (by character count) than its [informal counterpart](#). Such proofs pose several practical drawbacks: they are (1) difficult for humans to comprehend, limiting their value as a source of mathematical insight; (2) less suitable as synthetic training data, since models may struggle to learn from convoluted proofs; and (3) computationally inefficient to compile in Lean, which is especially problematic when integrated into existing formal libraries like mathlib [141].

These challenges motivated the need for *proof simplification: transforming existing formal proofs into simpler forms while preserving correctness*. In this work, we adopted a natural notion of simplicity: *proof length*, measured by the number of Lean tokens. However, our approach was agnostic to the choice of simplicity metric: it was not restricted to proof length, but applied to any automatically computable measure [142].

By focusing on proof simplicity as the target metric, we developed *ProofOptimizer*, an LLM-based system for simplifying long and convoluted proofs in Lean. ProofOptimizer integrates three components: (i) a symbolic Lean linter that identifies and removes redundant steps, (ii) a 7B parameter language model finetuned specifically for proof simplification, and (iii) an iterative inference-time algorithm for progressively shortening proofs. Given an input proof, the Lean linter first eliminates the most obvious redundancies. The language model then generates multiple candidate simplifications, and the iterative algorithm repeatedly applies the model to the currently shortest proof, further reducing its length. Training follows

two paradigms. In expert iteration, the model proposes simplifications that are verified by Lean and incorporated into the training data for supervised finetuning. In reinforcement learning, proof length and correctness serve as the reward signal. Both approaches enable continual improvement without requiring any human-annotated simplification data.

We evaluated ProofOptimizer on long proofs generated by state-of-the-art neural theorem provers. Our final models achieved decent results (Fig. 4.3), shortening MiniF2F proofs by an average of 63% in a single shot and PutnamBench proofs by 26% with 32 attempts, substantially outperforming Gemini-2.5-Pro (Sec. 4.4.2). At test-time, iterative shortening helped us achieve per-proof average reductions of 87% (MiniF2F) and 57% (PutnamBench) and reduced the length of three out of four Seed-Prover IMO 2025 proofs by more than half.

Following the second lesson of this thesis, we did further analysis and found two downstream benefits of proof shortening. Training our base model on shortened proofs led to 2% better performance on miniF2F relative to training on unshortened proofs (Sec. 4.4.4). Also, shortening proofs often decreased their execution time, with 28% of proofs showing at least a 1.5x speedup after shortening (Sec. 4.4.4).

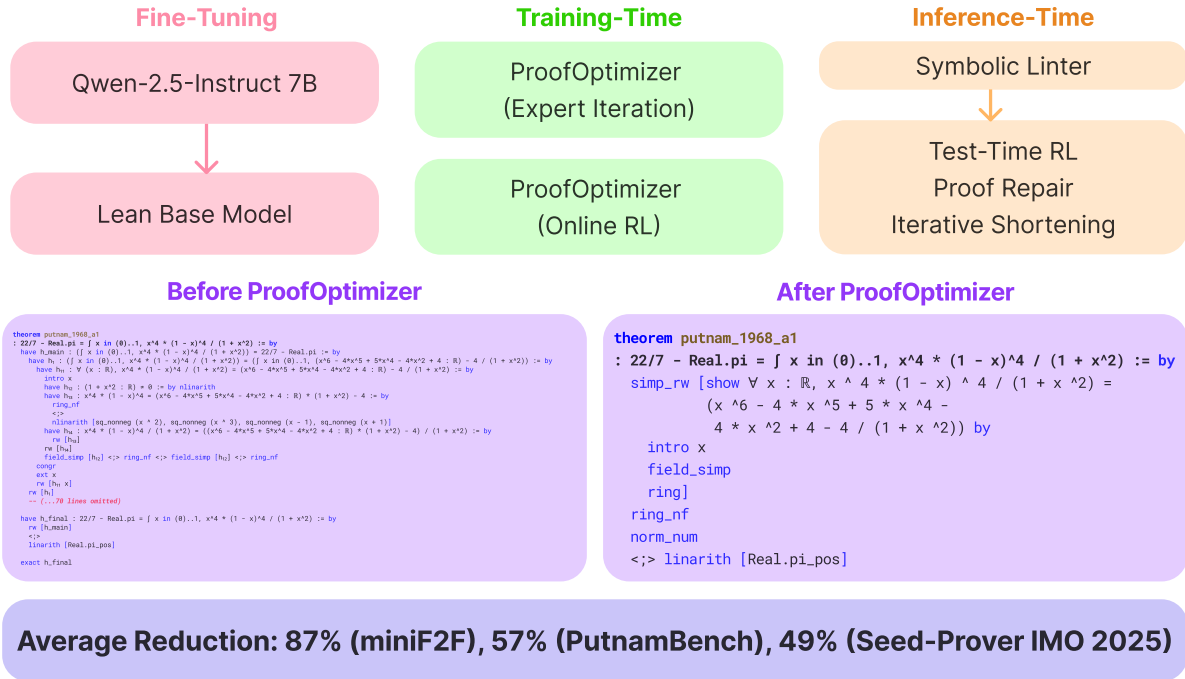


Figure 4.3: Overview of our pipeline. ProofOptimizer reduces the shortest generated proof of a Putnam problem from 1097 to 76 tokens.

4.4.1 Proof Simplification: Task and Metrics

Task Definition We formalize the proof simplification task as minimizing the complexity of a given proof. Specifically, for a valid formal statement s with proof p , the goal is to produce an alternative proof p^* of s that minimizes a complexity measure $\mathcal{L}$:

$$p^* = \arg \min_{x \text{ proves } s} \mathcal{L}(x)$$

Our method is agnostic to the choice of complexity measure $\mathcal{L}$, provided that it is deterministic and can be automatically computed from the proof. This flexibility encompasses the metrics used in prior work [143]. Here, we adopt proof length as the measure of complexity, defined as the number of tokens produced by a Lean-specific tokenizer. Our proof length measure correlates with character count but does not penalize long identifier names, and it ignores comments and line breaks. We denote the length of a proof x by $|x|$, i.e., $\mathcal{L}(x) = |x|$.

Evaluation Metrics Given an original proof p and k candidate simplifications generated by the model, $p'_1, p'_2, \dots, p'_k$, we define $l_i = \min(|p|, |p'_i|)$ if p'_i is a valid proof and $l_i = |p|$ otherwise. (Intuitively, an invalid attempt reverts to the original proof length). We evaluate proof simplification using two metrics:

- $\text{min@k} \triangleq \min_i \{l_i\}$ denotes the minimum shortened proof length (lower is better).
- $\text{red@k} \triangleq \max_i \left\{ \frac{|p| - l_i}{|p|} \right\} = 1 - \frac{\text{min@k}}{|p|}$ denotes the maximum relative proof length reduction from the original proof (higher is better).

Note that these metrics may not always be correlated: a method that only excels at shortening long proofs has a lower min@k and red@k than one that only excels at shortening short proofs. As with the pass@k metric [144], we report our metrics via an unbiased estimator using $n > k$ samples. We average min@k and red@k across samples in a dataset to get overall length and reduction metrics.

4.4.2 ProofOptimizer: LLMs for Proof Simplification

By focusing on proof simplicity as the target metric, we developed a training and inference-time pipeline for simplifying Lean proofs.

Training

First, we trained a general-purpose Lean model by fine-tuning **Qwen-2.5-7B-Instruct** on a combination of five tasks: natural language problem solving, Lean 4 code completion, auto-formalization (problems and solutions), formal theorem proving, and tactic/proof state prediction. Next, we describe our two training recipes: expert iteration and online reinforcement learning. For both recipes, we generated a data mix using a four-stage pipeline involving problem collection, proof sketching, theorem extraction and filtering, and proof generation.

ProofOptimizer-ExpIt: Expert Iteration: We leveraged a STaR-like [145] iterative training algorithm to improve our model. At a high level, we started with our base model π_0 and the collection of 145K proofs P_0 . At each iteration, we attempted to simplify each proof, trained our model on successful proof simplifications, and used the collection of simplified proofs as seed proofs for the next iteration. More precisely, at each iteration i , we did the following:

1. **Sample:** For each proof $x \in P_i$, use π_i to sample 4 simplifications $Y_p \triangleq \{y_x^1, y_x^2, y_x^3, y_x^4\} \sim \pi_i(x)$.
2. **Filter:** Use the Lean compiler to find the shortest correct simplification $y_x \in \{x\} \cup Y_x$. Create a training dataset of proof simplifications $D_i = \{(x, y_x) \mid \text{len}(y_x) \leq 0.8 \cdot$

$\text{len}(x), x \in P_i\}$. The length constraint is designed to encourage the model to learn more substantial simplifications rather than trivial ones. For iterations after the first, as x may have been simplified from a more complex proof $x' \in P_0$, we also add (x', y_x) pairs to D_i , which are valid and larger proof simplifications. Also, collect simplified proofs $\pi_{i+1} = \{s_x \mid x \in P_i\}$ for the next iteration.

3. **Train:** Fine-tune π_i on D_i to get π_{i+1} .

ProofOptimizer-RL: Online Reinforcement Learning In addition to expert iteration as described in the previous section, we trained a proof optimizer model with online reinforcement learning. Using the same dataset as in expert iteration, the reinforcement learning task consists in producing a valid but shorter proof y for a statement given an initial proof x . The reward is defined as the relative shortening $R(x, y) = \frac{|y| - |x|}{|x|}$ if y is valid and $|y| \leq |x|$, and $R(x, y) = 0$ otherwise. We employed an asynchronous variant of the GRPO algorithm [146] with advantage $A_i = R_i - \frac{1}{k} \sum_{j \leq k} R_j$ baselined with the average reward of $k = 8$ samples, no advantage normalization by standard deviation [147], no KL regularization, and omitting sequences with zero advantage.

Inference-Time

At inference time, we first implemented a symbolic linter that removes extraneous tactics via Lean’s `linter.unusedTactic` linter, which detects tactics that do not change the proof state and provides messages like `'norm_num' tactic does nothing`. Then, we performed iterative proof shortening. For a given proof, we sampled k candidate shortenings and took the shortest correct one. Then, we sampled k shortenings of the new proof, took the shortest correct one – and so on.

4.4.3 Results

For all evaluations, we used proofs generated by Goedel-Prover-V2 [148] on two popular datasets in formal math, miniF2F [149] and PutnamBench [137]. For miniF2F, we used $n = 194$ proofs (average length 334), and for PutnamBench, we used $n = 75$ proofs (average length 1468).

Training-Time Results

In Table 4.2, we show the min@k and red@k scores of our two models with respect to linted proofs. We observed steady improvements during each round of expert iteration for both @1 and @32 metrics. **Our final model outperformed Gemini-2.5-Pro**, a strong reasoning model, even when given proof state annotations similar to Chain-of-States in ImProver [143].

Table 4.2: **Min@k and Red@k throughout expert iteration and online RL.** Our RL model has strong @1 results, while our ExpIt model has strong @32 results. RL metrics are Gaussian-smoothed.

Dataset	Category	Model	Min@1 ↓	Min@32 ↓	Red@1 ↑	Red@32 ↑
miniF2F		<i>Linted</i>		<i>302</i>		<i>0.0%</i>
		Gemini-2.5-Pro	280	207	24.3%	57.2%
		Gemini-2.5-Pro + States	283	207	26.4%	58.7%
		Base (7B)	283	202	17.6%	56.2%
	<i>ExpIt</i>	Base + It 1	266	178	33.4%	67.0%
		Base + It 2	251	166	45.1%	70.6%
		<i>ProofOptimizer-ExpIt</i>	241	153	49.0%	72.3%
	<i>RL</i>	<i>ProofOptimizer-RL</i>	190	152	63.6%	70.9%
Putnam Bench		<i>Linted</i>		<i>1359</i>		<i>0.0%</i>
		Gemini-2.5-Pro	1348	1303	5.5%	18.0%
		Gemini-2.5-Pro + States	1371	1319	6.1%	19.2%
		Base (7B)	1341	1222	3.9%	20.5%
	<i>ExpIt</i>	Base + It 1	1341	1215	5.2%	22.5%
		Base + It 2	1335	1186	6.9%	24.7%
		<i>ProofOptimizer-ExpIt</i>	1328	1161	8.2%	26.3%
	<i>RL</i>	<i>ProofOptimizer-RL</i>	1303	1258	14.9%	21.1%

Inference-Time Results

In Fig. 4.4 (left), we show the results of iterative proof shortening on miniF2F and PutnamBench proofs using *ProofOptimizer-ExpIt*. First, we did 64 samples per iteration for 6 iterations, observing steady improvement at each iteration. To demonstrate the potential of further scaling, we did 1024 samples at iterations 7 and 8 and saw significant improvement. **Overall, ProofOptimizer combined with iterative proof shortening was very effective on miniF2F and PutnamBench, as average proof length was reduced from 334 $\rightarrow$ 75 and 1468 $\rightarrow$ 811, for an average per-proof reduction of 87.9%/57.2%.** In Fig. 4.4 (right), we plot the overall shortening against the length of the original proof, observing that longer proofs remained challenging to simplify.

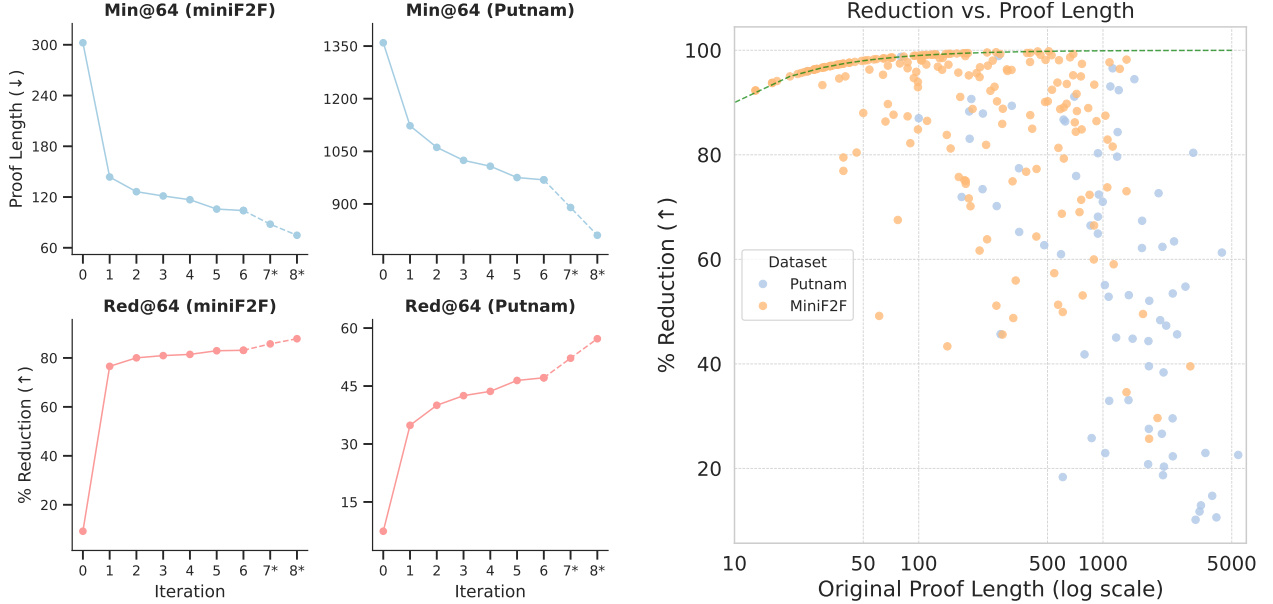


Figure 4.4: Iterative Shortening: per-iteration improvement (left) and effect of proof length (right)

Finally, in Table 4.3, we demonstrate the effectiveness of ProofOptimizer on an out-of-distribution dataset, Seed-Prover’s [four IMO 2025 proofs](#). With an order of magnitude higher sampling budget, we achieved a significant reduction in the proof length for all four problems, showcasing the potential of our model and technique.

Table 4.3: Iterative shortening achieves significant reduction for Seed-Prover’s IMO 2025 proofs.

	P1	P3	P4	P5
Original Proof Length	36478	16377	29147	8658
Simplified Proof Length	20506	7907	14531	4002
Length Reduction	43.8%	51.7%	50.1%	53.8%

4.4.4 Additional Benefits of Proof Simplification

By doing further analysis on simplified proofs, we were able to uncover two additional benefits of proof simplification, which we describe here:

Training on Simplified Proofs Improves Generation

Next, we investigated whether fine-tuning on simplified proofs could be advantageous compared to fine-tuning on longer, raw proofs. To do so, we prepared two datasets of identical problems, the first containing a set of proofs generated by `Goedel-Prover-V2` and the second containing the same proofs simplified by `ProofOptimizer-ExpIt`. The average proof length of the original

and simplified proofs was 147 and 85, respectively. We did continued supervised fine-tuning (SFT) starting from our base Lean model with a standard negative log-likelihood (NLL) loss.

In Fig. 4.5 (left), we compare the training loss between the two datasets. As expected, the initial loss when using original proofs was higher, as models had not seen such long proofs during initial fine-tuning. However, the losses quickly converged. We observed that training on original proofs caused occasional loss spikes, which we suspected were due to several data batches that were hard to learn (e.g. extremely long proofs). Decreasing the learning rate mitigated these training loss spikes but did not improve validation accuracy. In Fig. 4.5 (right), we compare the miniF2F scores of the two models during SFT, showing that training on simplified proofs resulted in slightly higher evaluation accuracy despite the two settings having identical training losses.

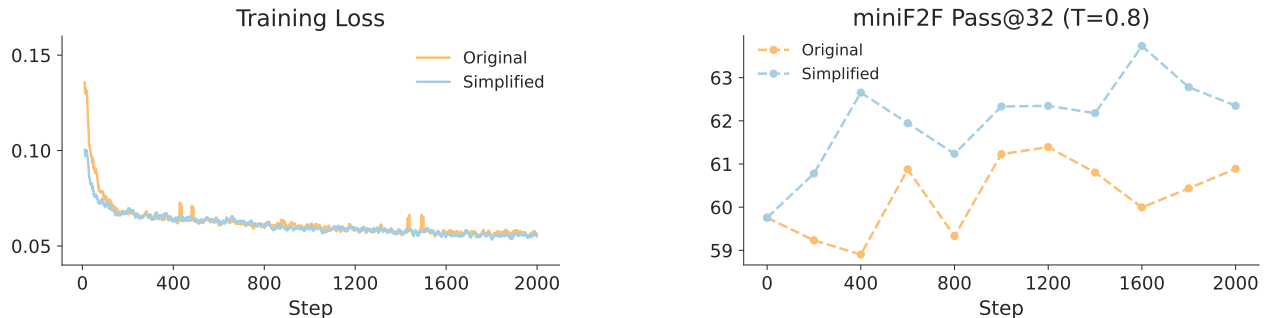


Figure 4.5: Training loss (left) and miniF2F score (right) after SFT on simplified vs. original proofs.

Simplified Proofs Have a Shorter Execution Time

We also observed that proofs simplified by ProofOptimizer often exhibited a faster execution time. We measured proof execution time with `lake env lean -profile`, excluding library import time (imports are always the same but actual time may vary due to caching effects). We compared the execution times of each proof before and after iterative shortening in Fig. 4.6 (scatter). For both datasets, we visibly observed that a majority of points lay below the $y = x$ line, signifying speedup. Fig. 4.6 (histograms) also show the distribution of speedup ratios $\frac{\text{time}_{\text{orig}}}{\text{time}_{\text{new}}}$. Of the 75 PutnamBench proofs, 50/75 had a speedup of over 10%, and 22/75 of those had a speedup of over 50%. We also observed that proofs with a higher original execution time tended to show more speedup. The same trends held for miniF2F, where 114/194 and 56/194 proofs had a speedup over 10% and 50%, respectively. Finally, we observed 25% and 81% speedups on Seed-Prover’s proofs for P3 and P4 of the IMO 2025.

Upon qualitatively analyzing the proofs, we observed that the original proofs often had extraneous tactics that were eliminated by the simplified proofs. However, we also found several cases where the simplified proofs were much slower than the original proof, which usually occurred when a faster proof algorithm was replaced by a shorter but slower method (e.g. brute force with `interval_cases`).

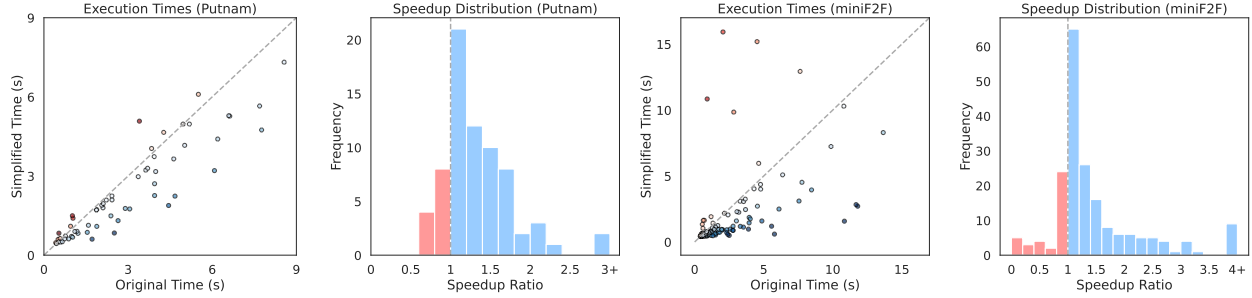


Figure 4.6: Simplified proofs are frequently faster than original proofs on miniF2F and PutnamBench.

4.4.5 Conclusion

In this section, we started by focusing on a different metric in theorem proving: proof simplicity rather than proof correctness. By honing in on this metric, we developed ProofOptimizer, the first language model trained to simplify Lean proofs. Unlike prior work that wraps existing LLMs around agentic scaffolding, we trained a model using expert iteration and RL, coupled with a symbolic linter and iterative proof shortening at inference time. While simple, our approach already yielded nontrivial results, reducing proof length by an average of 87% on MiniF2F, 57% on PutnamBench, and 49% on Seed-Prover’s IMO 2025 proofs. As AI becomes more tightly integrated with mathematics, we envision a future where AI-generated proofs are not only correct but also concise and readable, with simplification serving as a critical bridge between rigorous formal proofs and human intuitive understanding.

4.5 Discussion

In this chapter, we saw several examples of how benchmarks, metrics, and tools shape the research landscape that follows. In CRUXEval, we introduced code reasoning as an evaluation target, showing the community that important aspects of coding were worth measuring beyond code generation. Through LiveCodeBench, we made the idea of time-indexed evaluations more mainstream in the scientific community. LeanDojo provided an influential benchmark and toolkit that helped researchers advance the field of theorem proving with language models. ProofOptimizer then illustrated the same lesson at a smaller scale: once proof length became a measurable target, proof simplification became something a model could be trained to optimize.

Having seen how what we measure shapes what we build, we close this thesis with a forward-looking report on the future of software engineering: our perspective on what the field should measure and build next. Many of its ideas trace directly back to the themes of this thesis. CRUXEval’s lesson that generation is only one slice of coding ability is echoed in our call for measuring capabilities like code optimization and software testing. The contamination and overfitting problems that motivated LiveCodeBench are fleshed out in Chapter 5’s first challenge: evaluation and benchmarks. Despite being written a year ago in a fast-moving field, most of the challenges we identify remain open today, which suggests that they are central problems rather than fleeting artifacts.

Chapter 5

Future Research and Remaining Challenges Toward AI for Software Engineering

5.1 Introduction

In the previous chapters in this thesis, we have seen a collection of different evaluations that probe different capabilities of language models and reveal various insights about their behavior. Shaped by the lessons that we have learned from these evaluations, we now briefly situate the task space of AI for software engineering, then use formal verification as a representative task for discussing challenges and paths forward.

Our goal is threefold. In Sec. 5.2, we briefly summarize the main task families in AI for software engineering and focus on formal verification as a checkable target for reasoning about software correctness. The original position paper contains the broader taxonomy of software-engineering tasks beyond code generation and code completion, including the measures used to categorize their scope, logical complexity, and level of human intervention.

Moving forward to Sec. 5.3, we highlight several challenges in the field that today’s models face, each cross-cutting and applicable to several tasks. These challenges synthesize the lessons from the earlier chapters: future AI software engineers should be evaluated across broader tasks, with metrics that preserve real development difficulty, and in settings that account for feedback, code quality, and tool use.

In Sec. 5.4, we posit a set of promising research directions to tackle the challenges above. We hope that through this chapter, the reader can appreciate the progress the field has made, understand the shortcomings of today’s state-of-the-art models, and take inspiration from our suggested future ideas for tackling these challenges.

5.2 Tasks in AI Software Engineering

The full position paper describes a broader taxonomy of tasks in AI for software engineering. Here, I summarize the task families briefly to situate the rest of the chapter. **Code generation** covers writing code from natural-language intent, existing context, or partial programs. **Code**

transformation covers refactoring, migration, translation, and optimization while preserving intended behavior. **Software testing and program analysis** covers generating tests, finding bugs, reasoning about program behavior, and repairing failures. **Software maintenance** covers documentation, pull request review, codebase navigation, and question answering. **Scaffolding and meta-code** covers the surrounding infrastructure that makes software run, including CI/CD, configuration, deployment, authentication, and other project glue. **Formal verification** covers producing checkable proofs that software satisfies desired properties.

We describe the rest of these tasks in detail in the paper, but in this thesis I focus on formal verification, which appeared earlier through LeanDojo and ProofOptimizer. As AI writes more code, formal verification becomes increasingly important: generated software should not merely pass tests, but should also support stronger forms of checkable correctness when the domain demands it.

5.2.1 Formal Verification

The task of formal verification involves generating checkable, mechanized proofs that can guarantee that a piece of code works as intended. Formal verification of software is necessary in mission-critical applications such as aircraft software, where it is crucial that code is correct with absolute certainty. In some cases, software bugs have led to costly disasters. In the maiden flight of the Ariane 5 rocket, a floating-point conversion error caused it to explode forty seconds after liftoff. Another case is with the computer-controlled radiation therapy machine Therac-25, where concurrency bugs led to six people being massively overdosed, leading to serious injury and deaths.

In the formal methods literature, there are two major classes of formal verification: full functional verification (FFV) and property verification (PV). In FFV, the goal is to design a complete and precise formal specification that captures the desired behavior of the implementation. For example, Zee et al. [150] developed a method for full functional verification of data structures like mutable lists, trees, graphs, and hash tables. The challenge in full functional verification generally lies in writing the specification in a way that captures all the required aspects that need to be shown.

While FFV provides a complete set of guarantees, it is usually sufficient to opt for PV, where a few key properties of a system are proven correct. Examples include: ensuring that two threads do not simultaneously enter a critical section of a program, verifying that a complex program will always terminate, proving the absence of security vulnerabilities like buffer overflows, and guaranteeing memory safety. Unlike in FFV, the specification is usually simpler to write, and the challenge is usually finding the right domain-specific tool and designing the right algorithms to write the proof. Another challenge that makes PV difficult to use in practice is the issue of false positives, where functionally correct code often does not pass property checks. Tools like Coverity illustrate this practical tension: if a verifier produces too many false positives or changes its behavior across versions, developers may stop trusting it even when it finds real bugs [151].

Formal software verification has already produced a few striking successes. Astrée verified the absence of run-time errors in Airbus A340 flight-control software [152], seL4 showed that an operating-system microkernel could be mechanically verified [153], and CompCert showed that even a realistic optimizing compiler can be proved correct [154]. These examples are

still rare relative to the amount of software written today, but they show why the task is worth taking seriously.

Example. Verified Compiler: CompCert [154] is a formally verified optimizing C compiler that supports a restricted subset of C including most of the ISO C 99 language. CompCert has been formally verified using the Rocq/Coq proof assistant [155], eliminating the potential for compiler bugs.

To make formal verification easier, there have been many languages and proof environments designed specifically for it, including Rocq/Coq, Lean, Dafny, Isabelle, and Verus [155–159]. LLM-based work in this area is still young, but early benchmarks such as DafnyBench and miniCodeProps, along with systems such as AutoVerus, suggest that models may become useful for generating specifications, annotations, and proofs in constrained settings [160–162].

While formal verification tools have begun to see adoption in industry, they have not yet become mainstream because of these challenges. Code LLMs could greatly ease this burden and make it more feasible to verify code at larger scales, especially verifying properties requiring lower logical complexity.

Bottlenecks in LLMs for Software Verification: While LLMs can sometimes verify simple function-level properties, they have not been able to scale beyond that due to a few fundamental challenges. First, languages used in formal verification such as Dafny and Verus are low-resource languages, and LLMs often produce code with incorrect syntax and semantics. Second, writing precise specifications that fully capture all the desired properties in a sound and complete manner can sometimes be difficult. Third, writing proofs can be tricky for LLMs as well. In verification languages, invariants often need to be written in a precise form, because tools like SMT solvers can rely on heuristics that are sensitive to the way that constraints are written.

5.3 Challenges

Throughout this thesis, we have seen that the field of AI for code has made significant progress over the years. However, state-of-the-art models still struggle with software engineering tasks, especially at larger scopes and higher levels of logical complexity. Next, we discuss two key challenges in the field, motivated by the insights we have learned. The challenges below are not meant to be an exhaustive list, but rather a synthesis of key themes inspired by the work in this thesis.

5.3.1 Long-Horizon Code Planning

One of the key lessons from SlopCodeBench was that as models worked on longer-horizon tasks, their code quality often degraded. Similarly, in ProofOptimizer, we observed that models could produce formally correct code that was unreadable or difficult to maintain. This points to a broader issue: AI coding agents need to perform long-horizon planning, not merely produce locally correct artifacts. Large software engineering projects require planning around the design and structure of the code, and LLMs still struggle to design good, lasting abstractions while respecting modularity and code quality principles.

When working on large projects, engineers and tech leads often make complex decisions about how to design and structure the code to best support the various functionalities that will eventually be needed. To build a long-lasting software system, an engineer must know the potential paths that the system’s evolution might take. This requires domain expertise and experience in how different code structures require different forms of extension. We believe that today’s language models are unable to perform this level of sophisticated planning.

Designing Good Abstractions: One instance of long-horizon code planning is choosing the right abstractions from the outset. An API designed with good abstractions will allow new features to be implemented seamlessly with minimal user overhead, while an API designed with poor abstractions may lead to excessive code duplication, refactoring, or even debugging. We discuss two examples of this, library learning and data representation.

Library learning: Designing APIs and libraries are designed with useful abstractions often leads to more code reuse and more intuitive interfaces. The challenge of library learning is to derive a library of useful abstractions from a corpus of programs by abstracting out common reusable features [163–165]. While the traditional library learning literature has focused primarily on code reuse, a truly effective library must also prioritize ease of use and maintainability, as well as be robust and adaptable to future extensions.

Data representation: The choice between data structures leads to a variety of trade-offs when it comes to performance aspects such as memory usage and processing speed. For example, database engineers need to decide between various data models, storage formats, and indexing methods to balance performance.

Modularity and Code Quality: LLMs are trained and optimized primarily for code correctness with insufficient focus on other aspects of code like quality and maintainability. This is further exacerbated with large scale reinforcement learning being performed using test cases which can lead to unintended consequences regarding code quality, as correct but poor quality code is still often given a high reward. Empirically, it has been observed that LLM written solutions are often more complex than human-written counterparts. For example, Jain et al. [166] identified that LLMs prefer to repeat existing code instead of making use of abstractions in the existing code. One aspect of code quality is modularity, ensuring that code does not get duplicated too often. Here, Berlot-Attwell et al. [167] identified that library or tool reuse is non-trivial for LLMs in coding and formal math domains.

5.3.2 High Logical Complexity and OOD Domains

In Chapter 4, we saw how a benchmark can shape the direction of the research that follows. As models improve, this lesson becomes especially important: benchmarks should continue to expose capabilities that remain difficult, rather than only measuring saturated settings. This final challenge concerns high logical complexity and OOD domains, which most of today’s benchmarks do not address. Tasks such as writing highly concurrent code or discovering performance optimizations require high levels of algorithmic ingenuity and complex logic, often proving difficult even for strong human programmers.

Some programming tasks are challenging for even the best human programmers, requiring approaches with a very high logical complexity. Examples of tasks that fall into this category include superoptimizing programs, discovering attacks for purportedly secure code, writing performant compilers, optimizing GPU kernels [168], and writing very error-prone and very

technical code.

Example: Verifying File System Properties: In formal verification, working with new domains often requires devising new theories that faithfully represent the desired properties. FSCQ is a formally certified crash-proof file system with the provable guarantee that under any sequence of crashes followed by reboots, it will recover the file system correctly without losing data [169]. In this domain, one challenge is that proving safety cannot be done at the source-code level: because instructions are not atomic, data may be lost if the crash occurs within a non-atomic instruction. Instead, a new logic known as Crash Hoare Logic (CHL) needed to be developed, along with constructs representing crash conditions and recovery procedures. Constructing a logic like this would be very difficult for today’s AI systems.

As discussed throughout this thesis, what we measure shapes what we build. For tasks like these, evaluation is difficult, making progress sparse. It can be hard to even know which metric to focus on, since many domains lack clean models for reasoning about their properties. It is therefore worth asking what it would take for today’s AI systems to accomplish a task like this, rather than only measuring capabilities that already fit existing benchmark formats.

Limits of Symbolic Techniques: When it comes to applying symbolic techniques to these tasks, there are a few limiting factors that make them difficult to tackle. First, for synthesis-style tasks, the search space can be very large. Deductive and rewrite-based synthesis techniques are unable to explore a majority of the search space. Second, verifiers can be limited in power, such as when dealing with properties in concurrency or weak memory models. Third, many domains lack clean models to reason about properties, such as dealing with memory bandwidth in GPU kernels.

Because they are hard for humans, these tasks are very rarely in the training data of today’s language models. They have unique, domain-specific challenges that make generalizing from existing data difficult. For these problems, language models rely heavily on feedback-driven search algorithms [170], and it can be difficult to navigate the search space effectively. In addition, many of these tasks lack feedback mechanisms, which is crucial for AI to pick up learning signals. When designing a complex algorithm or data structure, it is often hard to know if you are on the right track until you get to the correct result. When writing code for a large multithreaded operation, it may be hard to know if the algorithm has concurrency issues until all the parts are fully fleshed out. Without feedback, incremental improvement is nearly impossible.

5.4 Paths Forward

The original position paper discusses a broader set of paths forward, including richer code data, training in executable environments, and adaptation to specialized codebases. In this thesis, I focus only on inference-time approaches: improving retrieval over code, combining neural models with symbolic program analysis, and integrating software-engineering tools into model workflows.

5.4.1 Inference-Time Approaches

Semantic-Aware Embeddings and Retrieval

Semantic and execution-aware code embeddings: Code embeddings often treat programs as pure token sequences rather than explicitly incorporating execution and semantics. As a result, code that is close in embedding space is often syntactically similar rather than semantically similar [171, 172]. Before the LLM era, however, there were efforts to learn program representations that encode semantics, including neural modules for program operations [173] and execution-aware latent representations for partial or full programs [174–176]. Revisiting these ideas may improve retrieval and make code representations more aligned with program behavior.

Better retrieval-augmented code generation: Retrieval is especially useful in specialized contexts where the model has not seen enough examples to fully understand a language, API, or codebase. Documentation, function definitions, and local usage examples can teach syntax, available abstractions, and conventions. However, retrieving relevant code is only half the problem. The model must also reuse the retrieval correctly, adapting algorithms, translating between languages, and recognizing subtle differences between the retrieved context and the current task. This suggests that retrieval-augmented code systems should be designed not only to consume retrieved snippets, but also to reason about how those snippets should be modified.

Retrieving via code navigation on the fly: Static retrieval indexes can be expensive to build and can become stale as a codebase evolves. Instead of relying only on embeddings, agents can retrieve information by navigating the codebase directly, using commands such as `cd`, `ls`, and `grep`, IDE actions such as jumping to definitions or finding references, and static-analysis tools that expose ASTs or dependency graphs. This frames retrieval as an interactive code-understanding task rather than a one-shot nearest-neighbor lookup.

Neurosymbolic Approaches

Code is a unique domain because there is a vast body of programming-languages research to build on, including abstract interpretation [177], concolic testing [178, 179], model checking [180], linting, and type-checking [181]. These tools can surface bugs, enforce invariants, narrow search spaces, and provide structured feedback. Traditional PL tools often require precise specifications, can be computationally expensive, and may produce false positives. LLMs can help mitigate these issues by proposing specifications, localizing likely errors, and deciding when a symbolic tool is useful.

LINC and ProofOptimizer highlight this broader pattern. In LINC, the model translated natural language reasoning into executable symbolic programs. In ProofOptimizer, the model proposed proof transformations while the compiler ensured correctness. Neurosymbolic methods potentially allow AI systems to explore a large search space while maintaining guarantees throughout.

We provide a few examples of this potential integration. During code generation, program-analysis techniques could be applied to shorter snippets of AI-generated code to surface bugs or prove properties. To improve code understanding, agents can inspect program structure such as abstract syntax trees [182]. When debugging a large codebase, AI could first narrow down the suspicious region before handing the problem to more precise symbolic tools. During code generation in DSLs, LLMs can leverage grammars for constrained decoding [183–185]. During refactoring, static analysis can identify whether new errors have been introduced and cut off unpromising search paths.

Incorporating SWE Tools

Software engineers integrate a variety of domain-specific tools when writing code. AI systems should similarly be able to invoke tools during inference and incorporate their outputs into the next step of reasoning. At inference time, this requires three skills: choosing the right tool, invoking it correctly, and incorporating its output into the next step of reasoning. This requires precise tool interfaces, reliable access to execution or analysis results, and agent workflows that can use tool feedback without losing track of the broader task. Useful tools include compilers, test runners, linters, type checkers, package managers, profilers, debuggers, and build systems; the core challenge is not simply exposing these tools to a model, but teaching the model when their feedback is relevant and how to act on it.

Taken together, these inference-time directions suggest a common principle: the next stage of AI for code should be organized around feedback-rich interaction with programs. Retrieval helps models understand local codebases, neurosymbolic approaches connect model reasoning to program semantics, and tool use grounds model reasoning in executable evidence.

5.5 Discussion

In this chapter, we first briefly describe several tasks that we believe should be considered when it comes to AI for software engineering, focusing on formal verification as one such task. We have also highlighted a few cross-cutting challenges that permeate throughout many software-engineering tasks. Finally, to drive progress in AI for code, we’ve pinpointed several promising research directions for alleviating these challenges and advancing AI towards being a more capable software engineer.

In many ways, the lessons developed throughout this thesis were used in developing this roadmap, but more broadly can guide future benchmark research and develop analogous roadmaps in other fields. In Chapter 2, we saw how each metric revealed a different slice of AI systems and showed different insights. When it comes to benchmark research, the first step is deciding what to measure. In this roadmap, we decided on our set of tasks after looking at the skills that software engineers needed to have and compared it with the skills that today’s coding agents have. We also thought more broadly about the longer-term horizon. For example, as AI generates more code, it will be more important for there to be correctness guarantees, which motivated tasks such as software testing, program analysis, and formal verification.

In Chapter 3, we showed the importance of deeper analysis, which we used to form the set of challenges discussed in the roadmap. We analyzed both benchmarks and real-world use cases guided by the tasks in the previous section. For example, by analyzing how a model fails tasks in SWE-Bench, we were able to find that models sometimes struggled at interpreting vague specifications the way that human programmers would. By trying today’s frontier coding agents on a set of out-of-distribution tasks, we were able to see how they struggled and failed. Overall, a section like this is created through looking at the tasks in the previous section, collecting intuition about failure modes, and cleanly categorizing them into a set of modular issues and challenges.

Finally, Chapter 4 showed how evaluations can shape the direction of future research and science. In our roadmap paper, we designed this section by first examining both the

tasks and the challenges in the previous sections. Then, we looked at the literature for techniques addressing each of the challenges, and synthesized both these and our own ideas into a collection of paths forward. Many of these techniques are inspired by benchmarks: for example, CRUXEval inspired a family of work training using execution feedback. These works inspired a section in our paths forward section about augmenting code data with program information: starting with execution, but also including program instrumentation, dynamic analysis, and beyond.

Overall, in order to design a roadmap, I believe we must first think critically about where a field is headed: not just in the near-term, but rather a longer-term vision about what objectives and metrics will be important a few years out. Then, we need to think about the core challenges today that are preventing us from achieving this vision. Finally, we should reflect on the research trajectory that the field has been on, build intuition for what seems promising, and use that to design a set of paths forward. Of course, there is no way to precisely predict the future: only time and experimentation will tell which directions prevail at the end. However, roadmaps can guide a community towards a specific set of visions and goals, helping everyone better understand the state of a field.

Chapter 6

Conclusion

I began this thesis with a simple principle: an evaluation is a lens through which we view the capabilities of a model. As models progressed rapidly, so did the experiments we ran and the insights that we drew from them. The three lessons in this thesis are not independent guidelines. Together, they describe the role I believe evaluations should play in the lifecycle of the scientific development of AI systems.

First, it is important to choose the right metrics without bias, because a benchmark that measures the wrong property or contains confounding artifacts may support a misleading conclusion. Second, we must look beyond the score, because many of the most valuable scientific insights live beyond surface-level aggregate metrics. Finally, we must design evaluations with the future in mind, because what we measure shapes what we build.

This final lesson, however, is a double-edged sword. We must keep Goodhart’s law in mind: while a well-designed benchmark can steer an entire community towards a worthwhile capability, when a measure becomes a target, it ceases to be a good measure. By choosing the right measurement, accounting for confounding factors, and analyzing beyond the headline metric, we can at least begin to mitigate this problem.

It is worth reflecting on just how quickly the target has changed over the last few years. When we built CRUXEval, GPT-4 was still failing to solve a third of HumanEval, and it was an open question whether language models could reason about the code they produced at all. Just a few years later, they are now autonomously solving competitive programming problems, writing software from scratch, and reasoning about research-level mathematics. This fast progress shaped the arc of the evaluations in this thesis: going from simulating simple execution behavior to solving competitive programming problems, proving challenging inequalities, writing iterative software engineering specifications, and beyond. We continuously adapt the metrics and analysis we choose to the progress in the field.

Indeed, several of the evaluations in this thesis have already become much easier for frontier models. Today’s top models achieve near-perfect scores on evaluations like CRUXEval and have made substantial progress on LiveCodeBench. However, their influence has outlasted their original difficulty. The idea of evaluating code execution and reasoning has been carried forward by a wave of follow-up benchmarks and training methods. The idea of *live* and contamination-free benchmarks has become a standard way to measure language models as they continue to improve. A benchmark’s value lies not just in how long it resists saturation, but in the lessons and insights it leaves behind.

As models become more capable and autonomous, their inner workings only become less and less transparent. Therefore, building fresh evaluations that serve as reliable lenses into these systems will be increasingly crucial. While the benchmarks in this thesis have already begun to saturate, the discipline of building them will remain intact.

Bibliography

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [3] GitHub. GitHub Copilot. <https://github.com/features/copilot>, 2023.
- [4] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- [5] OpenAI. GPT-4 technical report, 2023. URL <https://cdn.openai.com/papers/gpt-4.pdf>.
- [6] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.
- [7] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022.
- [8] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- [9] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.

- [10] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [11] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [12] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [13] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [14] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.
- [15] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.
- [16] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*, 2023.
- [17] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*, 2024.
- [18] Jiayi Sheng, Luna Lyu, Jikai Jin, Tanglin Xia, Alex Gu, James Zou, and Pan Lu. Solving inequality proofs with large language models. *Advances in Neural Information Processing Systems*, 38, 2026.
- [19] xAI. Grok 3 Mini. <https://x.ai/news/grok-3>, 2025. xAI official announcement.
- [20] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- [21] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. DeepSeek-Coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

- [22] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks, 2026. Manuscript in preparation.
- [23] Alex Gu, Wen-Ding Li, Naman Jain, Theo X Olausson, Celine Lee, Koushik Sen, and Armando Solar-Lezama. The counterfeit conundrum: Can code language models grasp the nuances of their incorrect generations? *arXiv preprint arXiv:2402.19475*, 2024.
- [24] Theo X Olausson, Alex Gu, Benjamin Lipkin, Cedegao E Zhang, Armando Solar-Lezama, Joshua B Tenenbaum, and Roger Levy. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. *arXiv preprint arXiv:2310.15164*, 2023.
- [25] Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Luke Benson, Lucy Sun, Ekaterina Zubova, Yujie Qiao, Matthew Burtell, et al. Folio: Natural language reasoning with first-order logic. *arXiv preprint arXiv:2209.00840*, 2022.
- [26] Ruiyang Xu, Jialun Cao, Yaojie Lu, Ming Wen, Hongyu Lin, Xianpei Han, Ben He, Shing-Chi Cheung, and Le Sun. CRUXEval-X: A benchmark for multilingual code reasoning, understanding and execution. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2408.13001.
- [27] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. Reasoning runtime behavior of a program with LLM: How far are we? In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. arXiv:2403.16437.
- [28] Changshu Liu, Shizhuo Dylan Zhang, and Reyhaneh Jabbarvand. Codemind: A framework to challenge large language models for code reasoning, 2024.
- [29] Nickil Maveli, Antonio Vergari, and Shay B. Cohen. What can large language models capture about code functional equivalence? In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.
- [30] Yangruibo Ding, Jinjun Peng, Marcus J Min, Gail Kaiser, Junfeng Yang, and Baishakhi Ray. Semcoder: Training code language models with comprehensive semantics. *arXiv preprint arXiv:2406.01006*, 2024.
- [31] Junlong Li, Daya Guo, Dejian Yang, Runxin Xu, Yu Wu, and Junxian He. Codei/o: Condensing reasoning patterns via code input-output prediction. *arXiv preprint arXiv:2502.07316*, 2025.
- [32] Gallil Maimon, Ori Yoran, Felix Kreuk, Michael Hassid, Gal Cohen, Pierre Chambon, and Yossi Adi. Self-execution simulation improves coding models, 2026.
- [33] DeepSeek-AI. DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence, 2024.

- [34] Qwen Team. Qwen2.5-Coder technical report, 2024.
- [35] Qwen Team. Qwen3 technical report, 2025.
- [36] Moonshot AI. Kimi K2: Open agentic intelligence, 2025.
- [37] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Eduardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.
- [38] FAIR CodeGen Team. CWM: An open-weights LLM for research on code generation with world models, 2025.
- [39] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J Prenger, and Animashree Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 36:21573–21612, 2023.
- [40] Alex Gu, Bartosz Piotrowski, Fabian Gloeckle, Kaiyu Yang, and Aram H. Markosyan. ProofOptimizer: Training language models to simplify proofs without human demonstrations, 2026. Manuscript in preparation.
- [41] Alex Gu, Naman Jain, Wen-Ding Li, Manish Shetty, Kevin Ellis, Koushik Sen, and Armando Solar-Lezama. Position: Future research and challenges remain towards AI for software engineering, 2025. Position paper.
- [42] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [43] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- [44] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

- [45] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.
- [46] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.
- [47] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*, 2023.
- [48] Michael Royzen, Justin Wei, and Russell Coleman. Phind, 2023. URL <https://www.phind.com>.
- [49] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [50] R OpenAI. GPT-4 technical report. arXiv 2303.08774. *View in Article*, 2023.
- [51] Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, Zeyao Ma, Kashun Shum, Xuwu Wang, Jinxi Wei, Jiaxi Yang, Jiajun Zhang, Lei Zhang, Zongmeng Zhang, Wenting Zhao, and Fan Zhou. Qwen3-Coder-Next technical report, 2026.
- [52] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *Advances in Neural Information Processing Systems*, volume 34, 2021. doi:[10.48550/arXiv.2105.09938](https://arxiv.org/abs/2105.09938). URL <https://arxiv.org/abs/2105.09938>.
- [53] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- [54] Meta Platforms, Inc. Llama-4-Scout-17B-16E. <https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E>, 2025. Hugging Face model card; accessed 2025-05-12.
- [55] Qwen Team. Qwen2.5-72B. <https://huggingface.co/Qwen/Qwen2.5-72B>, 2024. Hugging Face model card.
- [56] Meta Platforms, Inc. Llama-4-Maverick-17B-128E. <https://huggingface.co/meta-llama/Llama-4-Maverick-17B-128E>, 2025. Hugging Face model card.
- [57] Qwen Team. Qwen2.5-7B. <https://huggingface.co/Qwen/Qwen2.5-7B>, 2024. Hugging Face model card.

- [58] Google DeepMind. Gemini 2.0 Flash-Lite. <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash-lite>, 2025. Vertex AI model card.
- [59] OpenAI. GPT-4o mini. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024. OpenAI blog post.
- [60] OpenAI. GPT-4.1. <https://openai.com/index/gpt-4-1/>, 2025. OpenAI model announcement.
- [61] OpenAI. GPT-4o. <https://openai.com/index/hello-gpt-4o/>, 2024. OpenAI blog post.
- [62] Google DeepMind. Gemini 2.0 Flash. <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash>, 2025. Vertex AI model card.
- [63] xAI. Grok 3. <https://x.ai/news/grok-3>, 2025. xAI official announcement.
- [64] Alibaba Qwen Team. QwQ-32B. <https://huggingface.co/Qwen/QwQ-32B>, 2025. Hugging Face model card.
- [65] DeepSeek-AI. DeepSeek-R1-Distill-Llama-70B. <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-70B>, 2025. Hugging Face model card.
- [66] DeepSeek-AI. DeepSeek-R1-Distill-Qwen-14B. <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B>, 2025. Hugging Face model card.
- [67] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [68] Qwen Team. Qwen3-235B-A22B. <https://huggingface.co/Qwen/Qwen3-235B-A22B>, 2025. Hugging Face model card.
- [69] Anthropic. Claude 3.7 Sonnet. <https://www.anthropic.com/claude/sonnet>, 2025. Anthropic model card.
- [70] Google DeepMind. Gemini 2.5 Flash. <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-flash>, 2025. Vertex AI model card.
- [71] Google DeepMind. Gemini 2.5 Pro. <https://deepmind.google/technologies/gemini/pro/>, 2025. Google DeepMind model card.
- [72] OpenAI. OpenAI o1. <https://openai.com/o1/>, 2024. OpenAI official announcement.
- [73] OpenAI. Openai o3-mini system card, January 2025. URL <https://cdn.openai.com/o3-mini-system-card-feb10.pdf>. Accessed: 2025-03-19.
- [74] OpenAI. OpenAI o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>, 2025. OpenAI official announcement.
- [75] Zhiruo Wang, Shuyan Zhou, Daniel Fried, and Graham Neubig. Execution-based evaluation for open-domain code generation. *arXiv preprint arXiv:2212.10481*, 2022.

- [76] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210*, 2023.
- [77] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [78] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [79] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024.
- [80] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.
- [81] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Demystifying GPT self-repair for code generation. *arXiv preprint arXiv:2306.09896*, 2023.
- [82] Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint arXiv:2303.11366*, 2023.
- [83] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.
- [84] Arnav Gudibande, Eric Wallace, Charlie Snell, Xinyang Geng, Hao Liu, Pieter Abbeel, Sergey Levine, and Dawn Song. The false promise of imitating proprietary LLMs. *arXiv preprint arXiv:2305.15717*, 2023.
- [85] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.
- [86] Herbert B Enderton. *A mathematical introduction to logic*. Elsevier, 2001.
- [87] David Barker-Plummer, Jon Barwise, and John Etchemendy. *Language, proof, and logic*. Center for the Study of Language and Information, 2 edition, 2011.
- [88] W. McCune. Prover9 and mace4. [<http://www.cs.unm.edu/~mccune/prover9/>], 2005–2010.

- [89] Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3621–3634, Online, August 2021. Association for Computational Linguistics.
- [90] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- [91] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. **Large Language Models are Zero-Shot Reasoners**. *arXiv preprint arXiv:2205.11916*, 2022.
- [92] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [93] Julian Aron Prenner and Romain Robbes. ThrowBench: Benchmarking LLMs by predicting runtime exceptions, 2025.
- [94] Anjiang Wei, Jiannan Cao, Ran Li, Hongyu Chen, Yuhui Zhang, Ziheng Wang, Yuan Liu, Thiago S. F. X. Teixeira, Diyi Yang, Ke Wang, and Alex Aiken. EquiBench: Benchmarking large language models’ reasoning about program semantics via equivalence checking. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. arXiv:2502.12466.
- [95] Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.14119.
- [96] Changshu Liu, Alireza Ghazanfari, Yang Chen, and Reyhaneh Jabbarvand. Evaluating code reasoning abilities of large language models under real-world settings, 2025.
- [97] Jia Li, Yuxin Su, and Michael R. Lyu. From laboratory to real-world applications: Benchmarking agentic code reasoning at the repository level. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. arXiv:2601.03731.
- [98] Claas Beger and Saikat Dutta. CoCoNUT: Structural code understanding does not fall out of a tree. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, 2025. arXiv:2501.16456.
- [99] Junlong Li, Daya Guo, Dejian Yang, Runxin Xu, Yu Wu, and Junxian He. CodeI/O: Condensing reasoning patterns via code input-output prediction. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.07316.
- [100] Jordi Armengol-Estapé, Quentin Carbonneaux, Tianjun Zhang, Aram H. Markosyan, et al. What i cannot execute, i do not understand: Training and evaluating LLMs on program execution traces, 2025.

- [101] Nuo Chen, Zehua Li, Keqin Bao, Junyang Lin, et al. Chain of execution supervision promotes general reasoning in large language models, 2025.
- [102] Sijie Wang, Quanjiang Guo, Kai Zhao, Yawei Zhang, et al. CodeBoost: Boosting code LLMs by squeezing knowledge from code snippets with RL, 2025.
- [103] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, et al. Absolute zero: Reinforced self-play reasoning with zero data, 2025.
- [104] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.
- [105] Mayank Mishra, Matt Stallone, Gaoyuan Zhang, Yikang Shen, Aditya Prasad, et al. Granite code models: A family of open foundation models for code intelligence, 2024.
- [106] Mistral AI. Codestral: Hello, world! <https://mistral.ai/news/codestral>, 2024.
- [107] 01.AI. Meet Yi-Coder: A small but mighty LLM for code. <https://huggingface.co/blog/lorinma/yi-coder>, 2024.
- [108] ByteDance Seed. Seed-Coder: Let the code model curate data for itself, 2025.
- [109] Codefuse and Ling Team. Every sample matters: Leveraging mixture-of-experts and high-quality data for efficient and accurate code LLM, 2025.
- [110] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [111] DeepSeek-AI. DeepSeek-V3 technical report, 2024.
- [112] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddhartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. LiveBench: A challenging, contamination-free LLM benchmark, 2024. Published at ICLR 2025.
- [113] Junnan Liu, Hongwei Liu, Linchen Xiao, Ziyi Wang, Kuikun Liu, Songyang Gao, Wenwei Zhang, Songyang Zhang, and Kai Chen. Are your LLMs capable of stable reasoning? *arXiv preprint arXiv:2412.13147*, 2024.
- [114] Sadegh Mahdavi et al. Leveraging online olympiad-level math problems for LLMs training and contamination-resistant evaluation, 2025.
- [115] Linyang He, Qiyao Yu, Hanze Dong, Baohao Liao, Xinxing Xu, Micah Goldblum, Jiang Bian, and Nima Mesgarani. LiveMathematicianBench: A live benchmark for mathematician-level reasoning with proof sketches, 2026.
- [116] Zihan Zheng et al. LiveCodeBench Pro: How do olympiad medalists judge LLMs in competitive programming?, 2025.

- [117] Linghao Zhang et al. SWE-bench goes live!, 2025.
- [118] Nimrod Shabtay et al. LiveXiv: A multi-modal live benchmark based on arXiv papers content, 2024. Published at ICLR 2025.
- [119] Shahriar Golchin and Mihai Surdeanu. Time travel in LLMs: Tracing data contamination in large language models. *arXiv preprint arXiv:2308.08493*, 2023.
- [120] Manley Roberts, Himanshu Thakur, Christine Herlihy, Colin White, and Samuel Dooley. To the cutoff... and beyond? a longitudinal perspective on LLM data contamination. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=m2NVG4Htxs>.
- [121] Martin Riddell, Ansong Ni, and Arman Cohan. Quantifying contamination in evaluating code generation capabilities of language models. *arXiv preprint arXiv:2403.04811*, 2024.
- [122] Alexandre Matton, Tom Sherborne, Dennis Aumiller, Elena Tommasone, Milad Alizadeh, Jingyi He, Raymond Ma, Maxime Voisin, Ellen Gilsenan-McMahon, and Matthias Gall  . On leakage of code generation evaluation datasets. *arXiv preprint arXiv:2407.07565*, 2024.
- [123] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang,

- Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [124] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
 - [125] GLM-4.5 Team. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models, 2025.
 - [126] MiniMax. MiniMax-M1: Scaling test-time compute efficiently with lightning attention, 2025.
 - [127] Mistral AI. Magistral, 2025.
 - [128] Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, et al. Llama-Nemotron: Efficient reasoning models, 2025.
 - [129] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
 - [130] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics (TACL)*, 10:291–306, 2022.
 - [131] Jesse Michael Han, Jason Rute, Yuhuai Wu, Edward Ayers, and Stanislas Polu. Proof artifact co-training for theorem proving with language models. In *International Conference on Learning Representations (ICLR)*, 2022.
 - [132] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: a cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.
 - [133] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W Ayers, Dragomir Radev, and Jeremy Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.
 - [134] Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal mathematics statement curriculum learning. In *International Conference on Learning Representations (ICLR)*, 2023.

- [135] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Towards large language models as copilots for theorem proving in Lean. *arXiv preprint arXiv:2404.12534*, 2024.
- [136] Haohan Lin, Zhiqing Sun, Sean Welleck, and Yiming Yang. Lean-STaR: Learning to interleave thinking and proving. *arXiv preprint arXiv:2407.10040*, 2024.
- [137] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37:11545–11569, 2024.
- [138] Google. AI achieves silver-medal standard solving International Mathematical Olympiad problems. <https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/>, 2024.
- [139] Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, et al. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction. *arXiv preprint arXiv:2508.03613*, 2025.
- [140] Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, et al. Seed-prover: Deep and broad reasoning for automated theorem proving, 2025. URL <https://arxiv.org/abs/2507.23726>, 2025.
- [141] The mathlib Community. The Lean mathematical library. *CoRR*, abs/1910.09336, 2019. URL <http://arxiv.org/abs/1910.09336>.
- [142] Michael Kinyon. Proof simplification and automated theorem proving. *CoRR*, abs/1808.04251, 2018. URL <http://arxiv.org/abs/1808.04251>.
- [143] Riyaz Ahuja, Jeremy Avigad, Prasad Tetali, and Sean Welleck. Improver: Agent-based automated proof optimization. *arXiv preprint arXiv:2410.04753*, 2024.
- [144] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.

- [145] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35: 15476–15488, 2022.
- [146] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [147] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective, 2025. URL <https://arxiv.org/abs/2503.20783>.
- [148] Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, et al. Goedel-prover: A frontier model for open-source automated theorem proving. *arXiv preprint arXiv:2502.07640*, 2025.
- [149] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.
- [150] Karen Zee, Viktor Kuncak, and Martin Rinard. Full functional verification of linked data structures. *ACM SIGPLAN Notices*, 43(6):349–361, 2008.
- [151] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. A few billion lines of code later: using static analysis to find bugs in the real world. *Communications of the ACM*, 53(2):66–75, 2010.
- [152] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. The astrée analyzer. In *Programming Languages and Systems: 14th European Symposium on Programming, ESOP 2005, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2005, Edinburgh, UK, April 4-8, 2005. Proceedings 14*, pages 21–30. Springer, 2005.
- [153] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, et al. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 207–220, 2009.
- [154] Xavier Leroy, Sandrine Blazy, Daniel Kästner, Bernhard Schommer, Markus Pister, and Christian Ferdinand. CompCert: A formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [155] The Coq Development Team. The Coq reference manual – release 8.19.0. <https://coq.inria.fr/doc/V8.19.0/refman>, 2024.

- [156] Leonardo De Moura, Soonho Kong, Jeremy Avigad, Floris Van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *Automated Deduction-CADE-25: 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings 25*, pages 378–388. Springer, 2015.
- [157] K Rustan M Leino. Dafny: An automatic program verifier for functional correctness. In *International conference on logic for programming artificial intelligence and reasoning*, pages 348–370. Springer, 2010.
- [158] Tobias Nipkow, Markus Wenzel, and Lawrence C Paulson. *Isabelle/HOL: a proof assistant for higher-order logic*. Springer, 2002.
- [159] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, et al. Verus: A practical foundation for systems verification. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 438–454, 2024.
- [160] Chloe Loughridge, Qinyi Sun, Seth Ahrenbach, Federico Cassano, Chuyue Sun, Ying Sheng, Anish Mudide, Md Rakib Hossain Misu, Nada Amin, and Max Tegmark. Dafny-Bench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.
- [161] Evan Lohn and Sean Welleck. minicodeprops: a minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.
- [162] Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Jianan Yao, Weidong Cui, Yeyun Gong, Chris Hawblitzel, Shuvendu Lahiri, Jacob R Lorch, Shuai Lu, et al. AutoVerus: Automated proof generation for Rust code. *arXiv preprint arXiv:2409.13082*, 2024.
- [163] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pages 835–850, 2021.
- [164] Elias Stengel-Eskin, Archiki Prasad, and Mohit Bansal. Regal: refactoring programs to discover generalizable abstractions. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [165] Matthew Bowers, Theo X. Olausson, Lionel Wong, Gabriel Grand, Joshua B. Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. Top-down synthesis for library learning. *Proc. ACM Program. Lang.*, 7(POPL), January 2023. doi:[10.1145/3571234](https://doi.org/10.1145/3571234). URL <https://doi.org/10.1145/3571234>.
- [166] Naman Jain, Manish Shetty, Tianjun Zhang, King Han, Koushik Sen, and Ion Stoica. R2E: Turning any GitHub repository into a programming agent environment. In *Forty-first International Conference on Machine Learning*, 2024.

- [167] Ian Berlot-Attwell, Frank Rudzicz, and Xujie Si. Library learning doesn't: The curious case of the single-use" library". *arXiv preprint arXiv:2410.20274*, 2024.
- [168] Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.
- [169] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M Frans Kaashoek, and Nickolai Zeldovich. Using crash hoare logic for certifying the fscq file system. In *Proceedings of the 25th Symposium on Operating Systems Principles*, pages 18–37, 2015.
- [170] Daniel J Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lesciau, Alex Ahern, et al. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618 (7964):257–263, 2023.
- [171] Saiteja Utpala, Alex Gu, and Pin Yu Chen. Language agnostic code embeddings. *arXiv preprint arXiv:2310.16803*, 2023.
- [172] Yu Zhao, Lina Gong, Haoxiang Zhang, Yaoshen Yu, and Zhiqiu Huang. How to get better embeddings with code pre-trained models? an empirical study. *arXiv preprint arXiv:2311.08066*, 2023.
- [173] Maxwell Nye, Yewen Pu, Matthew Bowers, Jacob Andreas, Joshua B Tenenbaum, and Armando Solar-Lezama. Representing partial programs with blended abstract semantics. *arXiv preprint arXiv:2012.12964*, 2020.
- [174] Amit Zohar and Lior Wolf. Automatic program synthesis of long programs with a learned garbage collector. *Advances in neural information processing systems*, 31, 2018.
- [175] Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a repl. *Advances in Neural Information Processing Systems*, 32, 2019.
- [176] Xinyun Chen, Dawn Song, and Yuandong Tian. Latent execution for neural program synthesis beyond domain-specific languages. *Advances in Neural Information Processing Systems*, 34:22196–22208, 2021.
- [177] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 238–252, 1977.
- [178] Patrice Godefroid, Nils Klarlund, and Koushik Sen. Dart: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*, pages 213–223, 2005.
- [179] Koushik Sen, Darko Marinov, and Gul Agha. Cute: A concolic unit testing engine for c. *ACM SIGSOFT Software Engineering Notes*, 30(5):263–272, 2005.

- [180] Edmund M Clarke. Model checking. In *Foundations of Software Technology and Theoretical Computer Science: 17th Conference Kharagpur, India, December 18–20, 1997 Proceedings 17*, pages 54–56. Springer, 1997.
- [181] Luca Cardelli. Type systems. *ACM Computing Surveys (CSUR)*, 28(1):263–264, 1996.
- [182] Linyuan Gong, Mostafa Elhoushi, and Alvin Cheung. Ast-t5: Structure-aware pre-training for code generation and understanding. *arXiv preprint arXiv:2401.03003*, 2024.
- [183] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=KmtVD97J43e>.
- [184] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured nlp tasks without finetuning. *arXiv preprint arXiv:2305.13971*, 2023.
- [185] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 172–184, 2023.